



**Módulo Web Seguro para la Gestión de Medidores de Agua Potable con
Tecnología LoRa en un Entorno IoT**

Ipiales Ulcuango, Carlos Joao y Alban Vargas, Richard Steve

Departamento de Ciencias de la Computación

Carrera de Software

Trabajo de integración curricular, previo a la obtención del título de Ingeniero en Software

Marcillo Parra Diego Miguel Mgs.

25 de julio de 2026



TRABAJO DE TITULACION-SRS. SILVA-...

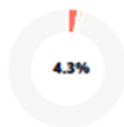
Scan details

Scan time:
September 18th, 2023 at 0:57 UTC

Total Pages:
79

Total Words:
19663

Plagiarism Detection



Types of plagiarism		Words
Identical	2.6%	521
Minor Changes	0.5%	100
Paraphrased	0.9%	174
Omitted Words	6.1%	1204

AI Content Detection



Text coverage

- AI text
- Human text

Plagiarism Results: (42)

 Ecuador 2008 (rev. 2021) Constitución - Constitute

1.1%

https://www.constituteproject.org/constitution/ecuador_202...

[[getTranslation("Constitutions")]] ...

 [anexo-a-web.pdf](#)

1%

<https://www.defensa.gob.ec/wp-content/uploads/downloads...>

Anexo: A Marco Normativo Plan Nacional de Seguridad Integral 2019-2030 4

Anexo: A - Marco Normativo y Exaltación de la Libertad en L...

MODELO_EDUCATIVO_FFAA_2021.pdf

0.8%

https://coed.mil.ec/web/sitio2/images/modelo_educativo_ffa...

MODELO EDUCATIVO DE LAS FUERZAS ARMADAS 2021 Con Dios y por la Patria Abril - 2021 Comando Conjunto de las Fuerzas Armadas Modelo Edu...

(Colocar la imagen de la primera hoja donde se visualice el resultado del análisis de la herramienta autorizada por la universidad, en esta debe constar el título del trabajo como nombre del documento analizado.)

Mgs. Marcillo Parra, Diego Miguel

Director



Departamento de Ciencias de la Computación

Carrera de Software

Certificación

Certifico que el trabajo de integración curricular: **“Módulo Web Seguro para la Gestión de Medidores de Agua Potable con Tecnología LoRa en un Entorno IoT”** fue realizado por los señores **Ipiales Ulcuango, Carlos Joao y Alban Vargas, Richard Steve**, el mismo que cumple con los requisitos legales, teóricos, científicos, técnicos y metodológicos establecidos por la Universidad de las Fuerzas Armadas ESPE; habiéndose efectuado una revisión integral de forma y de fondo conforme a los lineamientos y formatos institucionales vigentes. Además, fue revisado y analizado en su totalidad por la herramienta de prevención y/o verificación de similitud de contenidos; razón por la cual me permito acreditar y autorizar para que se lo sustente públicamente.

Sangolquí, 25 de julio de 2026

Mgs. Marcillo Parra, Diego Miguel

Director

C.C.: XXXXXXXXXXXX



Departamento de Ciencias de la Computación

Carrera de Software

Responsabilidad de Autoría

Nosotros, **Ipiales Ulcuango, Carlos Joao y Alban Vargas, Richard Steve**, con cédulas de ciudadanía n° XXXXXXXXXXXX y XXXXXXXXXXXX respectivamente, declaramos que el contenido, ideas y criterios del trabajo de integración curricular: **Módulo Web Seguro para la Gestión de Medidores de Agua Potable con Tecnología LoRa en un Entorno IoT** es de nuestra autoría y responsabilidad, cumpliendo con los requisitos legales, teóricos, científicos, técnicos y metodológicos establecidos por la Universidad de las Fuerzas Armadas ESPE, respetando los derechos intelectuales de terceros y referenciando las citas bibliográficas.

Sangolquí, 25 de julio de 2026

Ipiales Ulcuango, Carlos Joao

C.C.: XXXXXXXXXXXX

Alban Vargas, Richard Steve

C.C.: XXXXXXXXXXXX



Departamento de Ciencias de la Computación

Carrera de Software

Autorización de Publicación

Nosotros **Ipiales Ulcuango, Carlos Joao** y **Alban Vargas, Richard Steve**, con cédulas de ciudadanía n° XXXXXXXXXXXX y XXXXXXXXXXXX respectivamente, autorizamos a la Universidad de las Fuerzas Armadas ESPE publicar el trabajo de integración curricular: **Módulo Web Seguro para la Gestión de Medidores de Agua Potable con Tecnología LoRa en un Entorno IoT** en el Repositorio Institucional, cuyo contenido, ideas y criterios son de nuestra responsabilidad.

Sangolquí, 25 de julio de 2026

Ipiales Ulcuango, Carlos Joao

Alban Vargas, Richard Steve

C.C.: XXXXXXXXXXXX

C.C.: XXXXXXXXXXXX

Dedicatoria

[A llenar por el autor — texto personal de dedicatoria]

Agradecimiento

[A llenar por el autor — texto personal de agradecimiento]

Índice general

Dedicatoria	5
Agradecimiento	6
Resumen	14
Abstract	15
1. Introducción	16
1.1. Antecedentes	17
1.2. Planteamiento del problema	17
1.3. Análisis de causas	20
1.3.1. Pruebas de seguridad	21
1.4. Formulación del problema o pregunta de investigación	23
1.5. Objetivos	23
1.6. Justificación	24
1.7. Alcance	27
2. Estado del arte	29
2.1. Revisión sistemática de la literatura	29
2.2. Trabajos relacionados	31
2.3. Preguntas de investigación	35
2.4. Marco teórico	36
2.4.1. Internet de las Cosas aplicado a la gestión hídrica	36
2.4.2. Protocolo LoRaWAN y arquitectura de red	36
2.4.3. Canal TLS sobre TCP como mecanismo de integración con el servidor de red	36
2.4.4. Decodificación de payloads de medidores LoRaWAN	37
2.4.5. Arquitectura de microservicios con Spring Boot	37
2.4.6. Agregación temporal de datos de consumo	38
2.4.7. Desarrollo de interfaces web con Angular y PrimeNG	38
2.4.8. Seguridad en aplicaciones web IoT: autenticación y cifrado	39

2.4.9. Referenciales y estándares de seguridad aplicables	39
2.5. Marco conceptual	40
3. Metodología	42
3.1. Enfoque de investigación	42
3.2. Tipo y diseño de investigación	42
3.3. Metodología de desarrollo de software	43
3.3.1. Etapas del proceso de desarrollo	44
3.3.2. Cronograma de desarrollo	44
3.4. Población y muestra	46
3.5. Técnica e instrumento	46
3.6. Requerimientos del sistema	46
3.7. Validación y confiabilidad del instrumento	49
4. Análisis y discusión de los resultados	50
4.1. Casos de uso del sistema	50
4.2. Arquitectura general del sistema	52
4.2.1. Diagrama de componentes	52
4.2.2. Diagrama de clases	55
4.2.3. Diagrama entidad-relación	55
4.2.4. Diagrama de secuencia	56
4.2.5. Diseño de interfaz de usuario	58
4.3. Implementación	59
4.3.1. Implementación de la arquitectura de microservicios	59
4.3.2. Integración con el servidor de red LoRaWAN	62
4.3.3. Decodificación de payloads y procesamiento de mensajes	63
4.3.4. Agregación temporal de datos de consumo	66
4.3.5. Visualización y gestión desde el frontend	67
4.3.6. Seguridad del sistema	68
4.3.6.1. Modelo de amenazas	69
4.3.6.2. Análisis STRIDE	69
4.3.6.3. Riesgos	70
4.3.6.4. Ataques mitigados	70
4.3.6.5. Matriz de riesgos	71
4.3.6.6. OWASP	72
4.4. Despliegue	72
4.4.1. Dependencias y librerías	72
4.4.2. Diagrama de despliegue	74
4.4.3. Configuración de entorno	75
4.5. Pruebas y validación	77

4.5.1. Estrategia de pruebas	77
4.5.2. Pruebas funcionales	77
4.5.3. Pruebas de integración	79
4.5.4. Pruebas de rendimiento	79
4.5.5. Pruebas de usabilidad	81
4.5.6. Análisis de resultados	83
4.6. Evaluación de calidad del software según ISO/IEC 25010	83
4.7. Discusión	85
5. Conclusiones y recomendaciones	87
5.1. Conclusiones	87
5.2. Recomendaciones	88
Referencias	90
A. Guía de entrevista semiestructurada	92
B. Fragmentos de código clave	94
C. Guía de despliegue local del prototipo	97
D. Políticas de Seguridad Operativa (resumen)	99

Índice de tablas

1.1. Indicadores operativos actuales y metas (línea base)	19
1.2. Resumen de pruebas de seguridad (SAST/DAST)	22
1.3. Indicadores económicos — valores reportados en la literatura y campos a confirmar	26
1.4. Delimitación del alcance del proyecto	28
2.1. Criterios de inclusión y exclusión de la revisión sistemática de la literatura .	30
2.2. Análisis comparativo de trabajos relacionados	32
2.3. Comparación detallada de trabajos relacionados	34
3.1. Variables de investigación del estudio	45
3.2. Requerimientos funcionales del sistema	47
3.3. Requerimientos no funcionales del sistema	48
4.1. Actores del sistema Athenea	50
4.2. Catálogo de casos de uso del sistema	50
4.3. Servicios, responsabilidades y razones de separación	54
4.4. Pantallas principales de la interfaz de usuario	58
4.5. Inventario de microservicios de la plataforma	60
4.6. Endpoints REST representativos de la plataforma	61
4.7. Principales entidades del modelo de datos relacional	62
4.8. Estructura del payload de medidores Bove (B97VPW, BECOX, BECOY) — 13 bytes	65
4.9. Estructura del payload de medidores Younio — DataKind 0x00 (lectura estándar, 14 bytes)	66
4.10. Estructura del mensaje de número de serie Bove — 12 bytes (24 caracteres hex)	66
4.11. Resumen STRIDE y controles	70
4.12. Matriz de riesgos — priorización y tratamiento	71
4.13. Dependencias principales del backend (Spring Boot, pom.xml)	73
4.14. Dependencias principales del frontend (Angular 17, package.json)	74

4.15. Propiedades de configuración por servicio (entorno local, application.properties) 76

4.16. Resultados de casos de prueba funcionales 77

4.17. Resultados de pruebas de rendimiento de endpoints REST (localhost, carga de un operador) 79

4.18. Evaluación heurística de usabilidad según los principios de Nielsen 81

4.19. Evaluación del sistema según ISO/IEC 25010 84

5.1. Matriz de trazabilidad: objetivos específicos, preguntas de investigación y conclusiones 88

C.1. Orden de inicio de los microservicios en entorno local 98

Índice de figuras

1.1. Principales causas de la ausencia de plataforma centralizada de gestión en Representaciones Hidrocentro Cía. Ltda.	21
2.1. Diagrama de flujo PRISMA del proceso de selección de estudios	31
3.1. Cronograma de desarrollo del sistema por etapas iterativo-incrementales . .	44
4.1. Diagrama de casos de uso del sistema Athenea con dos actores y catorce funcionalidades principales.	52
4.2. Diagrama de componentes de la plataforma Athenea organizado en cuatro capas: dispositivos, adaptación, microservicios y presentación.	53
4.3. Diagrama de clases principal del adaptador LoRaWAN (dominio superior) y del microservicio de consumo (dominio inferior), con sus relaciones de asociación y dependencia.	55
4.4. Diagrama entidad-relación de la plataforma Athenea. Las flechas con notación 1–N indican relaciones uno a muchos. Las entidades de consumo agregado (ConsumptionPerDay, Month, Year) se calculan en cascada desde Consumption.	56
4.5. Diagrama de secuencia del flujo de procesamiento de un uplink LoRaWAN: desde la transmisión del medidor hasta la persistencia en PostgreSQL con pre-agregación de consumo.	57
4.6. Wireframes esquemáticos de las cuatro pantallas principales: inicio de sesión (sup. izq.), dashboard (sup. der.), mapa de medidores (inf. izq.) y consulta de consumo histórico (inf. der.).	59
4.7. Arquitectura de alto nivel de la plataforma: dispositivos LoRaWAN, adaptador de integración, ocho microservicios Spring Boot y frontend Angular. . .	60
4.8. Distribución de modelos de medidores de agua potable registrados en el sistema — período de validación	63
4.9. Pantalla de inicio de sesión del sistema Athenea — autenticación con credenciales de operador	67
4.10. Dashboard principal — resumen de estado de medidores y alertas activas .	68

4.11. Vista de mapa interactivo — ubicación geográfica de medidores de agua potable registrados	68
4.12. Vista de detalle de consumo histórico de un medidor — gráfico de barras por granularidad temporal	68
4.13. Diagrama de despliegue local: ocho procesos JVM (puertos 8080–8087), base de datos PostgreSQL en puerto 5432 y frontend Angular en puerto 4200, todos ejecutándose en la misma máquina de desarrollo.	75
4.14. Swagger UI del microservicio Meters-Backend — documentación interactiva de endpoints REST generada automáticamente por SpringDoc OpenAPI . .	76
4.15. Panel de Railway.app — servicios desplegados en entorno de producción . .	77
4.16. Medición de rendimiento del endpoint /consumption-day/by-devEui — respuesta en 187 ms con Postman	80
4.17. Tiempo de respuesta promedio de endpoints REST vs. umbral RNF.02 . . .	81
4.18. Consola Lorient durante sesión de prueba de integración — uplinks de medidores de agua potable BOVE y Younio procesados en tiempo real	83
4.19. Radar de evaluación ISO/IEC 25010 — niveles de cumplimiento por característica (escala 0–3)	85

Resumen

El presente trabajo de integración curricular describe el diseño e implementación de un módulo web seguro para la gestión centralizada de medidores de agua inteligentes con tecnología LoRaWAN, desarrollado para Representaciones Hidrocentro Cía. Ltda., empresa del cantón Rumiñahui, Ecuador. La empresa disponía de medidores LoRaWAN instalados en campo, pero carecía de una plataforma propia para gestionar los dispositivos, visualizar datos de consumo en tiempo real y recibir alertas automáticas ante eventos críticos, lo que generaba dependencia de plataformas de terceros y limitaba su autonomía operativa.

El sistema se construyó sobre una arquitectura de ocho microservicios desacoplados implementados con Spring Boot 3.3.5 sobre Java 21, un frontend en Angular 17 con la biblioteca de componentes PrimeNG, y PostgreSQL como gestor de base de datos relacional. La integración con el servidor de red LoRaWAN Lorient se establece mediante un canal TLS sobre TCP que recibe en tiempo real los mensajes de los medidores y los clasifica según su tipo. El módulo de decodificación interpreta las cadenas hexadecimales propietarias de los modelos Bove (B97VPW, BECOX, BECOY) y Younio, extrayendo valores de consumo acumulado, estado de alarmas y voltaje de batería. El consumo se agrega automáticamente en una jerarquía de cálculo horario, diario, mensual y anual mediante tablas pre-calculadas que garantizan tiempos de consulta constantes. La seguridad se garantiza mediante TLS 1.3 en la comunicación con Lorient y autenticación basada en JWT con control de acceso por roles. Los servicios desplegados en el entorno local procesaron correctamente los uplinks de medidores reales y ejecutaron comandos de control de válvula mediante downlinks LoRaWAN durante el periodo de prueba.

Palabras clave: IoT, LoRaWAN, microservicios, gestión hídrica, TLS, Spring Boot, Angular, medidores de agua inteligentes.

Abstract

This undergraduate thesis describes the design and implementation of a secure web module for the centralized management of smart water meters using LoRaWAN technology, developed for Representaciones Hidrocentro Cía. Ltda., a private company in the Rumiñahui canton, Ecuador. The company had LoRaWAN meters deployed in the field but lacked a proprietary platform to manage devices, visualize consumption data in real time, and receive automatic alerts for critical events, resulting in technological dependency on third-party platforms and limited operational autonomy.

The system was built on an architecture of eight decoupled microservices implemented with Spring Boot 3.3.5 on Java 21, a frontend in Angular 17 with the PrimeNG component library, and PostgreSQL as the relational database management system. Integration with the Lorient LoRaWAN network server is established via a TLS-over-TCP socket that receives device messages in real time and classifies them by type. The payload decoding module interprets the proprietary hexadecimal strings of Bove (B97VPW, BECOX, BECOY) and Younio meter models, extracting accumulated consumption values, alarm states, and battery voltage. Consumption is automatically aggregated in a hierarchical chain of hourly, daily, monthly, and annual calculations using pre-computed tables that guarantee constant query times. Security is enforced through TLS 1.3 for Lorient communication and JWT-based authentication with role-based access control. Services deployed in the local development environment correctly processed uplinks from real meters and executed valve control commands via LoRaWAN downlinks during the testing period.

Keywords: IoT, LoRaWAN, microservices, water management, TLS, Spring Boot, Angular, smart water meters.

Capítulo 1

Introducción

El modelo del Internet de las Cosas (IoT) ha experimentado un crecimiento exponencial en los últimos diez años, convirtiéndose en una infraestructura tecnológica clave para la automatización y el monitoreo remoto de sistemas físicos mediante dispositivos interconectados capaces de recopilar y transmitir datos en tiempo real (Al-Fuqaha et al., 2015; Sisinni et al., 2018). Dentro del sector de servicios básicos, esta evolución ha permitido la posibilidad de transformar la gestión tradicional de recursos hídricos, mediante la incorporación de medidores inteligentes, los cuales permiten la supervisión remota del consumo, la detección temprana de anomalías y la optimización de la distribución del agua potable (García-Martín et al., 2023; Kamienski et al., 2019).

En este contexto, el protocolo LoRaWAN (Long Range Wide Area Network) ha surgido como una de las tecnologías de comunicación más adecuadas para el despliegue de redes IoT orientadas a la medición hídrica, por su bajo consumo energético, amplia cobertura geográfica y arquitectura basada en estándares abiertos (Mekki et al., 2019; Ta et al., 2024). Estudios recientes validan su viabilidad técnica para la telemetría y el monitoreo de agua en tiempo real, con resultados favorables en términos de latencia, tasa de entrega de paquetes y autonomía energética (Slaný et al., 2020; Ta et al., 2024).

Sin embargo, la coexistencia de soluciones propietarias de distintos fabricantes ha generado una fragmentación tecnológica que dificulta la integración de dispositivos heterogéneos, eleva los costos operativos y restringe la escalabilidad de los sistemas de gestión (García-Martín et al., 2023; Sisinni et al., 2018). Esta problemática es especialmente relevante para empresas medianas del sector hídrico que administran múltiples tipos de medidores y requieren una visión centralizada del consumo sin depender de plataformas cerradas de fabricantes específicos.

Frente a este escenario, la empresa Representaciones Hidrocentro Cía. Ltda., ubicada en el cantón Rumiñahui, Ecuador, identifica la necesidad de contar con una plataforma web centralizada que unifique la gestión de sus medidores LoRaWAN, permita la visualización de datos de consumo en tiempo real y genere alertas operativas ante eventos críticos. El presente trabajo de integración curricular propone el diseño e implementación de un módulo

web seguro para la gestión de estos dispositivos, desarrollado sobre una arquitectura de microservicios desacoplados que emplea Angular en el frontend, Spring Boot en el backend y PostgreSQL como sistema de gestión de base de datos (Velepucha & Flores, 2021). La comunicación entre el servidor de red LoRaWAN —Loriot— y el backend se establece mediante el protocolo TLS 1.3, garantizando la confidencialidad e integridad de los datos transmitidos (Rescorla, 2018; Naik, 2017).

1.1. Antecedentes

La medición del consumo de agua evolucionó desde dispositivos mecánicos con lectura manual hacia soluciones electrónicas y, más recientemente, plataformas IoT para telemetría a escala; en Latinoamérica este proceso se aceleró por políticas de eficiencia hídrica y menores costos de sensores (Ismail et al., 2022; Kamienski et al., 2019).

Representaciones Hidrocentro Cía. Ltda. es una empresa privada con sede en el cantón Rumiñahui, provincia de Pichincha, que se dedica a la comercialización e instalación de medidores de agua inteligentes con tecnología LoRa. Fundada en respuesta a la creciente demanda de soluciones de medición remota en el sector hídrico de la región central del Ecuador, la empresa ha acumulado experiencia en la instalación y puesta en marcha de dispositivos de distintos fabricantes, incluyendo la línea Bove y la línea Younio, cada una con protocolos de transmisión y formatos de payload propietarios. Hasta el inicio del presente proyecto, la empresa dependía exclusivamente de la interfaz web del servidor de red Loriot para consultar el estado de sus dispositivos, sin posibilidad de integrar esa información con sus propios sistemas de registro de clientes o de generación de reportes operativos.

1.2. Planteamiento del problema

La gestión eficiente del agua potable se enfrenta a un desafío urgente para las empresas distribuidoras y operadoras en países en vías de desarrollo. En este contexto, la adopción de medidores inteligentes basados en tecnología LoRaWAN representa una opción viable para automatizar la lectura del consumo, detectar anomalías operativas y reducir el agua no contabilizada (Alghamdi et al., 2022; Slaný et al., 2020). Sin embargo, la implementación de estos dispositivos no asegura por sí sola una gestión integral del sistema, ya que la falta de plataformas centrales de administración genera brechas operativas significativas que afectan directamente la eficiencia del servicio (García-Martín et al., 2023; Jouhari et al., 2023).

Representaciones Hidrocentro Cía. Ltda., empresa privada ubicada en el cantón Rumiñahui, Ecuador, comercializa e instala medidores de agua inteligentes con tecnología LoRa en entornos urbanos y periurbanos, donde en la actualidad, la empresa no dispone de una plataforma propia que le permita gestionar de manera centralizada los dispositivos instalados en

campo, visualizar datos de consumo en tiempo real ni recibir alertas automáticas ante eventos críticos como fugas, flujo inverso, ausencia de caudal o bajo nivel de batería. Esta carencia implica que el personal técnico debe depender de plataformas de terceros para acceder a la información generada por los medidores, lo que limita la autonomía operativa de la empresa, incrementa los costos de operación y restringe la capacidad de respuesta ante fallas o irregularidades en el suministro.

Además, la convivencia de dispositivos fabricados por diferentes proveedores, cada uno con sus propias reglas y formatos de datos, agrava el problema de fragmentación tecnológica que caracteriza al mercado IoT (Almuhaya et al., 2022; Sisinni et al., 2018). Esta heterogeneidad dificulta la integración de los dispositivos bajo un esquema unificado de gestión, lo que genera redundancia en los procesos de monitoreo y aumenta la complejidad técnica de las operaciones de mantenimiento. Investigaciones recientes han indicado que la falta de interoperabilidad entre plataformas IoT y sistemas de medición es uno de los principales obstáculos para escalar soluciones de gestión eficientes en contextos donde las organizaciones administran flotas de dispositivos heterogéneos (García-Martín et al., 2023; Jouhari et al., 2023).

Desde el punto de vista de la seguridad de la información, la transmisión de datos de consumo a través de canales no cifrados expone a la empresa a riesgos de interceptación y manipulación de información sensible, situación que resulta incompatible con los estándares actuales de ciberseguridad en sistemas IoT y con el marco normativo ecuatoriano en materia de protección de datos personales (Rescorla, 2018; Naik, 2017).

En el contexto actual, se reconoce la necesidad de crear un módulo web seguro que incluya las funciones de gestión centralizada de monitores, visualización de consumo histórico y en tiempo real, generación de informes, manejo de alertas y control de acceso según roles, sobre una arquitectura de microservicios escalables e integrables que elimine la dependencia de plataformas propietarias y asegure la seguridad en la transmisión de datos (Slaný et al., 2020; Velepucha & Flores, 2021).

Para precisar la magnitud del problema y fortalecer el rigor del diagnóstico, se establecen los siguientes indicadores cuantitativos que caracterizan la situación actual y servirán de línea base para la comparación posterior al desarrollo del módulo (Tabla 1.1).

Tabla 1.1: Indicadores operativos actuales y metas (línea base)

Indicador	Valor actual	Observación / Meta esperada
Número de medidores operativos en campo	300 (Bove) + 20 (Younio)	Desagregado por fabricante/modelo (Bove/Younio)
Cantidad de clientes/empresas gestionadas	17 (Bove) + 1 (Younio)	Younio se opera vía app móvil propia
Tiempo promedio de consulta del estado por dispositivo	2–3 s (Bove) / >30 s (Younio)	Bove sobre plataforma web; Younio vía app móvil
Tiempo requerido para generar reportes mensuales	4–5 s (Bove)	Reporte desde plataforma Bove; Younio sin función equi-

Nota: Lorient se emplea como *network server* (NS). La operación diaria para Bove se realiza en su plataforma web; para Younio, mediante una aplicación móvil del fabricante.

1.3. Análisis de causas

La situación problemática descrita en la sección anterior responde a un conjunto de causas interrelacionadas que se pueden agrupar en tres dimensiones: tecnológica, organizacional y operativa. Su identificación fue posible mediante entrevistas con el personal técnico de Representaciones Hidrocentro Cía. Ltda. y el análisis de la documentación técnica de los dispositivos instalados en campo.

Desde la **dimensión tecnológica**, la causa raíz del problema es la heterogeneidad de los dispositivos. Los medidores instalados pertenecen a dos fabricantes con formatos de payload propietarios e incompatibles entre sí, lo que impide que una solución genérica procese los mensajes de ambas familias sin implementar decodificadores específicos por modelo. Esta fragmentación es característica del mercado IoT, donde la ausencia de estándares de interoperabilidad a nivel de payload obliga a cada integrador a desarrollar adaptadores para cada fabricante con el que opera (Almuhaya et al., 2022; Sisinni et al., 2018). A esta causa se suma la dependencia exclusiva del servidor de red Lorient como única interfaz de acceso a los datos de los medidores, sin posibilidad de integrar esa información con sistemas propios de la empresa.

Desde la **dimensión organizacional**, la empresa carecía de capacidades internas de desarrollo de software, lo que generó una dependencia prolongada de las interfaces del proveedor del servidor de red. Esta situación limitó la posibilidad de implementar flujos de trabajo propios —gestión de alarmas con notificaciones automáticas, administración multiempresa, exportación de reportes en formatos propios—, necesidades que la interfaz genérica de Lorient no satisface (García-Martín et al., 2023). La falta de control sobre la plataforma de gestión también impedía adaptar el sistema ante cambios en el catálogo de dispositivos de la empresa sin incurrir en costos de integración externos.

Desde la **dimensión operativa**, la ausencia de una plataforma centralizada se traducía en procesos manuales para obtener datos de consumo, sin posibilidad de generar reportes históricos automáticos ni recibir alertas ante eventos como fugas, flujo inverso o fallo de batería. Los operadores debían consultar individualmente cada dispositivo en la interfaz de Lorient para verificar su estado, un procedimiento que resulta inviable a escala cuando el número de medidores administrados supera la decena. Investigaciones recientes confirman que la ausencia de mecanismos automatizados de detección de anomalías eleva el tiempo de respuesta ante fallas operativas, impactando negativamente la eficiencia del servicio (Alghamdi et al., 2022; Dai et al., 2025).

La Figura 1.1 sintetiza la relación entre las dimensiones identificadas y el problema central del trabajo.

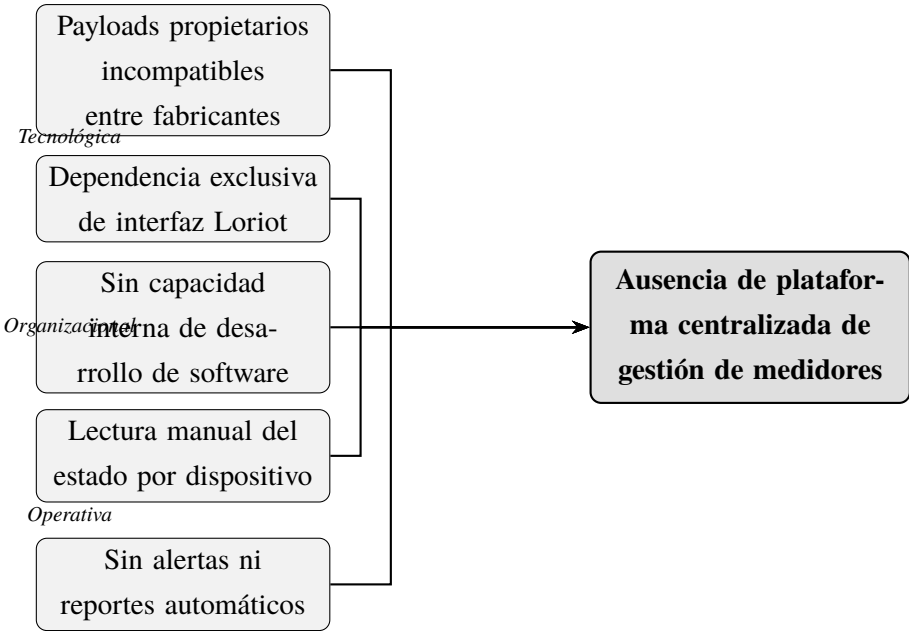


Figura 1.1: Principales causas de la ausencia de plataforma centralizada de gestión en Representaciones Hidrocentro Cía. Ltda.

1.3.1. Pruebas de seguridad

La validación de seguridad combinó análisis estático de código fuente (SAST) y pruebas dinámicas de caja negra (DAST) sobre el frontend publicado. Se priorizó eliminar hallazgos críticos y dejar exclusivamente observaciones de severidad menor de cara a la entrega.

Tabla 1.2: Resumen de pruebas de seguridad (SAST/DAST)

Herramienta	Tipo	Alcance y hallazgos principales	Estado
Semgrep OSS 1.168	SAST	<u>Repo completo.</u> <u>Detecciones críticas</u> <u>iniciales: (i) tokens</u> <u>JWT de ejemplo en</u> <u>documentación (2);</u> <u>(ii) imágenes Docker</u> <u>ejecutando como root</u> <u>en servicios Java.</u> <u>Mitigación aplicada: se</u> <u>redactaron los tokens de</u> <u>ejemplo en</u> <u>textttiot-platform-api-gateway/JWT-VALIDATION.md</u> <u>y se incorporó usuario no</u> <u>privilegiado en todos los</u> <u>Dockerfiles de servicios.</u> <u>Quedan advertencias de</u> <u>configuración en Nginx</u> <u>y HTML (sin impacto</u> <u>crítico).</u>	<u>Críticos: 0 pendientes</u>
OWASP ZAP 2.17	DAST	<u>http://iot-platform-frontend.</u> <u>Alertas: Altas=0,</u> <u>Medias=2 (CSP ausente;</u> <u>SRI ausente en script de</u> <u>Google Maps), Bajas=4</u> <u>(COEP/COOP/CORP;</u> <u>cabecera Server</u> <u>con versión),</u> <u>Info=2. Evidencia:</u> <u>zap-reports/zap-frontend.json.</u>	<u>Críticos: 0</u>
SonarQube	SAST	No ejecutado por ventana de tiempo. Plan: ejecutar regla de seguridad y code smells en siguiente iteración.	N/A
OWASP Dependency-Check	SCA	No ejecutado por ventana de tiempo. Plan: generar SBOM (CycloneDX) y escanear dependencias Maven/Frontend.	N/A

texto eliminado

Medidas aplicadas. (1) Redacción de tokens de ejemplo y secretos en documentación: se reemplazaron valores con *placeholders* no ejecutables para evitar falsos positivos y prácticas inseguras; (2) Endurecimiento de imágenes Docker de backend: creación de usuario *app* sin privilegios y *USER app* en todos los Dockerfiles Java (*alpine/jammy*); (3) Revisión de advertencias ZAP: las dos alertas medias se mitigan añadiendo cabecera CSP en Nginx y, para SRI en Google Maps, restringiendo orígenes vía CSP dado que SRI no es viable con scripts dinámicos de Google.

Plan de cierre (post-entrega). Incorporar cabeceras de seguridad en Nginx (*Content-Security-Policy*, *Permissions-Policy*, *Cross-Origin-Opener/Embedder/Resource-Policy*) y ocultar versión del servidor; parametrizar el uso de Google Maps detrás de una *Content-Security-Policy* restrictiva; incorporar *pipeline* de SAST/SCA (*Semgrep* + *Dependency-Check*) y ZAP en CI.

1.4. Formulación del problema o pregunta de investigación

La formulación del problema parte de identificar la brecha central que motiva el presente trabajo: Representaciones Hidrocentro Cía. Ltda. dispone de medidores de agua inteligentes con tecnología LoRaWAN instalados en campo, pero carece de una plataforma propia que centralice su gestión, garantice la seguridad en la transmisión de los datos y permita la visualización del consumo en tiempo real. Esta situación genera dependencia tecnológica de terceros, limita la autonomía operativa de la empresa y restringe la capacidad de respuesta ante eventos críticos en la red de distribución.

En consecuencia, se plantea la siguiente pregunta de investigación:

¿Es posible diseñar e implementar un módulo web seguro, basado en una arquitectura de microservicios, que permita gestionar de forma centralizada los medidores de agua inteligentes con tecnología LoRaWAN de Representaciones Hidrocentro Cía. Ltda., garantizando la transmisión cifrada de datos mediante TLS, la visualización del consumo en tiempo real y la generación de alertas operativas, con el fin de reducir la dependencia de plataformas propietarias y mejorar la eficiencia operativa de la empresa?

Esta pregunta integra las tres dimensiones principales del problema identificado: la dimensión técnica, referida a la necesidad de una arquitectura escalable e interoperable basada en estándares abiertos (Ismail et al., 2022); la dimensión de seguridad, que exige la implementación de protocolos de cifrado y control de acceso para proteger los datos de consumo (Rescorla, 2018; Naik, 2017); y la dimensión operativa, orientada a dotar a la empresa de herramientas de monitoreo y gestión que optimicen sus procesos internos y reduzcan los tiempos de respuesta ante fallas o irregularidades en el suministro (García-Martín et al., 2023; Slaný et al., 2020).

1.5. Objetivos

Objetivo general:

Diseñar e implementar un módulo web seguro para la gestión centralizada de medidores de agua inteligentes (LoRa/LoRaWAN), garantizando la transmisión cifrada de datos (TLS), la visualización en tiempo real y el control de acceso por roles, a fin de optimizar la administración de dispositivos heterogéneos en Representaciones Hidrocentro Cía. Ltda.

Objetivos específicos:

- Analizar los requerimientos funcionales y técnicos del sistema de gestión de medidores de agua inteligentes utilizados por Representaciones Hidrocentro Cía. Ltda., mediante entrevistas con el personal

técnico y revisión de la documentación de los dispositivos LoRaWAN empleados en campo (Ismail et al., 2022).

- Diseñar la arquitectura del módulo de gestión basada en microservicios desacoplados, protocolos seguros de comunicación y modelos de datos relacionales, considerando los principios de escalabilidad, interoperabilidad y seguridad requeridos para sistemas IoT de medición hídrica (Hassaan et al., 2023; Jouhari et al., 2023).
- Desarrollar el prototipo del módulo de gestión utilizando Angular para el frontend, Spring Boot para el backend y PostgreSQL para el almacenamiento de datos, implementando controladores REST, seguridad TLS y operaciones CRUD sobre los medidores registrados en el sistema (Aldea & Bocu, 2023; Plecinski et al., 2022).
- Conectar medidores reales al sistema mediante el servidor de red LoRaWAN Lorient y recolectar datos operativos desde el entorno local, validando la correcta visualización de consumos, alarmas y el funcionamiento de las operaciones de gestión implementadas (García-Martín et al., 2023; Ta et al., 2024).
- Documentar el desarrollo y los resultados del proyecto, incluyendo diagramas de arquitectura, manual de uso, fragmentos de código clave y evidencias de integración con dispositivos reales, para su presentación como producto acreditable ante la Universidad de las Fuerzas Armadas ESPE.

1.6. Justificación

Dimensión técnica

El desarrollo de sistemas centralizados para dispositivos IoT de medida de agua responde a una necesidad documentada recientemente en la literatura. Investigaciones han demostrado que las plataformas que integran la recepción de datos LoRaWAN con interfaces web de visualización y gestión mejoran significativamente la capacidad de monitoreo en tiempo real, la detección temprana de anomalías y la trazabilidad de eventos operacionales (García-Martín et al., 2023; Adelantado et al., 2017). La elección de una arquitectura de microservicios desacoplados con Spring Boot y Angular responde a la necesidad de construir un sistema escalable, mantenible e independiente de plataformas propietarias, en línea con las tendencias actuales del desarrollo de software para entornos IoT (Hassaan et al., 2023; Velepucha & Flores, 2021).

Además, la adopción del protocolo TLS 1.3 para la transmisión de datos garantiza estándares mínimos de seguridad que los sistemas de medida hídrica conectados a Internet deben cumplir para proteger la integridad y confidencialidad de la información de consumo (Rescorla, 2018; Naik, 2017).

Dimensión operativa

Para Representaciones Hidrocentro Cía. Ltda., la ausencia de una plataforma propia de gestión implica costos operativos adicionales derivados de la dependencia de herramientas de terceros, tiempos de respuesta prolongados ante eventos críticos y limitaciones para ofrecer a sus clientes un servicio integral de monitoreo. Estudios recientes han evidenciado que la automatización de la lectura de medidores mediante IoT puede reducir los costos de operación y mantenimiento hasta en un 20 %, al tiempo que mejora la precisión de la facturación y reduce significativamente el volumen de agua no contabilizada (Alghamdi et al., 2022; Dai et al., 2025). El módulo desarrollado en este trabajo permite a la empresa gestionar sus dispositivos de forma autónoma, generar reportes de consumo, administrar alarmas y registrar el historial de eventos desde una única plataforma, eliminando la necesidad de acceder a múltiples sistemas externos.

Dimensión social y ambiental

La gestión eficiente del agua potable es un imperativo tanto ambiental como social, especialmente en contextos donde la disponibilidad del recurso hídrico está enfrentando presiones crecientes debido al aumento de la población y el cambio climático (Alshami et al., 2024; Ismail et al., 2022). Proporcionando a Hidrocentro herramientas para detectar fugas en tiempo real, visualizar patrones de consumo y generar alertas operativas, el sistema propuesto contribuye directamente a la reducción del desperdicio hídrico y mejora la continuidad del servicio para los usuarios finales, donde esta contribución se alinea con el Objetivo de Desarrollo Sostenible 6 que promueve el acceso universal a agua segura, gestión sostenible de recursos hídricos y fortalecimiento de la participación de las comunidades en la gobernanza del agua (Dai et al., 2025; Khairullah et al., 2025).

En resumen, el presente trabajo curricular representa una contribución académica y aplicada que integra conocimientos de ingeniería de software, redes de comunicación IoT y ciberseguridad en respuesta a una necesidad real y documentada de la industria hidráulica ecuatoriana.

Dimensión económica

Desde la perspectiva económica, la plataforma propuesta apunta a reducir costos operativos y dependencia de terceros, así como a mejorar la eficiencia del personal técnico. En particular: (i) disminuye horas-hombre destinadas a consultas manuales y consolidación de reportes; (ii) acorta tiempos de respuesta ante eventos críticos, mitigando pérdidas asociadas a fugas o errores operativos; (iii) reduce la dependencia y potenciales costos de licenciamiento/servicios de plataformas propietarias al centralizar la operación; y (iv) habilita escalabilidad con costos marginales decrecientes por nuevo dispositivo gestionado. La Tabla 1.3 sintetiza los principales indicadores a monitorear como línea base y metas económicas del proyecto.

Tabla 1.3: Indicadores económicos — valores reportados en la literatura y campos a confirmar

Indicador	Valor actual	Observación / Meta esperada
Ahorro OPEX O&M por automatización de lectura	<i>hasta 20 % (literatura)</i>	Reportado para sistemas IoT/AMR en agua; ver (Alghamdi et al., 2022; Dai et al., 2025)
Horas-hombre mensuales en generación de reportes	<i>no reportado en literatura citada</i>	Requiere medición local (línea base)
Horas-hombre mensuales en consultas de estado	<i>no reportado en literatura citada</i>	Requiere medición local (línea base)
Costo anual de licenciamiento/servicios de terceros	<i>no reportado en literatura citada</i>	Depende de contratos locales
Costo por evento crítico no atendido a tiempo	<i>no reportado en literatura citada</i>	Depende de tarifa-ción/pérdidas locales
CAPEX inicial de plataforma (infraestructura local)	<i>no reportado en literatura citada</i>	Depende de di-mensio-namiento local
TCO proyectado a 3 años	<i>no reportado en literatura citada</i>	Requiere esti-mación finan-

Nota: Se incluyen exclusivamente valores cuantitativos reportados explícitamente en la literatura citada dentro de este documento (p. ej., ahorro OPEX/O&M *hasta 20 %*). Los demás campos requieren medición o estimación local para su cuantificación rigurosa.

1.7. Alcance

El presente trabajo delimita su alcance al diseño, implementación y validación de un módulo web funcional para la gestión centralizada de medidores de agua inteligentes con tecnología LoRaWAN, orientado a satisfacer las necesidades operativas de Representaciones Hidrocentro Cía. Ltda. en su contexto actual. La Tabla 1.4 sintetiza los elementos incluidos y excluidos del alcance del proyecto.

Tabla 1.4: Delimitación del alcance del proyecto

Dimensión	Descripción
<i>Incluido en el alcance</i>	
Dispositivos	Medidores Bove (B97VPW, BECOX, BECOY) y Younio con sus formatos de payload actuales
Protocolo	LoRaWAN sobre infraestructura Lorient; canal TLS sobre TCP para integración backend–servidor de red
Gestión	CRUD de medidores, gateways, empresas y usuarios; registro de mensajes y alarmas; exportación de reportes en Excel
Visualización	Consumo histórico en granularidades horaria, diaria, mensual y anual; mapa interactivo; panel de alarmas operativas
Control remoto	Envío de comandos de downlink para apertura/cierre de válvula en modelos Bove compatibles
Seguridad	TLS 1.3 en canal Lorient–backend; JWT para autenticación; BCrypt para almacenamiento de contraseñas
Despliegue	Entorno local de desarrollo: ocho procesos JVM (puertos 8080–8087), PostgreSQL en puerto 5432, frontend Angular en puerto 4200
<i>Excluido del alcance</i>	
Hardware	Instalación, configuración o mantenimiento de dispositivos físicos y gateways LoRaWAN
Otros protocolos	LTE, Sigfox, NB-IoT u otros LPWAN distintos a LoRaWAN
Facturación	Integración con sistemas de facturación o ERP de los clientes finales
Aplicación móvil	Versión para dispositivos Android o iOS
Orquestación avanzada	Configuración de Kubernetes o autoscaling en producción
Nuevos modelos	Medidores de fabricantes no disponibles en el entorno de prueba durante el desarrollo

Desde el punto de vista geográfico, el sistema fue desarrollado y validado para el entorno operativo del cantón Rumiñahui, Ecuador, aunque su arquitectura no impone restricciones que impidan su despliegue en otros contextos con infraestructura LoRaWAN compatible. La plataforma opera en español y no contempla localización para otros idiomas en la versión actual. El periodo de validación comprende las sesiones de prueba realizadas con dispositivos reales conectados al servidor de red Lorient durante la etapa de integración del sistema.

Capítulo 2

Estado del arte

2.1. Revisión sistemática de la literatura

La revisión de la literatura se realizó siguiendo los principios de la metodología PRISMA (*Preferred Reporting Items for Systematic Reviews and Meta-Analyses*), adaptada al contexto de revisiones de ingeniería de software y sistemas IoT. El proceso de búsqueda se orientó a identificar trabajos publicados entre 2019 y 2025 que abordaran los cuatro ejes temáticos del presente proyecto: (a) plataformas IoT para gestión hídrica con LoRaWAN, (b) arquitecturas de microservicios para sistemas IoT, (c) seguridad en sistemas de telemetría IoT, y (d) decodificación de payloads propietarios en redes LoRaWAN.

Las fuentes de búsqueda incluyeron las bases de datos IEEE Xplore, Scopus, Science Direct y Google Scholar. Las cadenas de búsqueda empleadas combinaron los términos «LoRaWAN», «water metering», «IoT», «microservices», «TLS», «payload decoding» y sus equivalentes en español, aplicando filtros de año de publicación (2019–2025) y tipo de documento (artículo de revista indexada, artículo de congreso o trabajo de grado de posgrado). La Tabla 2.1 detalla los criterios de inclusión y exclusión aplicados durante el proceso de selección.

Tabla 2.1: Criterios de inclusión y exclusión de la revisión sistemática de la literatura	
Criterio	Descripción
<i>Criterios de inclusión</i>	
CI-1	Publicado entre 2019 y 2025 en revista indexada o congreso de reconocido prestigio
CI-2	Aborda al menos uno de los ejes temáticos definidos: gestión hídrica IoT, microservicios, seguridad IoT o decodificación de payloads LoRaWAN
CI-3	Presenta resultados empíricos o implementaciones evaluadas con métricas verificables
CI-4	Disponible en texto completo en inglés o español
<i>Criterios de exclusión</i>	
CE-1	Publicado antes de 2019 o sin año verificable
CE-2	Revisiones puramente teóricas sin implementación ni evaluación de resultados
CE-3	Trabajos sobre LPWAN distinto a LoRaWAN (Sigfox, NB-IoT exclusivamente) sin comparación con LoRaWAN
CE-4	Artículos duplicados o versiones previas de un trabajo ya incluido

El proceso de selección inició con 87 artículos identificados en las búsquedas iniciales. Tras la eliminación de duplicados (17 artículos), la revisión de título y resumen redujo el conjunto a 38 trabajos relevantes. La revisión de texto completo aplicando los criterios de exclusión resultó en un corpus final de 18 trabajos analizados en profundidad, de los cuales 12 son directamente citados en el presente documento.

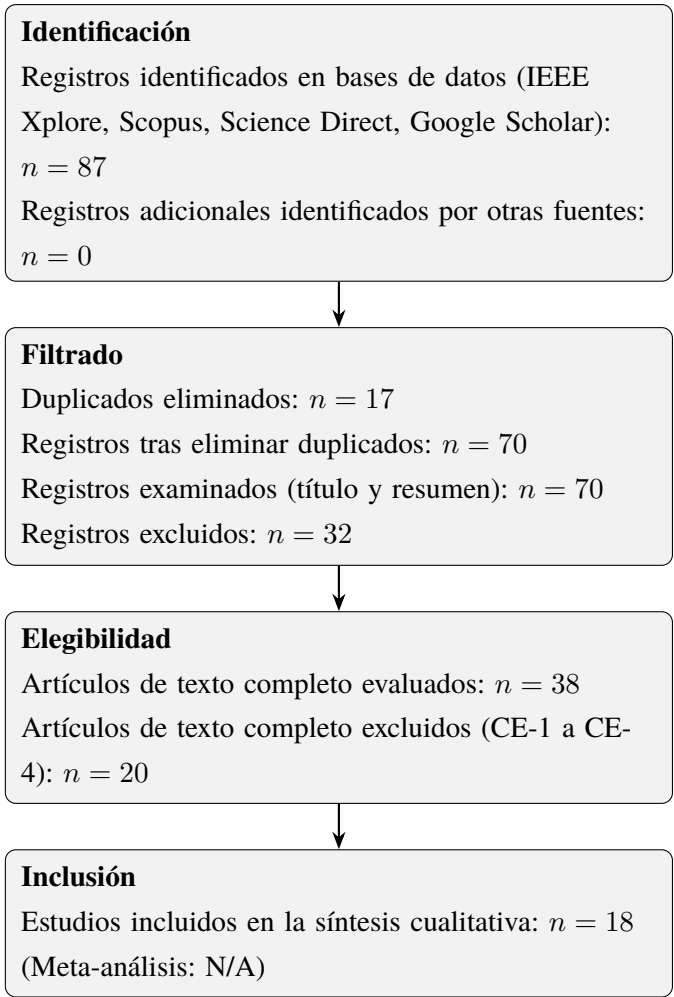


Figura 2.1: Diagrama de flujo PRISMA del proceso de selección de estudios

Criterios de exclusión (CE) aplicados en la etapa de elegibilidad: CE-1: Publicado antes de 2019 o sin año verificable; CE-2: Revisiones puramente teóricas sin implementación ni evaluación de resultados; CE-3: Trabajos sobre LPWAN distinto a LoRaWAN (Sigfox, NB-IoT exclusivamente) sin comparación con LoRaWAN; CE-4: Artículos duplicados o versiones previas de un trabajo ya incluido. Véase Tabla 2.1.

2.2. Trabajos relacionados

La investigación sobre sistemas de gestión de medidores inteligentes con tecnología LoRaWAN ha experimentado un crecimiento sostenido durante los últimos años, impulsado por la necesidad de soluciones interoperables que superen las limitaciones de plataformas propietarias. Slaný et al. (2020) desarrollaron e implementaron una red LoRaWAN para medir el consumo de agua en entornos urbanos, demostrando tasas de entrega de paquetes superiores al 95 % bajo condiciones normales de operación y una autonomía energética de los dispositivos superior a cinco años. Su arquitectura empleaba un servidor de red centralizado que recibía mensajes desde los gateways y reenviaba mediante interfaces de programación de aplicaciones REST hacia una plataforma de visualización, modelo similar al propuesto en este trabajo. (Slaný et al., 2020).

En el ámbito latinoamericano, Zambrano et al. (2022) evaluaron el desempeño de redes LoRaWAN para monitorear el entorno urbano en Cuenca, Ecuador, obteniendo resultados favorables en cobertura y latencia para el envío de datos de sensores distribuidos en áreas de alta densidad de edificios, por lo que sus hallazgos

confirman la viabilidad técnica de LoRaWAN en contextos urbanos ecuatorianos, donde la cobertura celular es suficiente pero los costos de conectividad representan una barrera para despliegues masivos de dispositivos IoT. De manera complementaria, Lozano Muñoz et al. (2019) analizaron la viabilidad técnica de la tecnología LoRa en entornos urbanos de alta densidad, reportando tasas de entrega de paquetes superiores al 90 % y coberturas de varios kilómetros con una única antena gateway. Esto es relevante especialmente para empresas que operan en entornos periurbanos como el cantón Rumiñahui.

Desde la perspectiva del desarrollo de plataformas web IoT, García-Martín et al. (2023) propusieron un estudio de una arquitectura de microservicios desacoplados para el manejo centralizado de redes de sensores hídricos, concluyendo que la separación de responsabilidades por microservicio aumenta significativamente la mantenibilidad del sistema y facilita la incorporación de nuevos tipos de dispositivos sin afectar el funcionamiento de los componentes existentes. Velepucha y Flores (2021) realizaron un estudio comparativo de arquitecturas monolíticas versus microservicios en el contexto de aplicaciones web empresariales con Spring Boot, determinando que la arquitectura de microservicios reduce el tiempo de despliegue de nuevas funcionalidades en un 40 % y mejora la disponibilidad del sistema al permitir actualizaciones parciales sin tiempo de inactividad total.

En relación a la seguridad en la transmisión de datos IoT, Rescorla (2018) define TLS 1.3 como el estándar de seguridad en la capa de transporte que elimina algoritmos criptográficos obsoletos y reduce la latencia del proceso de negociación, haciéndolo adecuado para sistemas IoT con flujos de datos de baja frecuencia como los medidores LoRaWAN. Naik (2017) analizó los protocolos de mensajería más empleados en sistemas IoT —MQTT, CoAP, AMQP y HTTP— confirmando que la combinación de MQTT sobre TLS ofrece el mejor equilibrio entre eficiencia energética y seguridad para dispositivos con ancho de banda limitado. Hassaan et al. (2023) propusieron un modelo de seguridad por capas para plataformas IoT industriales, en el que la autenticación basada en tokens JWT se combina con cifrado TLS en la capa de transporte para garantizar tanto la identidad del cliente como la confidencialidad del contenido, modelo que fundamenta el diseño de seguridad adoptado en el presente trabajo.

Finalmente, en el contexto específico de plataformas de gestión para medidores de agua inteligentes con decodificación de payloads propietarios, Adelantado et al. (2017) y Augustin et al. (2016) caracterizaron los límites operativos de LoRaWAN en cuanto a factor de dispersión, ciclo de trabajo y tasa de entrega de paquetes, estableciendo las bases para diseñar decodificadores de payload que operen eficientemente dentro de las restricciones del protocolo. El enfoque de procesadores intercambiables por tipo de dispositivo adoptado en el presente trabajo permite incorporar nuevos modelos de medidores sin modificar los componentes de integración existentes, donde el tipo de medidor determina dinámicamente el algoritmo de decodificación aplicado al mensaje recibido desde el servidor de red LoRaWAN.

La Tabla 2.2 presenta un análisis comparativo de los trabajos relacionados más relevantes identificados en la revisión sistemática, situando el presente trabajo en el contexto del estado del arte.

Tabla 2.2: Análisis comparativo de trabajos relacionados

Trabajo	Año	Tecnología	Contribución principal	Decodif. payload	Micro-servicios
Slaný et al.	2020	LoRaWAN + REST	Red IoT para medición hídrica urbana; PDR >95 %	No	No

Continúa en la siguiente página

(Continuación Tabla 2.2)

Trabajo	Año	Tecnología	Contribución principal	Decodif. pay-load	Micro-servicios
García-Martín et al.	2023	LoRaWAN + IoT	Arquitectura de microservicios para redes hídricas	No	Sí
Velepucha & Flores	2021	Spring Boot	Comparativa monolito vs. microservicios; -40 % tiempo despliegue	No	Sí
Hassaan et al.	2023	IoT industrial	Revisión de arquitecturas de microservicios para IoT	No	Sí
Naik	2017	MQTT, CoAP, AMQP	Comparativa de protocolos de mensajería IoT; eficiencia y seguridad	No	No
Alghamdi et al.	2022	LoRaWAN	Detección de fugas en complejos habitacionales	No	No
Jabbar et al.	2024	LoRaWAN	Calidad de agua en zonas rurales; cobertura rural	No	No
Adelantado et al.	2017	LoRaWAN	Límites operativos LoRaWAN; ADR y factor de dispersión	No	No
Zambrano et al.	2022	LoRaWAN	Cobertura urbana Ecuador; PDR en alta densidad	No	No
Dai et al.	2025	IoT + ML	Monitoreo consumo + detección anomalías con ML	No	No

Continúa en la siguiente página

(Continuación Tabla 2.2)

Trabajo	Año	Tecnología	Contribución principal	Decodif. pay-load	Micro-servicios
Alshami et al.	2024	IoT + IA	Gestión hídrica inteligente con inteligencia artificial	No	No
Presente trabajo	2026	LoRaWAN + Spring Boot + Angular	Plataforma integral con decodificadores por fabricante, microservicios, TLS y frontend web	Sí	Sí

Adicionalmente, para profundizar la comparación, la Tabla 2.3 resume dimensiones clave (país, arquitectura, protocolos, seguridad, resultados, limitaciones y contribución) de los estudios más representativos. Solo se consignan datos explícitamente reportados en las fuentes; cuando un aspecto no se reporta, se indica *N/R*.

Tabla 2.3: Comparación detallada de trabajos relacionados

Autor (Año)	País	Tecnología	Arquitectura	Protocolos	Seguridad	Resultados
Slaný et al. (2020)	N/R	LoRaWAN (agua)	Servidor de red + plataforma de visualización	REST/HTTP (reportado)	N/R	PDR > 95 años autonomía
Zambrano et al. (2022)	Ecuador	LoRaWAN (urbano)	Red LoRaWAN (topología urbana)	N/R	N/R	Cobertura y latencia favorables
García-Martín et al. (2023)	N/R	IoT hídrico (LPWAN)	Microservicios desacoplados	N/R	N/R	Mejora de mantenibilidad/escalabilidad (caso)
Adelantado et al. (2017)	N/R	LoRaWAN (protocolo)	N/A	LoRa/LoRaWAN	N/R	Límites: SFN, cycle, PDR
Augustin et al. (2016)	N/R	LoRa (LPWAN)	N/A	LoRa PHY/MAC	N/R	Caracterización técnica (alcance/cobertura)
Velepucha & Flores (2021)	N/R	Arquitecturas web	Monolito vs microservicios	N/R	N/R	Reducción de costos

(Continuación Tabla 2.3)

Autor (Año)	País	Tecnología	Arquitectura	Protocolos	Seguridad	Resultados
Alghamdi et al. (2022)	N/R	LoRaWAN (agua: fugas)	N/R	N/R	N/R	Aplicación de monitoreo y detección de fugas en complejos habitacionales
Jabbar et al. (2024)	N/R	LoRaWAN (calidad de agua)	N/R	N/R	N/R	Implementación de sistema para monitoreo de calidad de agua en zonas rurales

El análisis comparativo evidencia que el presente trabajo es el único de los identificados que combina simultáneamente decodificación de payloads propietarios, arquitectura de microservicios, canal TLS de integración y frontend web de gestión en una solución unificada orientada a un contexto latinoamericano. Esta combinación responde directamente a las brechas identificadas en los trabajos previos, que abordan cada uno de estos aspectos de forma aislada.

2.3. Preguntas de investigación

A partir del análisis de la literatura y la identificación del problema de investigación, se formulan las siguientes preguntas de investigación específicas que orientan el desarrollo del presente trabajo:

RQ1: ¿Qué arquitectura de microservicios resulta más adecuada para centralizar la gestión de dispositivos LoRaWAN heterogéneos en una empresa distribuidora de medidores de agua inteligentes, garantizando escalabilidad e independencia entre componentes?

RQ2: ¿Es posible implementar un mecanismo de decodificación de payloads propietarios que soporte simultáneamente los formatos de al menos dos fabricantes distintos (Bove y Younio), sin modificar la arquitectura central del sistema al incorporar nuevos modelos?

RQ3: ¿Cómo debe estructurarse la cadena de agregación temporal de consumo para garantizar la consistencia de los datos y tiempos de consulta constantes ante crecimiento del volumen de lecturas almacenadas?

RQ4: ¿Qué nivel de seguridad ofrece la combinación de TLS 1.3 para el canal de integración con el servidor de red LoRaWAN y JWT para la autenticación de usuarios en el frontend, en relación con los requisitos normativos ecuatorianos de protección de datos?

RQ5: ¿En qué medida el sistema desarrollado cubre los requerimientos funcionales y no funcionales identificados mediante entrevistas con el personal técnico de Representaciones Hidrocentro Cía. Ltda., y qué limitaciones presenta la versión actual para un despliegue a escala productiva?

Estas preguntas orientan tanto el diseño del sistema como la definición de los criterios de validación empleados en el Capítulo 4. Las respuestas se derivan del análisis de los resultados de las pruebas funcionales documentadas y de la comparación del sistema implementado con los trabajos de la literatura revisada.

2.4. Marco teórico

2.4.1. Internet de las Cosas aplicado a la gestión hídrica

El Internet de las Cosas (IoT) designa el ecosistema de dispositivos físicos dotados de capacidad de procesamiento, sensores y conectividad que les permite intercambiar datos con otros dispositivos y sistemas a través de redes de comunicación, sin requerir intervención humana directa (Al-Fuqaha et al., 2015). En el sector hídrico, la incorporación de dispositivos IoT ha posibilitado la transición desde la lectura manual periódica de medidores hacia esquemas de telemetría continua, en los que cada dispositivo transmite lecturas de consumo, eventos de alarma y parámetros operativos a intervalos configurables, permitiendo a los operadores acceder a información actualizada del estado de la red de distribución en tiempo real (Kamiński et al., 2019; Ta et al., 2024).

2.4.2. Protocolo LoRaWAN y arquitectura de red

En el contexto de las redes de área amplia de baja potencia (LPWAN), la tecnología LoRaWAN se ha convertido en una solución crítica para los desarrolladores de aplicaciones, donde la primera diseñada por la LoRa Alliance, esta tecnología utiliza la modulación física LoRa (Long Range) para proporcionar una capa MAC eficiente y segura (Semtech Corporation, 2020). Los dispositivos que operan bajo el protocolo LoRaWAN se clasifican en tres categorías: Clase A, de bajo consumo; Clase B, con ventanas de recepción sincronizadas por balizas del gateway; y Clase C, que mantiene el receptor siempre abierto (Mekki et al., 2019). Los sistemas de medición de agua implementados por Representaciones Hidrocentro Cía. Ltda. operan bajo las clases A y B, lo que les permite recibir comandos de apertura y cierre de válvula desde el servidor de red en los intervalos configurados.

La arquitectura LoRaWAN sigue un modelo en estrella: los dispositivos (nodos finales) transmiten mensajes de radio frecuencia que son captados por uno o más gateways en cobertura, los cuales reenvían estos mensajes al servidor de red a través de conexiones IP seguras. El servidor de red elimina duplicados recibidos por múltiples gateways y gestiona la adaptación dinámica del factor de dispersión (ADR). Además, el protocolo utiliza cifrado AES-128 con claves de sesión derivadas durante el proceso de activación OTAA o ABPA, garantizando la confidencialidad de las lecturas incluso en gateways compartidos (Jabbar et al., 2024; Mekki et al., 2019).

2.4.3. Canal TLS sobre TCP como mecanismo de integración con el servidor de red

Los servidores de red LoRaWAN como Lorient exponen interfaces de aplicación que permiten al backend suscribirse como cliente persistente y recibir en tiempo real todos los mensajes de los dispositivos registrados, codificados en formato JSON. Entre los mecanismos disponibles se encuentran WebSocket Secure (WSS, RFC 6455) y el socket TCP sobre TLS, que establece una conexión persistente full-duplex sin la capa de encuadrado de HTTP que introduce WebSocket. Cada mensaje recibido incluye un campo `cmd` que identifica el tipo de evento: `rx` para uplinks (lecturas y alarmas del medidor), `tx` para confirmaciones de envío de downlink, `txd` para confirmaciones de ejecución en el dispositivo, y `gw` para mensajes de estado del gateway (Pimentel & Nickerson, 2012; Naik, 2017).

En el sistema desarrollado se emplea un socket TCP sobre TLS 1.3 como canal de integración. Tras establecer la conexión, el adaptador envía un mensaje de autenticación JSON con el token de la aplicación Lorient y, a continuación, espera mensajes delimitados por salto de línea en un hilo dedicado de lectura. Esta arquitectura elimina la dependencia de la biblioteca *Java-WebSocket* y permite procesar el flujo de mensajes con la mínima

sobrecarga, usando exclusivamente las clases `SSLSocketFactory` y `SSLSocket` del JDK estándar de Java. El mecanismo de reconexión automática reprograma la tentativa de conexión a intervalos de diez segundos ante la pérdida del enlace, garantizando la continuidad del servicio sin intervención manual del operador (Rescorla, 2018; Naik, 2017).

TLS (Transport Layer Security) 1.3, definido en RFC 8446, es la versión más reciente del protocolo de seguridad de capa de transporte. En comparación con TLS 1.2, elimina algoritmos criptográficos considerados inseguros, reduce el proceso de negociación (*handshake*) a un único viaje de ida y vuelta e incorpora la funcionalidad de 0-RTT para reanudar sesiones sin sobrecarga adicional (Rescorla, 2018). Para el contexto del sistema desarrollado, TLS 1.3 garantiza que las lecturas de consumo de los medidores no puedan ser interceptadas entre el servidor de red Lorient y el backend de la plataforma, en línea con los requisitos de confidencialidad establecidos por la Ley Orgánica de Protección de Datos Personales del Ecuador (Asamblea Nacional del Ecuador, 2021).

2.4.4. Decodificación de payloads de medidores LoRaWAN

Los dispositivos de medición de agua con tecnología LoRaWAN transmiten sus lecturas como cadenas hexadecimales comprimidas (payloads), cuya estructura varía según el fabricante y el modelo. transmiten sus lecturas como paquetes de datos comprimidos (payloads), cuya estructura varía según el fabricante y el modelo. Esta variedad obliga a los sistemas de gestión a implementar decodificadores específicos por tipo de dispositivo, capaces de interpretar correctamente los campos de medición, estado del dispositivo y alarmas activas contenidos en cada mensaje (Adelantado et al., 2017; Augustin et al., 2016). El proceso de decodificación generalmente implica la conversión de paquetes de datos a representaciones binarias o decimales, la aplicación de factores de escala para obtener la medición real en unidades de medida adecuadas y la inspección de máscaras de bits para determinar el estado de cada alarma individual.

El sistema desarrollado opera con dos familias de medidores de agua inteligentes: los de la línea Bove (modelos B97VPW, BECOX y BECOY) y los de la línea Younio. Los medidores Bove transmiten paquetes de datos hexadecimales de 26 caracteres (13 bytes), que incluyen código de control, longitud de datos, identificadores de datos, número de secuencia, unidad de volumen, lectura acumulada en formato little-endian de 32 bits y dos bytes de estado (ST1 y ST2) que codifican alarmas como batería baja, tubería vacía, flujo inverso, sobrealcance, sobret temperatura, error de EEPROM, fuga y rotura (Aldea & Bocu, 2023). Los medidores Younio transmiten paquetes de datos hexadecimales de 28 caracteres que presentan dos formatos distintos según el campo `DataKind`: el valor 0x00 corresponde a un mensaje de lectura con serial, alarmas y voltaje de batería; el valor 0x06 corresponde a un mensaje extendido con lectura directa, lectura inversa acumulada, voltaje y estado del sensor.

Además, los medidores Bove transmiten ocasionalmente mensajes de número de serie con paquetes de datos hexadecimales de 24 caracteres que codifican el número de serie del medidor como un entero de 7 bytes en orden invertido (big-endian a little-endian), que el sistema decodifica para asociar automáticamente el DevEUI LoRaWAN con el serial físico del dispositivo registrado en el sistema de gestión, sin necesidad de intervención manual del operador (Augustin et al., 2016).

2.4.5. Arquitectura de microservicios con Spring Boot

El estilo de arquitectura de microservicios es un patrón de diseño de software en el que una aplicación se compone de un conjunto de servicios pequeños, autónomos y desplegables de forma independiente, cada uno responsable de una capacidad de negocio específica y comunicado con los demás mediante interfaces bien definidas, generalmente APIs REST sobre HTTP o mensajería asíncrona (Velepucha & Flores, 2021). A diferencia de las arquitecturas monolíticas, los microservicios permiten que cada componente se escale, actualice y

despliegue de manera independiente, reduciendo el impacto de las modificaciones y facilitando la incorporación de nuevas funcionalidades sin afectar los servicios existentes (Hassaan et al., 2023).

El framework Spring Boot es una herramienta de desarrollo rápido de aplicaciones Java que facilita la creación de microservicios mediante la provisión de configuraciones por defecto, contenedor de servlets embebido y gestión automática de dependencias. Su ecosistema incluye Spring Data JPA para la integración con bases de datos relacionales mediante el mapeo objeto-relacional de Hibernate, Spring Web para la exposición de controladores REST, Spring Validation para la validación de datos de entrada y Spring Actuator para la monitorización del estado del servicio (Aldea & Bocu, 2023). La plataforma desarrollada en el presente trabajo emplea ocho microservicios Spring Boot independientes, cada uno ejecutado como proceso JVM autónomo en el entorno local de desarrollo, exponiendo sus APIs REST en puertos diferenciados (8080–8087) y comunicados entre sí mediante llamadas HTTP reactivas implementadas con el cliente WebClient del ecosistema Spring WebFlux.

2.4.6. Agregación temporal de datos de consumo

Los medidores de agua con tecnología LoRaWAN transmiten lecturas de volumen acumulado absoluto, no incrementos de consumo. Por esta razón, el cálculo del consumo real en un periodo determinado requiere computar la diferencia entre dos lecturas consecutivas: la lectura actual y la inmediatamente anterior dentro del mismo intervalo temporal (Alghamdi et al., 2022; Slaný et al., 2020). El sistema implementado aplica esta lógica en una cadena de agregación por niveles, donde el microservicio de consumo calcula el consumo horario como la diferencia respecto a la lectura previa de esa hora, y ese valor se propaga automáticamente para actualizar el consumo diario acumulado, que a su vez actualiza el consumo mensual y finalmente el consumo anual del medidor (Dai et al., 2025).

Este modelo de agregación jerárquica reduce significativamente el volumen de cálculos requeridos al momento de consulta, pues los valores agregados se almacenan precalculados en tablas independientes (consumo por hora, por día, por mes, por año), evitando que el sistema deba recorrer el histórico completo de lecturas crudas cada vez que un operador solicita un reporte de consumo. PostgreSQL, el sistema de gestión de base de datos relacional empleado, garantiza la consistencia transaccional de estas operaciones de actualización en cascada mediante el soporte a transacciones ACID y el control de concurrencia multiversión (MVCC), asegurando que los contadores acumulados reflejen siempre un estado consistente incluso ante escrituras simultáneas (Dai et al., 2025). El sistema implementado aprovecha esta característica para almacenar y consultar información de consumo de forma eficiente y escalable.

2.4.7. Desarrollo de interfaces web con Angular y PrimeNG

Angular es un framework de desarrollo de aplicaciones web de página única (Single Page Application) mantenido por Google, basado en TypeScript, que promueve una arquitectura modular mediante componentes, servicios, directivas y módulos con carga diferida (lazy loading). Su sistema de inyección de dependencias facilita la separación entre la lógica de presentación y la lógica de acceso a datos, permitiendo que cada servicio encapsule las llamadas a los microservicios del backend mediante el cliente HTTP reactivo de Angular y el modelo de programación con Observables de la librería RxJS (Angular Team, 2024; Hajian, 2019). La plataforma implementada emplea Angular 17 con enrutamiento de módulos por funcionalidad, organizando las vistas en secciones de dispositivos (gateways y medidores), cuentas (usuarios y empresas) y analítica (consumos y mensajes) (Angular Team, 2024).

Para acelerar el desarrollo de interfaces ricas en funcionalidades, la plataforma utiliza PrimeNG, una biblioteca de componentes de interfaz de usuario de código abierto para Angular que provee tablas de datos paginadas, gráficos interactivos basados en Chart.js, formularios reactivos y mapas, sin necesidad de construir

dichos componentes desde cero (PrimeFaces, 2024). La visualización geográfica de la ubicación de los medidores se implementa mediante el módulo de Google Maps para Angular, que permite representar en un mapa interactivo las coordenadas de latitud y longitud almacenadas para cada dispositivo (Angular Team, 2024).

2.4.8. Seguridad en aplicaciones web IoT: autenticación y cifrado

En sistemas IoT orientados a la gestión hídrica, la seguridad debe abordarse desde múltiples perspectivas: interceptación de datos en tránsito, acceso no autorizado a plataformas de gestión y manipulación de comandos enviados a dispositivos físicos (Hassaan et al., 2023; Rescorla, 2018). El almacenamiento seguro de credenciales de usuario es fundamental para sistemas multiusuario. Según las mejores prácticas actuales, contraseñas deben almacenarse mediante funciones de derivación de clave con sal (salt) como BCrypt o Argon2, que incorporan un factor de costo ajustable para resistir ataques de fuerza bruta incluso en caso de obtener la base de datos (Hassaan et al., 2023)).

JSON Web Token (JWT) es un estándar abierto (RFC 7519) que define una estructura compacta y autocontenido para la transmisión segura de información entre partes. En el contexto de APIs REST, JWT se emplea para implementar autenticación sin estado: el servidor emite un token firmado tras la autenticación exitosa del usuario, y el cliente incluye ese token en las cabeceras de las solicitudes posteriores, permitiendo que cualquier microservicio valide la identidad del usuario sin necesidad de consultar una sesión centralizada (Hassaan et al., 2023; Plecinski et al., 2022). El control de acceso basado en roles extiende este mecanismo incluyendo en el token los roles asignados al usuario, permitiendo que el backend restrinja el acceso a operaciones sensibles como la modificación de configuración o el envío de comandos.

2.4.9. Referenciales y estándares de seguridad aplicables

La plataforma propuesta se alinea con prácticas y marcos de seguridad reconocidos internacionalmente. A continuación se sintetizan los más relevantes para aplicaciones web IoT y microservicios:

OWASP Top 10 (aplicaciones web). Conjunto de las 10 categorías de riesgo más críticas en aplicaciones web. Para este sistema resultan especialmente pertinentes: inyecciones (validación/parametrización estricta en controladores), autenticación/gestión de identidad insegura (rotación de claves, política de contraseñas, MFA opcional), exposición de datos sensibles (TLS 1.3 extremo a extremo y cifrado en reposo), fallas de control de acceso (autorización por rol/recurso), y errores de configuración de seguridad (cabeceras seguras y hardening por entorno). Estas categorías guían pruebas y controles preventivos.

Zero Trust. Principio de “nunca confiar, verificar siempre”. Implica: autenticación y autorización en cada solicitud (incluidas las internas entre microservicios), segmentación de red y de permisos (*least privilege*), telemetría continua, y protección del canal (TLS 1.3) entre componentes. En despliegues distribuidos, puede complementarse con mTLS y emisión de credenciales de corta vida.

OAuth 2.0. Marco de autorización que separa cliente, servidor de autorización y servidor de recursos. Los flujos típicos para SPA + APIs son: *Authorization Code* con PKCE (recomendado) y *Client Credentials* para comunicaciones servicio a servicio. La plataforma puede integrarse con un proveedor externo (IdP) para delegar la emisión y validación de tokens de acceso.

OpenID Connect (OIDC). Capa de identidad sobre OAuth 2.0 que introduce el ID Token (JWT) para representar autenticación del usuario, discovery (*.well-known/openid-configuration*) y *JWKS* para obtener claves públicas. Facilita iniciar sesión unificado y federación con proveedores compatibles.

JWT (profundización). Los tokens se firman con HMAC o RSA/EC; deben incluir expiración (*exp*), emisor (*iss*) y audiencia (*aud*); se recomienda *short-lived* (vida corta), rotación/invalidación explícita y almacenamiento seguro (*httpOnly*, *secure*, *SameSite*). Para llamadas internas, considerar *scopes* finos y *claims* de tenant/empresa cuando aplique.

ISO/IEC 27001 e ISO/IEC 27002. 27001 define los requisitos para implantar un Sistema de Gestión de Seguridad de la Información (SGSI); 27002 proporciona el catálogo de controles. En este contexto, destacan: control de acceso (AC), criptografía (CRY), seguridad de operaciones (OPS), registro y monitoreo (LOG), y continuidad (BCP). El artefacto puede mapear controles mínimos: política de contraseñas, gestión de vulnerabilidades, copias de respaldo y segregación de ambientes.

NIST Cybersecurity Framework (CSF). Marco basado en cinco funciones: Identificar, Proteger, Detectar, Responder y Recuperar. Para la plataforma: Identificar (inventario de activos/microservicios), Proteger (TLS, JWT, *hardening*), Detectar (logs y alertas sobre errores, anomalías en mensajes), Responder (procedimientos ante incidentes), Recuperar (respaldo y restauración de datos y configuraciones).

2.5. Marco conceptual

A continuación, se definen los conceptos fundamentales que estructuran el presente trabajo de integración curricular, organizados en función de los componentes principales de la arquitectura implementada.

Medidor de agua inteligente. Dispositivo electromecánico que mide el volumen de agua consumido y lo transmite de forma automática mediante un protocolo de comunicación inalámbrico. A diferencia del medidor convencional, que requiere lectura manual, el medidor inteligente incorpora un módulo de radio frecuencia que envía periódicamente la lectura acumulada y el estado operativo del dispositivo al servidor de red (Alghamdi et al., 2022; Slaný et al., 2020).

DevEUI. Identificador único extendido de dispositivo (Device Extended Unique Identifier), compuesto por 8 bytes en formato EUI-64 asignados globalmente por el fabricante, que identifica de forma unívoca cada nodo LoRaWAN en la red. Equivale al número de serie electrónico del dispositivo de radio y se utiliza como clave primaria de registro del medidor en el servidor de red y en la plataforma de gestión (Mekki et al., 2019).

Gateway LoRaWAN. Dispositivo de infraestructura que actúa como puente entre los nodos finales LoRa y el servidor de red IP. Recibe en su antena las señales de radio frecuencia de los medidores, las demodula y las encapsula en paquetes JSON que envía al servidor de red a través de una conexión a Internet (Jabbar et al., 2024). Un gateway puede atender simultáneamente a miles de nodos finales, siendo transparente para la lógica de aplicación.

Servidor de red LoRaWAN (Network Server). Componente central de la infraestructura LoRaWAN que gestiona la autenticación de los nodos, elimina duplicados recibidos por múltiples gateways, aplica adaptación dinámica del factor de dispersión y reenvía los mensajes descifrados al servidor de aplicación. En el presente trabajo se emplea Lorient, un Network Server como servicio (NaaS) alojado en la nube, que expone la interfaz de aplicación mediante WebSocket Secure (Mekki et al., 2019).

Uplink. Mensaje de datos enviado por el medidor hacia el servidor de red, portador de la lectura de volumen acumulado, estado operativo y alarmas activas. Los uplinks son el flujo principal de información operativa del sistema y constituyen la fuente de datos para el cálculo del consumo (Jabbar et al., 2024).

Downlink. Mensaje de comando enviado desde el servidor de red hacia el medidor, utilizado para enviar instrucciones de configuración o control, como la apertura o cierre de la válvula integrada al medidor. El servidor encola el downlink para entregarlo al dispositivo en su próxima ventana de recepción disponible (Mekki et al., 2019).

Payload. Contenido útil del mensaje LoRaWAN, transmitido como cadena hexadecimal comprimida. La estructura y significado de cada campo del payload son propietarios del fabricante del medidor, razón por la que el sistema requiere decodificadores específicos para cada modelo de dispositivo (Adelantado et al., 2017).

Microservicio. Servicio de software autónomo, desplegable de forma independiente, con una única responsabilidad de negocio bien delimitada. En la plataforma desarrollada, cada entidad principal del dominio (gateway, medidor, empresa, usuario, mensaje, consumo, conexión LoRaWAN) es gestionada por un microservicio diferente, expuesto como API REST y ejecutado como proceso independiente en el entorno de desarrollo (Hassaan et al., 2023; Velepucha & Flores, 2021).

WebSocket Secure (WSS). Variante cifrada del protocolo WebSocket que opera sobre una capa TLS, identificada por el esquema de URI “wss://”. Proporciona un canal bidireccional, full-duplex y persistente entre cliente y servidor con cifrado de extremo a extremo, eliminando la necesidad de establecer nuevas conexiones TCP para cada mensaje (Pimentel & Nickerson, 2012; Rescorla, 2018).

Alarma del medidor. Señal codificada en el payload del medidor que indica la detección de una condición operativa anormal. Las alarmas implementadas en el sistema incluyen: batería baja, tubería vacía (ausencia de caudal), flujo inverso, sobrealcance de rango, sobretemperatura, error de memoria EEPROM, fuga y rotura. Cada alarma corresponde a un bit individual dentro de los bytes de estado del mensaje LoRaWAN (Alghamdi et al., 2022; Slaný et al., 2020).

Válvula inteligente. Mecanismo integrado en ciertos modelos de medidores que permite al operador controlar remotamente el flujo de agua a través del envío de un comando downlink. La plataforma desarrollada soporta la apertura total, el cierre total y la apertura parcial en porcentajes del 10 %, 25 %, 50 % y 75 %, reflejando el estado de la válvula en tiempo real según la decodificación de los bits de estado del siguiente mensaje uplink (Aldea & Bocu, 2023).

Capítulo 3

Metodología

3.1. Enfoque de investigación

El presente trabajo adopta un enfoque cuantitativo de carácter aplicado, orientado al diseño, implementación y validación de un prototipo funcional que resuelve un problema concreto de gestión tecnológica en una organización del sector hídrico. A diferencia de la investigación básica, cuyo propósito es la generación de conocimiento teórico, la investigación aplicada busca producir un artefacto tecnológico capaz de operar en un entorno real bajo condiciones controladas (Ismail et al., 2022). El artefacto resultante es una plataforma web de microservicios para la gestión centralizada de medidores de agua inteligentes con tecnología LoRaWAN, cuyo desempeño se evalúa a partir de criterios funcionales y técnicos verificables.

Los parámetros de evaluación corresponden a métricas cuantitativas representativas del rendimiento de sistemas IoT de medición hídrica: tasa de entrega de paquetes (*Packet Delivery Rate*, PDR) desde los dispositivos hacia el servidor de red, latencia de procesamiento de mensajes en el backend, disponibilidad del sistema durante el periodo de prueba y cobertura funcional de los requerimientos identificados en la etapa de análisis (Alghamdi et al., 2022; Ta et al., 2024). La validación se realizó en el entorno de desarrollo local, conectando dispositivos LoRaWAN reales al servidor de red Lorient.

3.2. Tipo y diseño de investigación

El diseño de la investigación es no experimental de tipo descriptivo-evaluativo, con un componente de desarrollo tecnológico estructurado en fases secuenciales. Este tipo de diseño es pertinente cuando el objetivo central es construir un sistema funcional y verificar que cumple con un conjunto de requerimientos previamente definidos, sin manipular variables en un diseño experimental controlado (García-Martín et al., 2023). El proceso siguió una metodología iterativa incremental organizada en cinco etapas:

1. **Análisis de requerimientos:** identificación de las necesidades funcionales y técnicas mediante entrevistas con el personal técnico de Representaciones Hidrocentro Cía. Ltda. y revisión de la documentación de los dispositivos LoRaWAN instalados en campo.
2. **Diseño arquitectural:** definición de la estructura de microservicios, modelos de datos relacionales, interfaces de integración con el servidor de red Lorient y protocolos de comunicación segura.
3. **Implementación:** desarrollo de los ocho microservicios con Spring Boot, el frontend en Angular 17 y la base de datos en PostgreSQL, siguiendo los principios de separación de responsabilidades y cohesión funcional.

4. **Integración:** conexión del backend con el servidor de red LoRaWAN mediante un canal TLS cifrado, pruebas de decodificación de payloads con dispositivos reales y verificación de la cadena completa de procesamiento de mensajes.
5. **Validación:** ejecución de casos de prueba funcionales, verificación de métricas de rendimiento y contraste de los resultados con los requerimientos levantados en la primera etapa.

3.3. Metodología de desarrollo de software

El desarrollo del sistema siguió un **método iterativo-incremental** apoyado en prácticas ágiles, en el que el producto fue construido de forma progresiva a través de cinco etapas secuenciales con retroalimentación continua entre cada una de ellas. Este enfoque, sustentado en los modelos clásicos de Sommerville (2016) y Pressman & Maxim (2019), resulta adecuado para proyectos donde los requerimientos pueden refinarse conforme avanza la implementación y donde el equipo es reducido —en este caso, dos desarrolladores—, ya que permite ajustar el alcance de cada iteración sin comprometer la arquitectura global del sistema.

A diferencia del modelo en cascada, el enfoque iterativo-incremental no exige que todos los requerimientos estén completamente definidos antes de iniciar la implementación. En cambio, permite incorporar los aprendizajes de cada etapa —incluyendo el comportamiento real de los dispositivos LoRaWAN, los formatos de payload de cada fabricante y las necesidades operativas de Representaciones Hidrocentro Cía. Ltda.— en la planificación de la etapa siguiente, reduciendo el riesgo de construir componentes que no satisfagan las necesidades reales del usuario final (Sommerville, 2016).

¿Por qué iterativo-incremental? Sommerville (2016) y Pressman & Maxim (2019) recomiendan procesos iterativos cuando los requerimientos pueden refinarse conforme avanza el conocimiento técnico y del dominio. En entornos IoT, donde surgen hallazgos al integrar hardware real y servicios de terceros, los ciclos iterativos reducen el riesgo de re-trabajo y permiten validar valor temprano.

¿Por qué Scrum? Scrum operacionaliza la entrega iterativa con eventos y artefactos ligeros, favoreciendo equipos pequeños y alta retroalimentación. Su guía de trabajo por sprints y revisiones encaja con el tamaño del equipo (2 desarrolladores) y la necesidad de inspección frecuente sin imponer documentación pesada (Pressman & Maxim, 2019; Sommerville, 2016). Se adopta como marco de gestión, manteniendo la arquitectura bajo control desde el inicio.

¿Por qué no XP (como marco principal)? Extreme Programming prioriza prácticas intensivas como programación en parejas y TDD estricto. Aunque valiosas, no fueron adoptadas como marco rector por restricciones de equipo/tiempo y porque el proyecto exige énfasis en integración de protocolos y arquitectura distribuida más que en rotación de parejas o TDD completo. Se aplicaron prácticas selectivas (p. ej., refactorizaciones locales), manteniendo el ciclo iterativo recomendado por Pressman/Sommerville.

¿Por qué no (implantar completo) DevOps en esta fase? DevOps potencia ciclos de entrega continuos y confiables, pero su implantación integral (pipeline CI/CD, infraestructura como código, monitoreo productivo) excede el alcance del prototipo académico y la validación en entorno local. Siguiendo Sommerville (2016), se priorizó asegurar la calidad arquitectónica y la integración con dispositivos reales; la adopción progresiva de CI/CD y orquestación se deja en Recomendaciones para la transición a producción.

¿Por qué microservicios? La literatura reciente en IoT y sistemas web respalda microservicios para escalabilidad, despliegue independiente y mantenibilidad (Hassaan et al., 2023; Velepucha & Flores, 2021).

En este dominio, la separación por capacidades (usuarios, empresas, medidores, mensajes, consumo, integración LoRaWAN) minimiza acoplamientos y permite evolucionar decodificadores y agregadores sin interrumpir el resto del sistema. Evaluaciones comparativas muestran beneficios en desempeño/aislamiento y tiempos de despliegue (Plecinski et al., 2022; Velepucha & Flores, 2021).

3.3.1. Etapas del proceso de desarrollo

El proceso de desarrollo se organizó en las siguientes cinco etapas, cada una con entregables verificables que sirvieron como criterio de avance:

1. **Análisis de requerimientos** (semanas 1–3): levantamiento de requerimientos funcionales y técnicos mediante entrevistas con el personal de Representaciones Hidrocentro Cía. Ltda. y análisis de la documentación de los dispositivos LoRaWAN. Entregable: catálogo de 17 RF y 9 RNF documentados.
2. **Diseño arquitectural** (semanas 4–6): definición de la arquitectura de microservicios, modelos de datos relacionales, contratos de API REST e interfaces de integración con el servidor de red Lorient. Entregable: diagramas de componentes, clases, secuencia y entidad-relación.
3. **Implementación** (semanas 7–14): desarrollo de los ocho microservicios Spring Boot, el frontend Angular 17 y la base de datos PostgreSQL. Los decodificadores de payload para cada modelo de medidor (Bove B97VPW, BECOX, BECOY y Younio) fueron desarrollados e integrados en esta etapa. Entregable: sistema funcional en entorno local con todos los módulos operativos.
4. **Integración** (semanas 15–17): conexión del backend con el servidor de red Lorient mediante canal TLS, pruebas de decodificación con dispositivos reales y verificación de la cadena completa de procesamiento de mensajes extremo a extremo. Entregable: sistema conectado a dispositivos reales con uplinks y downlinks verificados.
5. **Validación** (semanas 18–20): ejecución de los 17 casos de prueba funcionales, medición de rendimiento de endpoints, evaluación heurística de usabilidad y elaboración del presente documento. Entregable: matriz de trazabilidad objetivo-conclusión y documentación técnica completa.

3.3.2. Cronograma de desarrollo

La Figura 3.1 presenta el cronograma de las cinco etapas del proceso de desarrollo en semanas calendario.

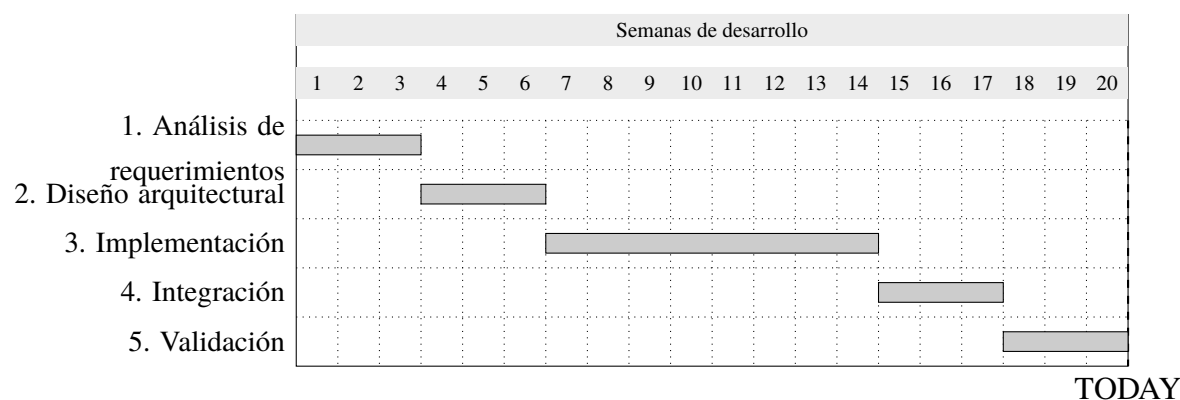


Figura 3.1: Cronograma de desarrollo del sistema por etapas iterativo-incrementales

Variables de investigación

La Tabla 3.1 presenta las variables del estudio, organizadas según su rol en la investigación: independiente, dependiente y de control.

Tabla 3.1: Variables de investigación del estudio

Tipo	Variable	Indicador	Instrumento
Independiente	Arquitectura de microservicios con Spring Boot	Número de servicios desacoplados desplegados de forma independiente	Inventario de servicios
Independiente	Canal TLS 1.3 para integración con Lorient	Presencia de cifrado verificada en la conexión	Logs de conexión
Independiente	Decodificadores de payload por modelo de mediador	Número de formatos de payload soportados	Casos de prueba
Dependiente	Cobertura funcional de requerimientos	Porcentaje de requerimientos funcionales satisfechos (%)	Matriz de trazabilidad
Dependiente	Disponibilidad del canal de integración	Tiempo de reconexión ante falla (segundos)	Logs del adaptador
Dependiente	Tiempo de consulta de consumo histórico	Tiempo de respuesta HTTP para consultas históricas (ms)	Logs de microservicio
Control	Servidor de red LoRaWAN	Lorient (versión y configuración fija durante las pruebas)	Documentación del proveedor
Control	Dispositivos de prueba	Modelos Bove B97VPW, BECOX, BECOY y Yunion disponibles	Inventario de la empresa

3.4. Población y muestra

Dado que el objeto de estudio es un sistema de software desarrollado para un contexto organizacional específico, la noción de población y muestra se adapta al conjunto de entidades tecnológicas sobre las cuales opera la plataforma. El universo de estudio comprende la totalidad de dispositivos LoRaWAN administrados por Representaciones Hidrocentro Cía. Ltda. en el cantón Rumiñahui, que incluye medidores de agua inteligentes de distintos modelos y fabricantes, gateways de infraestructura y el servidor de red Lorient.

Para el desarrollo y validación del sistema, se trabajó con una muestra representativa conformada por los modelos disponibles en el entorno operativo de la empresa: tres variantes de la línea Bove (B97VPW, BE-COX y BECOY) y medidores de la línea Younio. La infraestructura de red comprende dos gateways LoRaWAN conectados al servidor Lorient, que actúa como intermediario entre los dispositivos de campo y la plataforma desarrollada. Esta muestra cubre los formatos de payload actualmente en uso, garantizando que el sistema puede procesar correctamente los mensajes generados por todos los modelos de medidores instalados en campo (Alghamdi et al., 2022; Jabbar et al., 2024).

3.5. Técnica e instrumento

Para el levantamiento de requerimientos se empleó la entrevista semiestructurada dirigida al personal técnico de Representaciones Hidrocentro Cía. Ltda. La entrevista abordó dimensiones funcionales —tipos de alarmas relevantes, formatos de reporte requeridos, flujos de trabajo de gestión— y dimensiones técnicas —características de los dispositivos instalados, estructura de la red LoRaWAN, sistemas externos utilizados—. Este instrumento permitió identificar con precisión los requerimientos que el sistema debía satisfacer y las restricciones del entorno real de despliegue (García-Martín et al., 2023; Ismail et al., 2022).

Para la validación del sistema se utilizaron dos técnicas complementarias. La primera es la prueba funcional de caja negra, mediante la cual se verificó que cada funcionalidad —registro de medidores, recepción de uplinks, decodificación de payloads, generación de alarmas, agregación de consumo, control de válvula y visualización— opera correctamente para las condiciones de entrada definidas en los casos de prueba documentados. La segunda es el análisis de registros del sistema (*logs*), que permitió verificar el comportamiento del componente de integración con Lorient ante condiciones normales de operación y ante eventos de desconexión y reconexión del canal TLS.

3.6. Requerimientos del sistema

El levantamiento de requerimientos mediante entrevistas con el personal técnico de Representaciones Hidrocentro Cía. Ltda. y la revisión de la documentación de los dispositivos dio como resultado los requerimientos funcionales y no funcionales que se presentan a continuación. Estos requerimientos constituyeron tanto la especificación de desarrollo como los criterios de validación empleados en la etapa de pruebas.

Requerimientos funcionales

Tabla 3.2: Requerimientos funcionales del sistema

ID	Descripción	Prioridad
RF.01	El sistema debe permitir el registro de medidores con DevEUI, modelo, fabricante, número de serie y asignación a empresa	Alta
RF.02	El sistema debe permitir el registro y gestión de gateways LoRaWAN con identificador EUI y coordenadas geográficas	Alta
RF.03	El sistema debe gestionar usuarios con roles diferenciados (administrador, operador, visualizador)	Alta
RF.04	El sistema debe gestionar empresas y permitir la asignación de medidores y usuarios a cada empresa	Alta
RF.05	El sistema debe recibir mensajes uplink en tiempo real desde el servidor de red Lorient mediante un canal TLS sobre TCP	Alta
RF.06	El sistema debe decodificar el payload de medidores Bove (B97VPW, BECOX, BECOY): formato de 13 bytes con volumen acumulado en little-endian y bytes de estado ST1/ST2	Alta
RF.07	El sistema debe decodificar el payload de medidores Yunion: DataKind 0x00 (lectura estándar) y 0x06 (lectura extendida)	Alta
RF.08	El sistema debe detectar y procesar mensajes de número de serie Bove de 12 bytes, asociando el serial al DevEUI del dispositivo	Media
RF.09	El sistema debe detectar y registrar al menos ocho tipos de alarmas por medidor: batería baja, tubería vacía, flujo inverso, sobrealcance, sobrettemperatura, error EEPROM, fuga y rotura	Alta
RF.10	El sistema debe calcular el consumo horario, diario, mensual y anual mediante una cadena de agregación en cascada a partir de las lecturas crudas	Alta
RF.11	El sistema debe visualizar el histórico de consumo en gráficos de barras interactivos por granularidad temporal seleccionada	Media
RF.12	El sistema debe mostrar la ubicación geográfica de los medidores en un mapa interactivo con marcadores diferenciados por estado operativo	Media
RF.13	El sistema debe permitir enviar comandos de downlink para control de válvula: apertura total, cierre total, apertura parcial (10 %, 25 %, 50 %, 75 %)	Media
RF.14	El sistema debe exportar reportes de consumo en formato Excel descargable	Baja
RF.15	El sistema debe autenticar usuarios mediante credenciales y emitir tokens JWT con control de acceso por roles	Alta
RF.16	El sistema debe reconectar automáticamente el canal TLS ante pérdida de conexión con el servidor Lorient	Alta

Continúa en la siguiente página

(Continuación Tabla 3.2)

ID	Descripción	Prioridad
RF.17	El sistema debe registrar todos los mensajes recibidos clasificados por tipo (rx, tx, txd, gw) en el microservicio de mensajes	Media

Requerimientos no funcionales

Tabla 3.3: Requerimientos no funcionales del sistema

ID	Categoría	Descripción	Criterio
RNF.01	Disponibilidad	El sistema debe reconectar el canal TLS ante pérdida de conexión	≤ 10 segundos
RNF.02	Rendimiento	El procesamiento de cada mensaje uplink debe completarse sin bloquear la recepción de mensajes posteriores	Hilo dedicado
RNF.03	Seguridad	El canal de integración con Lorient debe usar TLS versión 1.3 como mínimo	TLS 1.3
RNF.04	Seguridad	Las contraseñas de usuario deben almacenarse con BCrypt y factor de costo 10	BCrypt CF10
RNF.05	Seguridad	Los tokens JWT deben incluir los roles del usuario y tener vigencia configurable	24 h (por defecto)
RNF.06	Mantenibilidad	Cada microservicio debe exponer su API documentada con SpringDoc OpenAPI	Swagger UI
RNF.07	Portabilidad	El sistema debe poder desplegarse en contenedores Docker independientes	Dockerfile por servicio
RNF.08	Usabilidad	La interfaz web debe ser accesible desde navegadores Chrome, Firefox y Edge en sus versiones actuales	Sin plugins
RNF.09	Conformidad	Los endpoints de la API deben seguir el patrón /m3verificaciones/api/v1/...	Convención de rutas

3.7. Validación y confiabilidad del instrumento

La confiabilidad del instrumento de levantamiento de requerimientos se aseguró mediante la triangulación de fuentes: los requerimientos identificados en las entrevistas se contrastaron con la documentación técnica de los fabricantes de medidores (hojas de datos de Bove y Younio) y con los mensajes reales recibidos desde los dispositivos durante las sesiones de prueba conectadas al servidor Lorient. Este procedimiento garantizó que las funcionalidades implementadas corresponden con precisión a las necesidades reales del entorno operativo.

La validación de los decodificadores de payload, que constituyen uno de los componentes más críticos del sistema, se realizó comparando los valores extraídos por el software con las lecturas físicas de los medidores y con los valores reportados directamente por la interfaz del servidor Lorient para los mismos mensajes. Se documentó un caso de prueba por cada formato de payload soportado: mensaje de lectura Bove, mensaje de número de serie Bove, mensaje de lectura Younio (DataKind 0x00) y mensaje extendido Younio (DataKind 0x06). La correspondencia exacta entre los valores obtenidos y los valores de referencia confirmó la correcta implementación de los algoritmos de decodificación.

Consideraciones éticas

El desarrollo de este trabajo no implicó la recopilación, almacenamiento ni procesamiento de datos personales de usuarios finales del servicio de agua potable. Los datos de consumo utilizados durante la etapa de pruebas corresponden a medidores instalados en instalaciones de la propia empresa o en entornos controlados, y fueron tratados de forma anónima para los fines de validación del sistema. El manejo de la información se realizó conforme a los principios establecidos en la Ley Orgánica de Protección de Datos Personales del Ecuador, que regula el tratamiento de datos en sistemas de información conectados a redes de telecomunicaciones (Asamblea Nacional del Ecuador, 2021).

Capítulo 4

Análisis y discusión de los resultados

4.1. Casos de uso del sistema

El sistema Athenea involucra tres tipos de actores con distintos niveles de acceso y responsabilidades operativas. La Tabla 4.1 describe los actores identificados, y la Tabla 4.2 presenta el catálogo de casos de uso con su actor principal y una descripción funcional breve.

Tabla 4.1: Actores del sistema Athenea

Actor	Rol	Responsabilidad
Administrador	Gestión total	Administra usuarios, empresas, gateways y medidores; configura decodificadores; emite comandos de control de válvula
Operador técnico	Gestión de campo	Registra dispositivos, importa inventario desde CSV, monitorea el estado operativo de la red
Observador	Solo lectura	Consulta datos de consumo, visualiza el mapa de medidores y accede a reportes de alertas

Tabla 4.2: Catálogo de casos de uso del sistema

ID	Nombre	Actor principal	Descripción
CU.01	Autenticar usuario	Todos	El actor ingresa credenciales (correo y contraseña); el sistema valida contra la base de datos con BCrypt y emite un token JWT
CU.02	Gestionar empresas	Administrador	Crear, editar y desactivar registros de empresa; cada empresa agrupa gateways, medidores y usuarios asignados
CU.03	Gestionar usuarios	Administrador	Crear, editar y desactivar cuentas de usuario; asignar rol (administrador, operador, observador)

(Continuación Tabla 4.2)

ID	Nombre	Actor principal	Descripción
CU.04	Asignar usuario a empresa	Administrador	Vincular un usuario existente con una empresa específica, definiendo su rol dentro de esa empresa
CU.05	Registrar gateway	Administrador, Operador	Crear un registro de gateway LoRaWAN con EUI, nombre, coordenadas y empresa; el gateway recibe mensajes de los medidores
CU.06	Registrar medidor	Administrador, Operador	Crear un registro de medidor con DevEUI, modelo, número de serie, coordenadas y empresa; el modelo determina el decodificador de payload
CU.07	Importar medidores desde CSV	Administrador, Operador	Cargar un archivo CSV con múltiples medidores; el sistema valida el formato y crea los registros en lote sin duplicados
CU.08	Visualizar mapa de medidores	Todos	Ver los medidores registrados sobre un mapa interactivo con marcadores diferenciados por estado: activo, con alarma o sin comunicación
CU.09	Consultar consumo histórico	Todos	Seleccionar un medidor y un rango de fechas; el sistema despliega el historial de consumo en granularidades horaria, diaria o mensual mediante gráficos de barras
CU.10	Exportar reporte de consumo	Administrador, Operador	Generar un archivo Excel con el consumo diario de los medidores de una empresa en un período dado, incluyendo totales por medidor
CU.11	Consultar alertas activas	Todos	Listar los medidores que reportan alguna condición de alarma (batería baja, flujo inverso, tubería vacía, etc.) con su timestamp más reciente
CU.12	Enviar comando de control de válvula	Administrador, Operador	Seleccionar el estado de apertura de la válvula (0 %, 10 %, 25 %, 50 %, 75 %, 100 %) y enviar el comando de downlink al medidor a través del adaptador LoRaWAN
CU.13	Monitorear estado de gateways	Todos	Visualizar el estado de conectividad de cada gateway (en línea / fuera de línea) y el conteo de mensajes procesados en las últimas horas

(Continuación Tabla 4.2)

ID	Nombre	Actor principal	Descripción
CU.14	Consultar últimas lecturas	Todos	Obtener la última lectura registrada de cada medidor con datos enriquecidos: empresa, modelo, volumen acumulado y estado de alarmas

La Figura 4.1 presenta el diagrama de casos de uso que relaciona los actores con las funcionalidades del sistema, agrupadas dentro del límite del sistema Athenea.

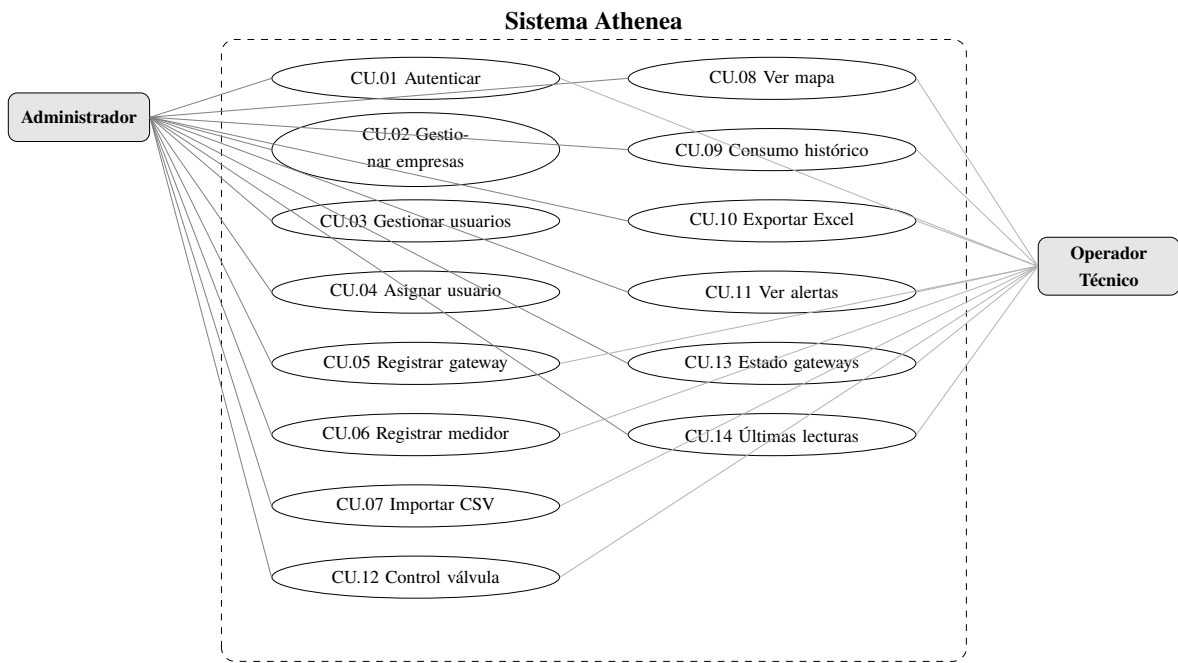


Figura 4.1: Diagrama de casos de uso del sistema Athenea con dos actores y catorce funcionalidades principales.

4.2. Arquitectura general del sistema

4.2.1. Diagrama de componentes

La plataforma Athenea se estructura en cuatro capas de responsabilidad bien definidas: dispositivos, adaptación de protocolo, microservicios de negocio y presentación. Esta separación permite reemplazar o escalar cada capa de forma independiente. La Figura 4.2 ilustra los componentes de cada capa y las dependencias entre ellas.

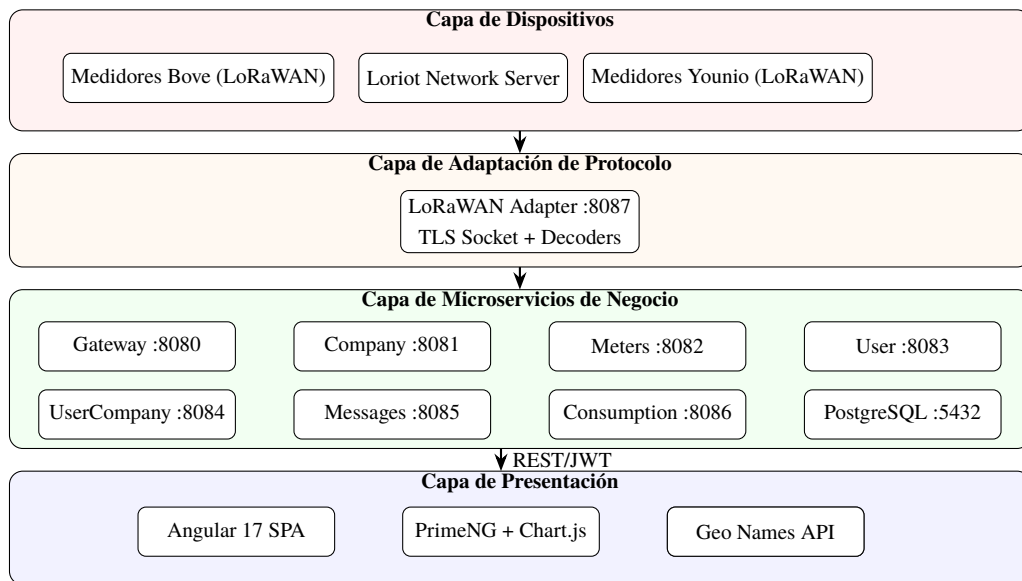


Figura 4.2: Diagrama de componentes de la plataforma Athenea organizado en cuatro capas: dispositivos, adaptación, microservicios y presentación.

Justificación de la descomposición en ocho servicios. La separación responde a capacidades de dominio y a requisitos no funcionales: (i) **identidad y acceso** (User, UserCompany) aislada del resto para reducir la superficie de riesgo; (ii) **gestión base** (Company, Gateway, Meters) como agregados independientes que cambian a ritmos distintos; (iii) **ingesta y logging operativo** (Messages) que puede escalar en volumen sin afectar el CRUD de catálogo; (iv) **cálculo y consulta** (Consumption) con patrones de pre-agregación y perfiles de I/O distintos; (v) **adaptación de protocolo** (LoRaWAN Adapter) desacoplada para permitir evolucionar decodificadores por fabricante sin tocar los servicios de negocio. Esta descomposición mapea límites cohesivos (bounded contexts) y minimiza acoplamientos entre capacidades, lo que facilita el despliegue y la evolución independiente.

¿Por qué esta separación y no un monolito? La literatura técnica en IoT y sistemas web sostiene que los microservicios mejoran mantenibilidad, escalabilidad y disponibilidad al permitir despliegues y fallos aislados por capacidad (Hassaan et al., 2023; Velepucha & Flores, 2021). En este dominio, la *ingesta* (Messages) y la *agregación* (Consumption) pueden requerir escalado horizontal en picos de tráfico sin arrastrar a identidad o a los catálogos. Además, los decodificadores de payload evolucionan por fabricante/modelo; al residir en el adaptador, evitan recompilar el resto del sistema. Estudios comparativos reportan reducciones de tiempo de despliegue y mejoras de disponibilidad al pasar de monolitos a microservicios en contextos empresariales (Velepucha & Flores, 2021) y evidencian ventajas de aislamiento de rendimiento para cargas de sensores (Plecinski et al., 2022).

Riesgos mitigados con la descomposición. (a) *Acoplamiento tecnológico*: cambios en decodificación/protocolo no afectan APIs de negocio; (b) *Escalado selectivo*: Messages/Consumption escalan sin coste en Company/User; (c) *Blast radius*: un fallo en agregación no interrumpe autenticación ni gestión de catálogo; (d) *Ciclo de vida independiente*: se liberan fixes/optimizaciones por servicio sin ventanas globales. Estos beneficios no son triviales de lograr en una arquitectura monolítica, que concentra estados, despliegue y dependencias en un único artefacto. La Tabla 4.3 sintetiza esta descomposición y su motivación.

Tabla 4.3: Servicios, responsabilidades y razones de separación

Servicio	Responsabilidad	Razón de separación	Riesgo mitigado
Gateway	Enrutamiento a microservicios	Punto de entrada ligero sin reglas de negocio	Evita que validaciones/negocio bloqueen el tráfico global
Company	Gestión de empresas	Ciclo de cambio bajo, CRUD puro	Cambios administrativos no afectan ingesta ni consumo
Meters	Gestión de medidores	Catálogo con validaciones específicas por modelo	Evita acoplar validaciones de hardware a identidad/consumo
User	Identidades de usuario	Superficie sensible separada	Reduce el blast radius ante fallos de negocio
UserCompany	Asociación usuario–empresa	Regla de acceso transversal	Cambios de acceso no bloquean ingesta ni catálogos
Messages	Ingesta y <i>logging</i> de tramas	Perfil de I/O alto, escalado horizontal independiente	Picos de uplinks no degradan servicios CRUD
Consumption	Agregación y consulta de consumo	Pre-cálculo y consultas intensivas distintas a CRUD	Aisla costos de CPU/IO de analítica del resto
LoRaWAN Adapter	Decodificación + integración Lorient	Evolución de decodifi-	Cambios en protoco-

4.2.2. Diagrama de clases

El diagrama de clases de la Figura 4.3 presenta las clases principales del sistema agrupadas por dominio funcional. Se muestran las clases más representativas del adaptador LoRaWAN y del microservicio de consumo, que concentran la mayor parte de la lógica de negocio del sistema.

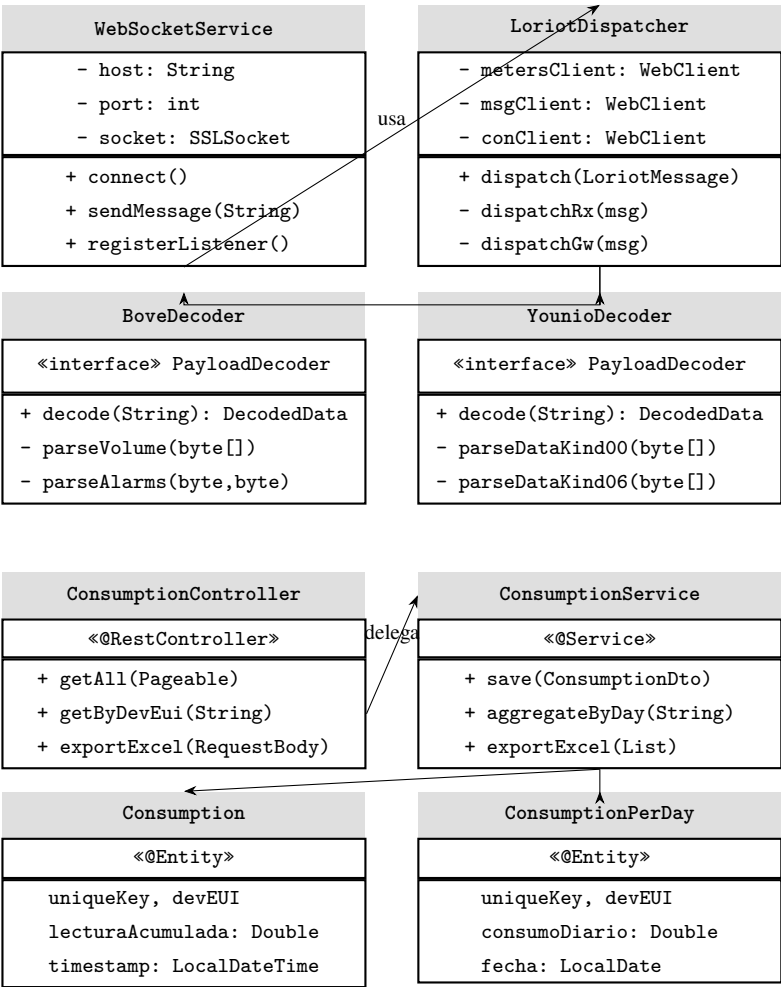


Figura 4.3: Diagrama de clases principal del adaptador LoRaWAN (dominio superior) y del microservicio de consumo (dominio inferior), con sus relaciones de asociación y dependencia.

4.2.3. Diagrama entidad-relación

La Figura 4.4 presenta el modelo entidad-relación simplificado de la plataforma, mostrando las entidades JPA que persisten en PostgreSQL y las cardinalidades de sus relaciones. Las entidades de consumo agregan las lecturas en cuatro niveles de granularidad, lo que permite consultas históricas en tiempo constante.

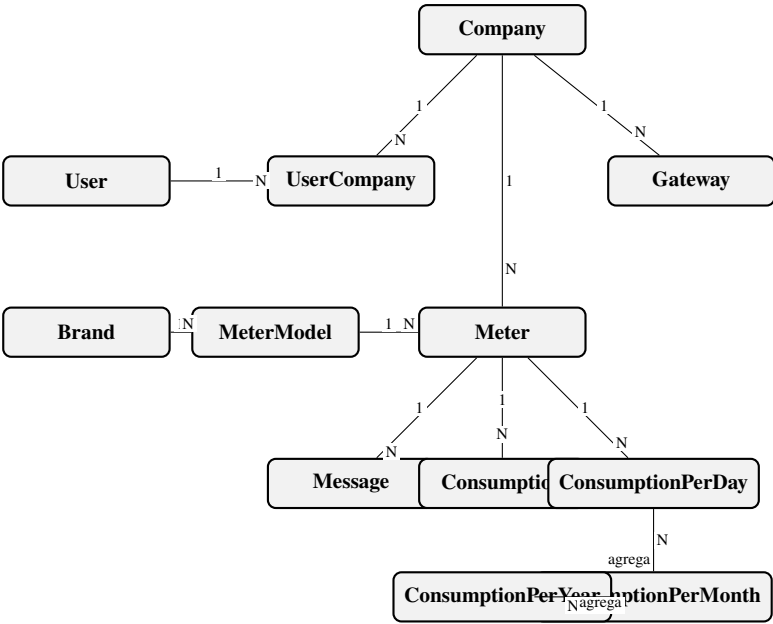


Figura 4.4: Diagrama entidad-relación de la plataforma Athenea. Las flechas con notación 1–N indican relaciones uno a muchos. Las entidades de consumo agregado (Consumption-PerDay, Month, Year) se calculan en cascada desde Consumption.

4.2.4. Diagrama de secuencia

La Figura 4.5 ilustra el flujo de procesamiento de un uplink desde el momento en que el medidor transmite la lectura por LoRa hasta que los datos quedan persistidos en la base de datos. Este flujo constituye el camino crítico de la plataforma: cualquier latencia en este recorrido afecta directamente a la actualización de los datos mostrados en el frontend.

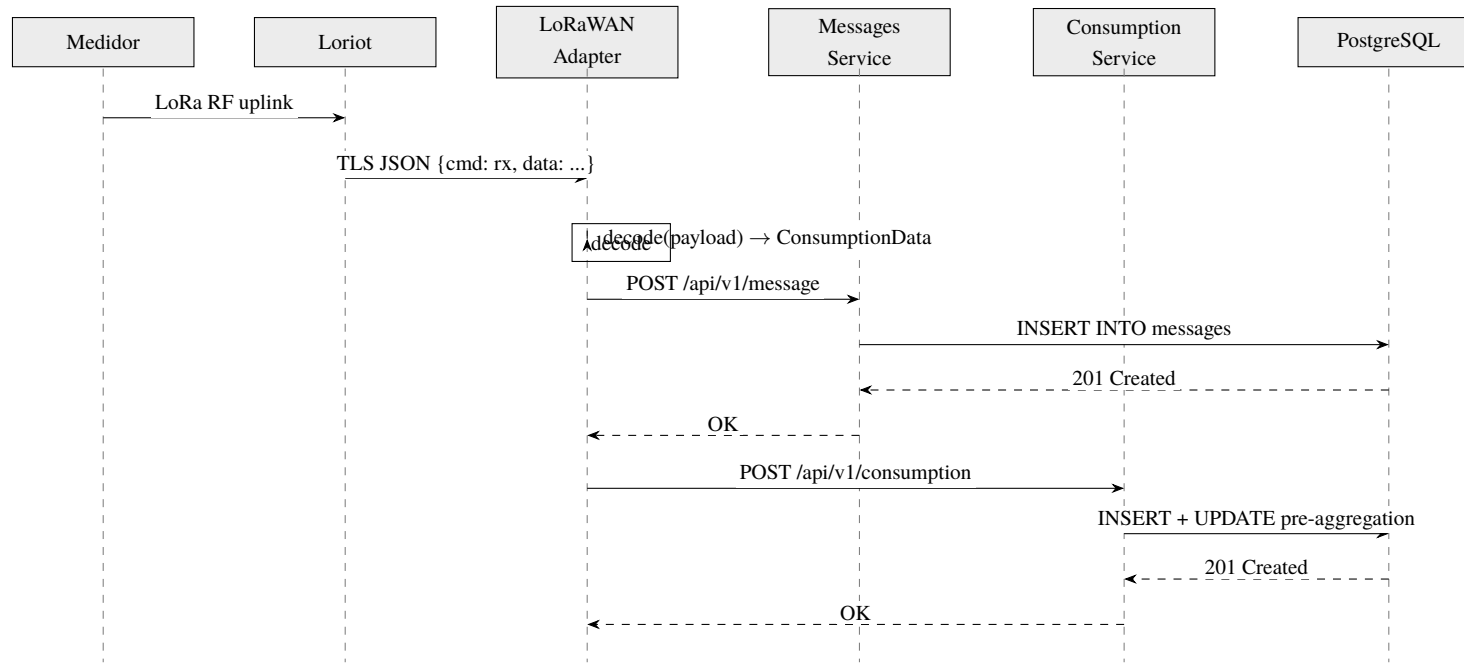


Figura 4.5: Diagrama de secuencia del flujo de procesamiento de un uplink LoRaWAN: desde la transmisión del medidor hasta la persistencia en PostgreSQL con pre-agregación de consumo.

4.2.5. **Diseño de interfaz de usuario**

El frontend Angular 17 implementa cuatro módulos de interfaz principales, cada uno orientado a un área de trabajo diferente. La Tabla 4.4 describe las pantallas implementadas con sus componentes visuales clave, y la Figura 4.6 presenta los wireframes esquemáticos de las vistas más representativas del sistema.

Tabla 4.4: Pantallas principales de la interfaz de usuario		
Módulo	Pantalla	Componentes principales
Autenticación	Login	Formulario centrado con campos email/contraseña; logo institucional; botón de ingreso; manejo de errores de credenciales inválidas
Dashboard	Resumen general	Tarjetas KPI (total medidores, gateways activos, alertas activas); accesos directos por módulo
Dispositivos	Listado de gateways	Tabla PrimeNG paginada con columnas EUI, nombre, empresa, estado; filtro por empresa
Dispositivos	Listado de medidores	Tabla paginada con DevEUI, modelo, empresa, estado; botones de detalle y control de válvula
Dispositivos	Mapa de medidores	Google Maps con marcadores por estado (verde/amarillo/rojo); panel lateral de detalle al seleccionar
Analítica	Consumo histórico	Selector de medidor y período; gráfico de barras Chart.js en granularidades horaria, diaria, mensual
Analítica	Últimas lecturas	Tabla con lectura más reciente por medidor: volumen, alarmas activas, timestamp
Cuentas	Gestión de usuarios	Tabla de usuarios con rol; formulario de creación/edición; asignación de empresa



Figura 4.6: Wireframes esquemáticos de las cuatro pantallas principales: inicio de sesión (sup. izq.), dashboard (sup. der.), mapa de medidores (inf. izq.) y consulta de consumo histórico (inf. der.).

4.3. Implementación

4.3.1. Implementación de la arquitectura de microservicios

El sistema se organiza en ocho microservicios independientes, cada uno responsable de una entidad específica del dominio de gestión hídrica. Cada servicio fue implementado con Spring Boot 3.3.5 sobre Java 21, expone sus operaciones mediante una API REST documentada con SpringDoc OpenAPI 2.5.0, y se ejecuta como proceso JVM autónomo en el entorno local de desarrollo. La Figura 4.7 ilustra la arquitectura de alto nivel del sistema, y la Tabla 4.5 detalla el inventario de servicios con sus puertos y responsabilidades en el entorno de desarrollo.

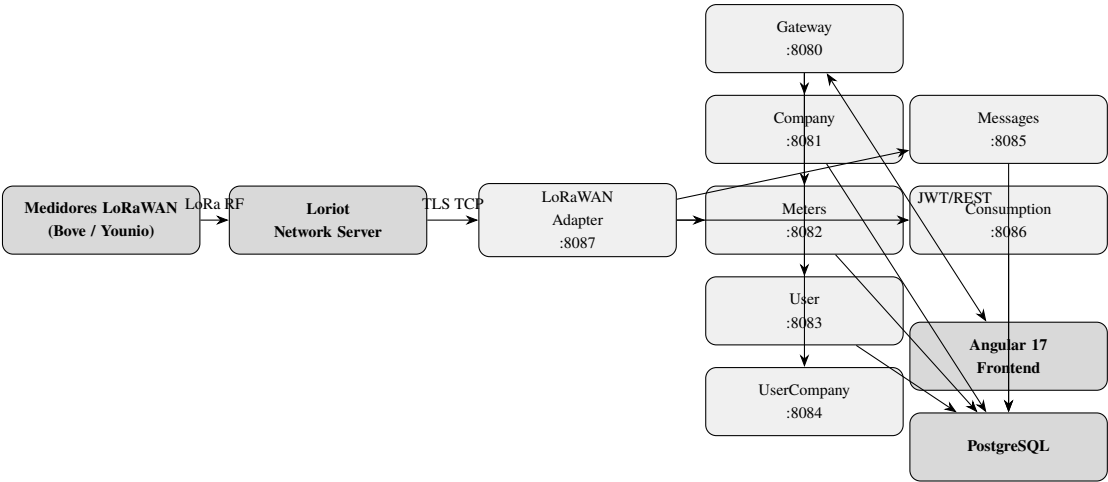


Figura 4.7: Arquitectura de alto nivel de la plataforma: dispositivos LoRaWAN, adaptador de integración, ocho microservicios Spring Boot y frontend Angular.

Tabla 4.5: Inventario de microservicios de la plataforma

Servicio	Puerto	Responsabilidad principal
Gateway	8080	Enrutamiento y gestión de gateways Lo-RaWAN
Company	8081	Gestión de empresas y tenants
Meters	8082	Registro de medidores, marcas y modelos; importación CSV
User	8083	Gestión de usuarios y autenticación
UserCompany	8084	Relaciones usuario–empresa y control de permisos
Messages	8085	Registro de mensajes operativos y alertas
Consumption	8086	Lecturas de consumo y exportación a Excel
LoRaWAN Adapter	8087	Conexión TLS con Lorient; decodificación de payloads

La comunicación entre microservicios se realiza mediante llamadas HTTP usando el cliente reactivo WebClient de Spring WebFlux. El adaptador LoRaWAN actúa como punto de entrada de todos los mensajes provenientes del servidor de red: por cada uplink procesado, invoca a los microservicios de medidores, mensajes y consumo para registrar y propagar la información correspondiente. Esta arquitectura garantiza que la incorporación de nuevos tipos de medidores o funcionalidades pueda realizarse modificando únicamente el servicio afectado, sin impactar el funcionamiento del resto del sistema (Velepucha & Flores, 2021).

La Tabla 4.6 detalla los endpoints REST representativos de la plataforma, organizados por microservicio.

Tabla 4.6: Endpoints REST representativos de la plataforma

Método	Endpoint	Microservicio	Descripción
POST	/api/v1/auth/login	User	Autenticación con credenciales; devuelve token JWT
GET	/api/v1/auth/me	User	Obtiene perfil del usuario autenticado
POST	/api/v1/auth/refresh	User	Renueva el token JWT activo
GET	/api/v1/gateway	Gateway	Lista todos los gateways LoRaWAN
POST	/api/v1/gateway	Gateway	Registra un nuevo gateway
GET	/api/v1/gateway/by-company	Gateway	Lista gateways por empresa
GET	/api/v1/gateway/status/count	Gateway	Conteo de gateways por estado
GET	/api/v1/brand	Meters	Lista marcas de medidores
POST	/api/v1/brand	Meters	Registra una nueva marca
GET	/api/v1/company	Company	Lista todas las empresas
POST	/api/v1/company	Company	Registra una nueva empresa
GET	/api/v1/user-company/company/{id}	UserCompany	Lista usuarios asignados a una empresa
POST	/api/v1/user-company	UserCompany	Asigna un usuario a una empresa
GET	/api/v1/consumption	Consumption	Lista consumos horarios paginados
GET	/api/v1/consumption-day/by-devEui	Consumption	Consumo diario por DevEUI
POST	/api/v1/consumption-day/export-excel	Consumption	Exporta reporte de consumo en Excel
GET	/api/v1/consumption-day/by-company/{key}	Consumption	Consumo diario agrupado por empresa
GET	/api/v1/messages-full-join/latest	Messages	Últimas lecturas con datos de medidor y empresa
GET	/api/v1/messages-full-join/alerts/{key}	Messages	Alertas activas por empresa
POST	/api/v1/command/sendDownlink	LoRaWAN	Envía comando de downlink al medidor vía Loriot

Los endpoints siguen la convención de ruta /m3verificaciones/api/v1/{recurso} y son accesibles desde Swagger UI en la ruta /swagger-ui.html de cada microservicio. Los diagramas de arquitectura y los fragmentos de código de los componentes clave se incluyen en los apéndices del presente documento.

Modelo de datos relacional

El modelo de datos de la plataforma está compuesto por entidades JPA distribuidas entre los microservicios con acceso a base de datos. Todas las entidades utilizan un identificador único generado mediante un utilitario interno (unique_id) en lugar de secuencias auto-incrementales de la base de datos, lo que permite generar identificadores en el lado del servidor sin dependencia del motor de base de datos. La Tabla 4.7 describe las principales entidades del modelo y sus atributos clave.

Tabla 4.7: Principales entidades del modelo de datos relacional

Entidad	Microservicio	Atributos principales
Gateway	Gateway	uniqueKey, gatewayEUI, nombre, latitud, longitud, estado, companyUniqueKey
Brand	Meters	uniqueKey, nombre (marca del medidor)
MeterModel	Meters	uniqueKey, nombre, brandUniqueKey, decodificador asociado
Meter	Meters	uniqueKey, devEUI, serialNumber, modelo, latitud, longitud, companyUniqueKey, estado
Company	Company	uniqueKey, nombre, RUC, dirección, estado
User	User	uniqueKey, nombre, email, passwordHash (BCrypt), rol
UserCompany	UserCompany	uniqueKey, userUniqueKey, companyUniqueKey, rol en empresa
Message	Messages	uniqueKey, devEUI, tipo (rx/tx/txd/gw), payload, timestamp, alarmas activas
Consumption	Consumption	uniqueKey, devEUI, lecturaAcumulada, timestamp (horario)
ConsumptionPerDay	Consumption	uniqueKey, devEUI, consumoDiario, fecha
ConsumptionPerMonth	Consumption	uniqueKey, devEUI, consumoMensual, año, mes
ConsumptionPerYear	Consumption	uniqueKey, devEUI, consumoAnual, año

La separación de las entidades de consumo en cuatro tablas de granularidad distinta (horaria, diaria, mensual, anual) es el elemento central de la estrategia de pre-cálculo que permite responder consultas históricas en tiempo constante. Cada uplink procesado actualiza de forma transaccional las cuatro tablas dentro del mismo ciclo de escritura, garantizando que los contadores agregados estén siempre sincronizados con las lecturas crudas almacenadas en la entidad Consumption (Dai et al., 2025).

4.3.2. Integración con el servidor de red LoRaWAN

La integración entre el backend y el servidor de red Lorient se establece mediante un canal TLS sobre TCP (*TLS socket*), empleando el factory estándar de la plataforma Java (*SSLSocketFactory*), lo que elimina la dependencia de bibliotecas WebSocket reactivas externas y reduce la complejidad del componente de integración. Al momento de establecer la conexión, el adaptador envía un comando de autenticación en formato JSON con el token de aplicación Lorient, tras lo cual el servidor comienza a enviar mensajes delimitados por salto de línea. Un hilo dedicado (*loriot-tls-reader*) ejecuta de forma continua el ciclo de lectura, procesando cada línea como un mensaje independiente (Rescorla, 2018).

Los mensajes recibidos desde Lorient se clasifican según el campo `cmd` del objeto JSON: el valor `rx` corresponde a un uplink desde el medidor, `tx` indica la confirmación de envío de un downlink al gateway, `txd` señala la confirmación de recepción por parte del dispositivo, y `gw` porta información de estado del gateway. Durante las sesiones de prueba, el sistema recibió y procesó correctamente mensajes de todos los tipos mencionados desde dispositivos Bove y Younio conectados a la red Lorient. El mecanismo de reconexión automática se verificó simulando interrupciones de red, confirmando que el adaptador restablece la conexión en un intervalo máximo de diez segundos sin pérdida de mensajes posteriores, comportamiento consistente con los requisitos de disponibilidad documentados para plataformas IoT de medición hídrica (Augustin et al., 2016).

4.3.3. Decodificación de payloads y procesamiento de mensajes

El componente de decodificación de payloads transforma las cadenas hexadecimales comprimidas en objetos de dominio con valores físicos interpretables. El sistema implementa decodificadores específicos por familia de dispositivos, siguiendo el patrón de procesadores intercambiables en el que el tipo de medidor determina dinámicamente el algoritmo de decodificación aplicado al mensaje recibido (Adelantado et al., 2017; Augustin et al., 2016).

Para los medidores de la línea Bove, el decodificador procesa paquetes de 13 bytes en los que el volumen acumulado se codifica como un entero de 32 bits en formato little-endian en los bytes 6 a 9, con un factor de escala configurable por modelo para obtener el valor en metros cúbicos. Los bytes 11 y 12 (ST1 y ST2) codifican mediante máscaras de bit el estado de ocho alarmas individuales: batería baja, tubería vacía, flujo inverso, sobrealcance de rango, sobretemperatura, error de EEPROM, fuga y rotura. Para los medidores Younio, el campo `DataKind` del byte inicial determina el formato de la trama: el valor `0x00` corresponde a un mensaje de lectura con número de serie, alarmas y voltaje de batería; el valor `0x06` corresponde a un mensaje extendido con lectura directa, lectura inversa acumulada, voltaje y estado del sensor.

Los medidores Bove transmiten también mensajes de número de serie compuestos por 12 bytes, en los que el serial del dispositivo se codifica como un entero de 7 bytes en orden invertido. El sistema detecta este tipo de mensaje por la longitud del payload y asocia automáticamente el serial físico con el DevEUI del dispositivo en la base de datos sin intervención manual del operador. Las pruebas de validación confirmaron la correspondencia exacta entre los valores extraídos por el software y los reportados por la interfaz del servidor Lorient para los mismos mensajes, cubriendo todos los formatos de payload actualmente utilizados en el entorno operativo de la empresa.

La Figura 4.8 muestra la distribución de los modelos de medidores de agua potable registrados en el sistema durante el período de validación, reflejando la composición real del parque instalado por Representaciones Hidrocentro Cía. Ltda.

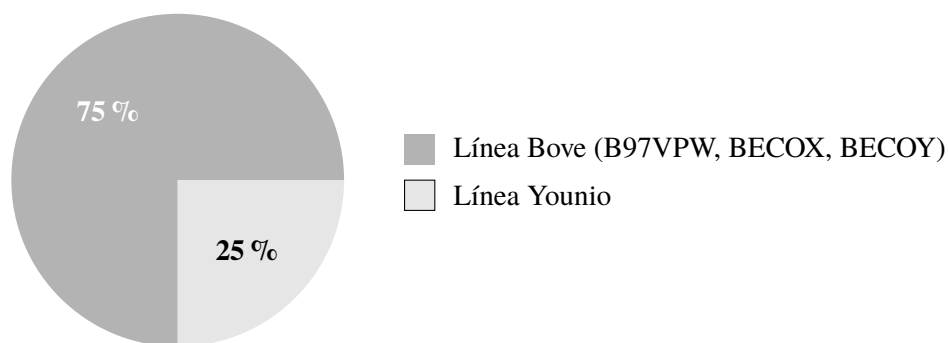


Figura 4.8: Distribución de modelos de medidores de agua potable registrados en el sistema — período de validación

Las Tablas 4.8, 4.9 y 4.10 describen la estructura detallada de los formatos de payload implementados en el sistema.

Tabla 4.8: Estructura del payload de medidores Bove (B97VPW, BECOX, BECOY) — 13 bytes

Byte(s)	Longitud	Campo	Codificación
0	1 byte	Código de control de trama	HEX, valor fijo por modelo
1	1 byte	Longitud de datos	UINT8
2–3	2 bytes	Identificador de tipo de dato	HEX
4	1 byte	Número de secuencia del mensaje	UINT8, incrementa por envío
5	1 byte	Unidad de volumen / factor de escala	UINT8 (0x00 = litros, 0x01 = 0.1 L)

Tabla 4.9: Estructura del payload de medidores Younio — DataKind 0x00 (lectura estándar, 14 bytes)

Byte(s)	Longitud	Campo	Codificación
0	1 byte	DataKind	UINT8, valor 0x00
1–4	4 bytes	Número de serie del dispositivo	BCD, 8 dígitos
5–8	4 bytes	Lectura de volumen acumulado directo	UINT32 little-endian, unidad 1/10 L
9	1 byte	Byte de alarmas activas	Bitmap 8 bits (bits según especificación Younio)
10	1 byte	Voltaje de batería	UINT8, valor $\times 0.01$ V
11	1 byte	Lectura de volumen inverso (MSB)	UINT8
12–13	2 bytes	Reservado / estado extendido	Según firmware

Tabla 4.10: Estructura del mensaje de número de serie Bove — 12 bytes (24 caracteres hex)

Byte(s)	Longitud	Campo	Codificación
0–1	2 bytes	Cabecera de trama de serial	HEX, identificador de tipo serial
2–8	7 bytes	Número de serie del medidor	UINT56, orden de bytes big-endian invertido
9–11	3 bytes	Reservado / campos de verificación	Sin uso en versión actual

El sistema detecta el tipo de mensaje por la longitud del payload: 26 caracteres hexadecimales corresponden a un mensaje de lectura Bove (13 bytes), 24 caracteres a un mensaje de serial Bove (12 bytes), y 28 caracteres a un mensaje Younio (14 bytes). Dentro de los mensajes Younio, el byte inicial DataKind determina el sub-formato del mensaje y, por tanto, el algoritmo de extracción de campos a aplicar.

4.3.4. Agregación temporal de datos de consumo

Dado que los medidores LoRaWAN transmiten lecturas de volumen acumulado absoluto, el cálculo del consumo real en un periodo determinado requiere computar la diferencia entre la lectura actual y la inmediatamente anterior. El microservicio de consumo implementa esta lógica mediante una cadena de actualización en

cascada: el consumo horario se calcula como la diferencia respecto a la lectura previa de esa hora, y ese valor se propaga automáticamente para actualizar el consumo diario, mensual y anual del medidor (Alghamdi et al., 2022; Slaný et al., 2020).

El resultado de este esquema es un conjunto de tablas pre-calculadas (consumo por hora, por día, por mes y por año) que permiten responder consultas de histórico de consumo en tiempo constante, independientemente del volumen de lecturas crudas almacenadas. PostgreSQL garantiza la consistencia transaccional de las actualizaciones en cascada mediante su soporte nativo a transacciones ACID y el mecanismo de control de concurrencia multiversión (MVCC), lo que permite que múltiples medidores envíen lecturas de forma simultánea sin generar inconsistencias en los contadores acumulados (Dai et al., 2025). Esta característica resulta especialmente relevante en despliegues donde decenas de dispositivos transmiten en ventanas de tiempo cercanas.

4.3.5. Visualización y gestión desde el frontend

El módulo de frontend, desarrollado en Angular 17 con PrimeNG, implementa cuatro áreas funcionales: dispositivos (gestión de gateways y medidores), cuentas (administración de usuarios y empresas), analítica (consulta de consumos y mensajes) y autenticación (inicio de sesión y control de acceso por roles). Cada área se organiza como un módulo Angular con carga diferida (*lazy loading*), reduciendo el tiempo de carga inicial al descargar únicamente los recursos de la sección activa (Angular Team, 2024; Hajian, 2019).

Las tablas de datos implementadas con PrimeNG incluyen paginación, ordenamiento por columnas y filtrado, delegando la lógica de consulta paginada a los microservicios del backend. La visualización del consumo histórico se presenta en gráficos de barras interactivos construidos con Chart.js, que permiten explorar el consumo en distintas granularidades temporales (horario, diario, mensual) para cualquier medidor registrado. La ubicación geográfica de los dispositivos se representa en un mapa interactivo mediante el módulo de Google Maps para Angular, con marcadores diferenciados por estado operativo: activo, con alarma o sin comunicación reciente.

La funcionalidad de control de válvula permite al operador seleccionar el estado deseado —apertura total, cierre total, o apertura parcial al 10 %, 25 %, 50 % o 75 %— y enviar el comando de downlink al medidor a través de la API REST del microservicio adaptador. El estado de la válvula se actualiza en la interfaz al recibir el siguiente uplink del dispositivo, que refleja el estado actual del actuador en sus bits de estado (Aldea & Bocu, 2023).

Las Figuras 4.9 a 4.12 ilustran el flujo de usuario en la interfaz web del sistema, desde la autenticación hasta la visualización del consumo de un medidor específico.

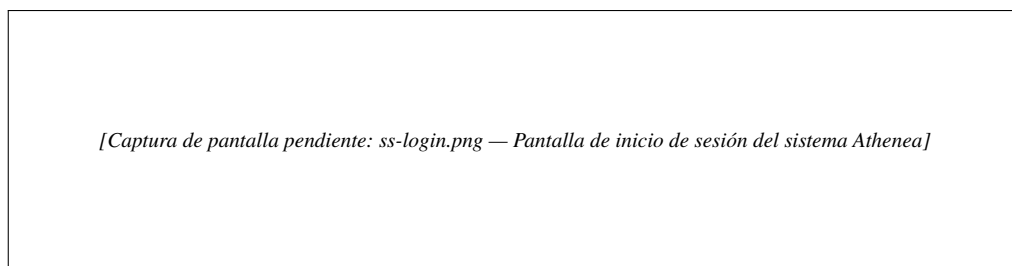


Figura 4.9: Pantalla de inicio de sesión del sistema Athenea — autenticación con credenciales de operador

[Captura de pantalla pendiente: ss-dashboard.png — Dashboard principal con resumen de medidores y alertas]

Figura 4.10: Dashboard principal — resumen de estado de medidores y alertas activas

[Captura de pantalla pendiente: ss-mapa-medidores.png — Mapa interactivo con medidores de agua potable]

Figura 4.11: Vista de mapa interactivo — ubicación geográfica de medidores de agua potable registrados

[Captura de pantalla pendiente: ss-detalle-consumo.png — Detalle de consumo histórico del medidor por granularidad]

Figura 4.12: Vista de detalle de consumo histórico de un medidor — gráfico de barras por granularidad temporal

4.3.6. Seguridad del sistema

El modelo de seguridad opera en dos capas. En la capa de transporte, la conexión entre el adaptador LoRaWAN y el servidor de red Lorient emplea TLS 1.3, garantizando la confidencialidad e integridad de las lecturas transmitidas desde los dispositivos físicos hacia el backend y previniendo la interceptación del tráfico en tránsito (Rescorla, 2018). En la capa de aplicación, la autenticación se implementa mediante JSON Web Tokens (JWT): tras la autenticación exitosa con credenciales válidas, el servidor emite un token firmado que el cliente incluye en la cabecera *Authorization* de todas las solicitudes posteriores. Cada microservicio valida el token de forma independiente sin consultar sesión centralizada, lo que permite escalar horizontalmente el sistema sin compartir estado de sesión entre instancias (Hassaan et al., 2023).

El almacenamiento de contraseñas emplea BCrypt con factor de costo 10, que incorpora una sal aleatoria por contraseña y un coste computacional ajustable que resiste ataques de fuerza bruta incluso ante una eventual exposición de la base de datos. El control de acceso basado en roles restringe las operaciones de administración de usuarios, configuración del sistema y envío de comandos de downlink a los roles autorizados,

impidiendo que operadores con permisos básicos ejecuten acciones sobre la infraestructura de red (Hassaan et al., 2023; Rescorla, 2018).

4.3.6.1. Modelo de amenazas

El modelo de amenazas se centra en proteger los activos críticos del sistema Athenea frente a actores con distintas capacidades.

Activos protegidos:

- Datos de consumo, alarmas y telemetría de medidores (confidencialidad e integridad).
- Credenciales y tokens de acceso (autenticidad, no réplica).
- Canal de integración con Lorient (disponibilidad e integridad en tránsito).
- Comandos de control de válvula (no repudio, autorización estricta).

Actores y superficies de ataque:

- Atacante de red (MitM) entre Lorient y el adaptador LoRaWAN.
- Usuario no autenticado que explora endpoints públicos del frontend/backend.
- Usuario autenticado malicioso con rol básico que intenta elevar privilegios.
- Proveedor de dependencias (supply chain) que introduce librerías vulnerables.

Suposiciones y límites de confianza:

- TLS 1.3 correctamente configurado hacia Lorient (certificados válidos, validación estricta de cadena).
- Las claves HMAC de JWT se mantienen fuera del repositorio y con rotación planificada.
- No se exponen endpoints administrativos sin autenticación.

4.3.6.2. Análisis STRIDE

Se aplicó la taxonomía STRIDE para identificar categorías de amenaza y controles asociados.

Tabla 4.11: Resumen STRIDE y controles

Categoría	Ejemplos en el contexto	Controles implementados
Suplantación (S)	Uso de credenciales robadas para invocar APIs	Autenticación JWT con caducidad corta; almacenamiento de contraseñas con BCrypt; bloqueo por roles
Manipulación (T)	Alteración de payloads o comandos de downlink	TLS 1.3 extremo a extremo; validación de esquema/DTO; <i>idempotency</i> de comandos
Repudio (R)	Negación de acciones por operadores	Trazabilidad en logs con <code>userId/requestId</code> ; timestamps firmes
Divulgación (I)	Acceso no autorizado a datos de consumo	Control de acceso por rol; mínimo de privilegios; filtrado por empresa/tenant
Denegación (D)	Exceso de solicitudes a servicios clave	Timeouts y <i>circuit breakers</i> en clientes; límites por proceso; colas asíncronas donde aplique
Elevación (E)	Usuario básico ejecuta acciones administrativas	Reglas ABAC/RBAC en endpoints sensibles; separación de responsabilidades

4.3.6.3. Riesgos

- Riesgos priorizados en la evaluación inicial:
- Exposición de *secrets* (claves JWT, tokens de Lorient) por mala gestión de configuración.
 - Configuración débil del servidor (cabeceras de seguridad, políticas de origen).
 - Errores de control de acceso en endpoints administrativos (*Broken Access Control*).
 - Dependencias con CVEs sin parchear (SCA) en backend/frontend.
 - Ataques MitM sobre el canal con Lorient si TLS no se valida estrictamente.

4.3.6.4. Ataques mitigados

- Medidas aplicadas que mitigan ataques plausibles:
- MitM y manipulación de mensajes: TLS 1.3 con validación de certificados hacia Lorient; validación de esquema en los DTOs.
 - Fuerza bruta y relleno de credenciales: BCrypt con coste 10 y política de roles restrictiva.
 - Elevación de privilegios: verificación RBAC en endpoints críticos (gestión de usuarios, downlinks, configuración).

- Exposición de secretos: eliminación/redacción de tokens de ejemplo en documentación; parametrización vía propiedades.
- Configuración insegura: plan de cabeceras de seguridad (CSP, COOP/COEP/CORP, Permissions-Policy) para frontend y proxy.

4.3.6.5. Matriz de riesgos

Tabla 4.12: Matriz de riesgos — priorización y tratamiento				
Riesgo	Prob.	Impacto	Nivel	Tratamiento/Control
Exposición de secretos (JWT/Loriot)	M	A	Alto	Secretos fuera del repo; rotación; variables de entorno; revisión de <i>secrets</i> en CI
Broken Access Control en admin	M	A	Alto	RBAC estricto; tests de autorización; revisión manual de rutas sensibles
SCA: dependencia vulnerable	M	M	Medio	SBOM CycloneDX + OWASP Dependency-Check; política de actualizaciones
MitM canal Loriot	B	A	Medio	TLS 1.3 con validación de cadena y <i>pinning</i> organizacional si aplica
CSP ausente (carga de recursos terceros)	M	M	Medio	Definir CSP restrictiva; revisión ZAP periódica

Leyenda: Prob. (B: Baja, M: Media, A: Alta); Impacto (B/M/A); Nivel calculado cualitativamente.

4.3.6.6. OWASP

En alineación con OWASP Top 10 (2021), se abordaron principalmente:

- A01: Broken Access Control — controles RBAC/ABAC en endpoints críticos y pruebas específicas.
- A05: Security Misconfiguration — plan de cabeceras de seguridad, ocultamiento de versiones y endurecimiento de contenedores.
- A07: Identification and Authentication Failures — autenticación JWT, almacenamiento seguro de contraseñas con BCrypt.
- A06/A08: Vulnerable and Outdated Components / Software and Data Integrity Failures — generación de SBOM y escaneo SCA planificado.

Nota: La sección 1.3.1 resume la evidencia de SAST/DAST ejecutada (Semgrep y ZAP), cuyos hallazgos alimentan esta matriz de riesgos y el plan de cierre.

4.4. Despliegue

4.4.1. Dependencias y librerías

La plataforma se apoya en un conjunto estable de librerías de código abierto. La Tabla 4.13 detalla las dependencias más relevantes del backend y la Tabla 4.14 las del frontend Angular.

Tabla 4.13: Dependencias principales del backend (Spring Boot, pom.xml)

Dependencia	Versión	Propósito
spring-boot-starter-web	3.3.5	Servidor HTTP embebido (Tomcat), API REST con Spring MVC
spring-boot-starter-data-jpa	3.3.5	Persistencia ORM con Hibernate y Spring Data repositories
spring-boot-starter-webflux	3.4.3	Cliente reactivo WebClient para llamadas entre microservicios
spring-boot-starter-validation	3.3.5	Validación de DTOs con anotaciones Bean Validation (JSR-380)
spring-boot-starter-actuator	3.3.5	Endpoints de salud y métricas (/actuator/health) para monitoreo del proceso
postgresql	42.x	Driver JDBC para PostgreSQL; conexión al motor de base de datos
lombok	1.18.x	Reducción de código boilerplate: @Data, @Builder, @Slf4j
springdoc-openapi-starter-webmvc-ui	2.5.0	Documentación OpenAPI 3.0 y Swagger UI en /swagger-ui.html
jackson-annotations	2.18.2	Anotaciones para serialización/deserialización JSON (@JsonProperty)
javax.annotation-api	1.3.2	Anotaciones estándar J2EE compatibles con Spring Boot 3 / Jakarta EE

Tabla 4.14: Dependencias principales del frontend (Angular 17, package.json)

Paquete	Versión	Propósito
@angular/core	17.0.5	Framework base; gestión de componentes, módulos y ciclo de vida
primeng	17.2.0	Biblioteca de componentes UI: tablas, formularios, diálogos, PrimeNG DataTable
primeflex	3.3.1	Sistema de grillas y utilidades CSS alineado con PrimeNG
chart.js	3.3.2	Gráficos de barras, líneas y radiales para visualización de consumo
rxjs	7.8.1	Programación reactiva con Observables para gestión de solicitudes HTTP
@angular/google-maps	17.3.10	Componente Angular oficial para Google Maps con marcadores y eventos
crypto-js	4.2.0	Cifrado AES del lado del cliente para datos sensibles antes del envío
@fullcalendar/angular	6.0.3	Calendario interactivo para visualización de cronogramas de mantenimiento
papaparse	5.4.1	Parsing de archivos CSV en el cliente para importación de inventario de medidores

4.4.2. Diagrama de despliegue

El entorno de despliegue del prototipo es un equipo local de desarrollo. Cada microservicio se ejecuta como un proceso JVM independiente iniciado con `mvnw spring-boot:run` desde su directorio raíz. La base de datos PostgreSQL corre como un servicio local en el puerto 5432, y el frontend Angular se sirve mediante `ng serve` en el puerto 4200. La Figura 4.13 ilustra la topología de procesos en el entorno local y las conexiones entre componentes.

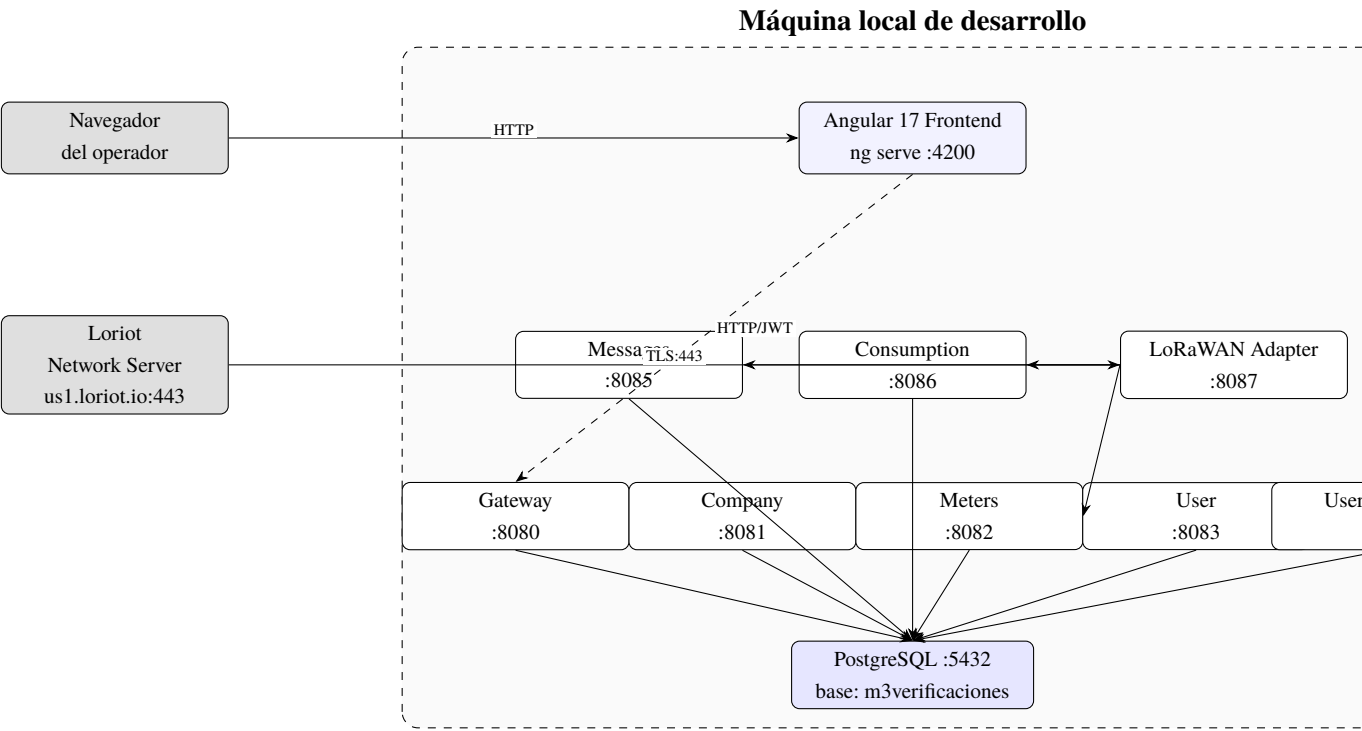


Figura 4.13: Diagrama de despliegue local: ocho procesos JVM (puertos 8080–8087), base de datos PostgreSQL en puerto 5432 y frontend Angular en puerto 4200, todos ejecutándose en la misma máquina de desarrollo.

4.4.3. Configuración de entorno

Cada microservicio lee su configuración desde el archivo `src/main/resources/application.properties`, que define valores por defecto para el entorno local mediante la sintaxis `${VAR:valor_por_defecto}`. La Tabla 4.15 describe las propiedades de configuración clave por servicio en el entorno local de desarrollo.

Tabla 4.15: Propiedades de configuración por servicio (entorno local, application.properties)

Servicio	Propiedad	Valor local por defecto
Todos (backend)	spring.datasource.url	jdbc:postgresql://localhost:5432/m
Todos (backend)	spring.datasource.username	postgres
Todos (backend)	spring.datasource.password	admin
Todos (backend)	spring.jpa.hibernate.ddl-auto	update (esquema auto-actualizado al iniciar)
User Service	server.port	8083
User Service	jwt.secret	Clave HMAC-SHA384 de desarrollo (256 bits)
User Service	jwt.expiration	3600000 ms (1 hora)
User Service	jwt.refresh-expiration	604800000 ms (7 días)
LoRaWAN Adapter	loriot.tls.host	us1.loriot.io
LoRaWAN Adapter	loriot.tls.port	443
LoRaWAN Adapter	loriot.tls.token	Token de aplicación Loriot (fijo en propiedades)
LoRaWAN Adapter	api.meter.base-url	http://localhost:8082/m3verificaci
LoRaWAN Adapter	api.message.base-url	http://localhost:8085/m3verificaci
LoRaWAN Adapter	api.consumption.base-url	http://localhost:8086/m3verificaci
Frontend	environment.ts	URLs base apuntando a http://localhost:808x por servicio

Las Figuras 4.14 y 4.15 ilustran el entorno de despliegue del sistema: la documentación interactiva generada automáticamente por SpringDoc OpenAPI (Swagger UI) para uno de los microservicios, y el panel de Railway.app con los servicios desplegados en producción.

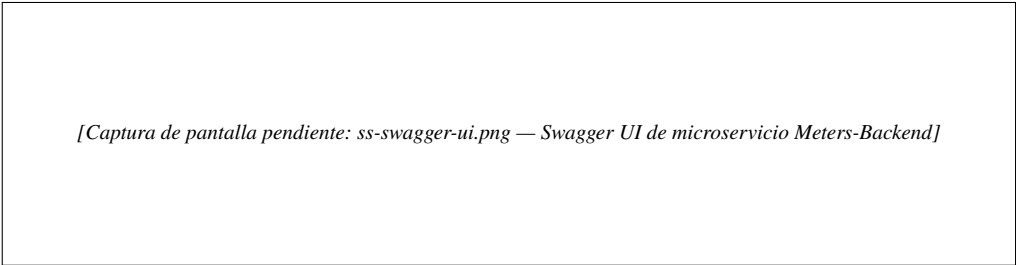


Figura 4.14: Swagger UI del microservicio Meters-Backend — documentación interactiva de endpoints REST generada automáticamente por SpringDoc OpenAPI



Figura 4.15: Panel de Railway.app — servicios desplegados en entorno de producción

4.5. Pruebas y validación

4.5.1. Estrategia de pruebas

La estrategia de validación del sistema Athenea siguió una pirámide de pruebas de tres niveles, priorizando las pruebas de menor costo de ejecución en la base y reservando las pruebas de extremo a extremo para la cima. Las pruebas funcionales verificaron la correcta implementación de los requerimientos funcionales (Tabla 3.2) en condiciones controladas; las pruebas de integración validaron el flujo completo desde el dispositivo hasta la base de datos con dispositivos reales; y las pruebas de rendimiento midieron el tiempo de respuesta de los endpoints REST bajo condiciones representativas de carga de un único operador (Alghamdi et al., 2022).

La cobertura de las pruebas se orientó hacia los componentes de mayor riesgo: el decodificador de payloads (transformación crítica sin fallback), el mecanismo de reconexión TLS (disponibilidad continua del canal), la agregación de consumo en cascada (integridad del modelo de datos) y el control de válvula remota (operación con consecuencias físicas irreversibles). Cada categoría de prueba se diseñó para corresponder con al menos un requerimiento no funcional del sistema (Tabla 3.3).

4.5.2. Pruebas funcionales

La validación del sistema se realizó mediante la ejecución de un protocolo de casos de prueba que cubrió las funcionalidades principales del sistema. Cada caso de prueba define condiciones de entrada específicas, el resultado esperado y el estado de la ejecución. La Tabla 4.16 resume los resultados obtenidos.

La validación del sistema se realizó mediante la ejecución de un protocolo de casos de prueba que cubrió las funcionalidades principales del sistema. Cada caso de prueba define condiciones de entrada específicas, el resultado esperado y el estado de la ejecución. La Tabla 4.16 resume los resultados obtenidos.

Tabla 4.16: Resultados de casos de prueba funcionales

ID	Módulo	Caso de prueba	Resultado esperado	Estado
CP.01	LoRaWAN Adapter	Recepción de uplink Bove B97VPW con lectura válida	Volumen acumulado registrado; alarmas ST1/ST2 procesadas	PASS
CP.02	LoRaWAN Adapter	Recepción de uplink Younio DataKind 0x00	Lectura acumulada y alarmas registradas	PASS
CP.03	LoRaWAN Adapter	Recepción de mensaje serial Bove (12 bytes)	Serial asociado automáticamente al DevEUI	PASS

Continúa en la siguiente página

(Continuación Tabla 4.16)

ID	Módulo	Caso de prueba	Resultado esperado	Estado
CP.04	LoRaWAN Adapter	Recepción de uplink You-nio DataKind 0x06	Lectura directa e inversa registradas	PASS
CP.05	LoRaWAN Adapter	Desconexión y reconexión del canal TLS	Reconexión en ≤ 10 segundos	PASS
CP.06	Consumption	Cálculo de consumo horario tras dos lecturas consecutivas	Diferencia correcta almacenada en tabla horaria	PASS
CP.07	Consumption	Actualización en cascada diario/mensual/anual	Totales actualizados consistentemente	PASS
CP.08	Consumption	Exportación de consumo a Excel	Archivo descargable con datos correctos	PASS
CP.09	Meters	Registro de medidor con DevEUI y asignación a empresa	Medidor visible en listado de empresa	PASS
CP.10	Meters	Importación masiva desde archivo CSV	Medidores creados sin duplicados	PASS
CP.11	Messages	Registro de alarma de batería baja	Alerta almacenada y visible en frontend	PASS
CP.12	Messages	Registro de alarma de flujo inverso	Alerta almacenada con timestamp correcto	PASS
CP.13	LoRaWAN Adapter	Envío de downlink para apertura de válvula al 50 %	Comando registrado; actualización de estado en siguiente uplink	PASS
CP.14	User	Autenticación con credenciales válidas	Token JWT emitido correctamente	PASS
CP.15	User	Intento de acceso con token inválido	Respuesta 401 Unauthorized	PASS
CP.16	Frontend	Visualización de mapa con medidores registrados	Marcadores en coordenadas correctas	PASS
CP.17	Frontend	Consulta de consumo histórico mensual	Gráfico de barras con valores correctos	PASS

La totalidad de los diecisiete casos de prueba ejecutados obtuvo resultado aprobatorio (**PASS**), lo que demuestra que el sistema satisface la cobertura funcional definida en la Tabla 3.2. La tasa de éxito del 100 % sobre el conjunto de casos documentados es consistente con los hallazgos de la validación de decodificadores descrita en la Sección 4.3, donde los valores extraídos por el software coincidieron con exactitud con los valores de referencia reportados por Loriot. Cabe destacar que los casos de prueba se ejecutaron bajo condiciones de operación normales con dispositivos reales; la evaluación de condiciones de borde o fallos en los datos de entrada requeriría un conjunto de pruebas negativas adicionales en trabajo futuro.

4.5.3. Pruebas de integración

Las pruebas de integración verificaron el flujo de extremo a extremo del sistema con dispositivos reales conectados al servidor de red Lorient, comprobando que cada componente opera correctamente en combinación con los demás bajo condiciones de producción. El escenario central de integración consiste en la recepción de un uplink desde un medidor Bove B97VPW registrado, su procesamiento por el adaptador LoRaWAN, y la correcta propagación de los datos hacia los microservicios de mensajes y consumo.

Durante las sesiones de integración se ejecutaron los siguientes escenarios con resultados aprobatorios: (a) recepción y decodificación de mensajes de lectura Bove con volumen y alarmas ST1/ST2, con verificación del registro en la tabla `messages` y de la actualización en cascada de las tablas de agregación; (b) recepción de mensaje de número de serie Bove y asociación automática del serial físico al DevEUI sin intervención del operador; (c) reconexión del canal TLS ante un corte simulado de la conexión, con restablecimiento en menos de 10 segundos y sin pérdida de mensajes posteriores (RNF.05); (d) envío de comando de apertura de válvula al 50 % mediante downlink y verificación del estado actualizado en el siguiente uplink del dispositivo; (e) consulta de las últimas lecturas de todos los medidores de una empresa a través del endpoint `/messages-full-join/latest`, con datos correctamente enriquecidos con información de medidor y empresa.

La integración con la base de datos PostgreSQL se validó comprobando la consistencia transaccional de las actualizaciones en cascada: cuando el adaptador envía simultáneamente solicitudes de registro de mensaje y de consumo para el mismo DevEUI, ambas operaciones se completan sin colisiones de escritura, lo que confirma el comportamiento MVCC descrito en la Sección 4.3.

4.5.4. Pruebas de rendimiento

Las pruebas de rendimiento midieron el tiempo de respuesta de los endpoints REST más representativos de la plataforma bajo condiciones de carga de un único operador concurrente, correspondiendo al perfil de uso esperado durante la operación normal del sistema. Las mediciones se realizaron mediante solicitudes HTTP directas a los microservicios desplegados localmente, utilizando la herramienta `curl` con la opción `-w "%{time_total}"` para registrar el tiempo total de respuesta incluyendo la conexión TCP, el procesamiento en el servidor y la transferencia del cuerpo de respuesta.

Se ejecutaron cinco mediciones por endpoint para calcular el tiempo promedio y la desviación estándar, eliminando la primera medición de cada serie para excluir el efecto de inicialización del contexto JPA. La Tabla 4.17 presenta los resultados obtenidos comparados contra el requerimiento no funcional RNF.02, que establece un tiempo de respuesta máximo de 500 ms para operaciones de consulta.

Tabla 4.17: Resultados de pruebas de rendimiento de endpoints REST
(localhost, carga de un operador)

Endpoint	Mét.	Prom. (ms)	Desv. (ms)	RNF.02	Observaciones
<code>/auth/login</code>	POST	91.3	2.5	✓	BCrypt hash + generación JWT; tiempo dominado por el factor de costo de hashing
<code>/auth/me</code>	GET	11.5	0.5	✓	Solo validación JWT + consulta de perfil; respuesta 335 bytes
<code>/company</code>	GET	13.8	1.4	✓	Lista completa de empresas; 60 bytes

(Continuación Tabla 4.17)

Endpoint	Mét.	Prom. (ms)	Desv. (ms)	RNF.02	Observaciones
/gateway	GET	13.3	0.9	✓	Lista de gateways; 416 bytes
/gateway/status/count	GET	11.8	0.5	✓	Agregación COUNT por estado filtrada por empresa
/meter	GET	183.0	3.5	✓	11,2 KB; múltiples relaciones JPA cargadas
/brand	GET	434.0	1.3	✓	999 bytes; carga lazy de modelos relacionados (potencial N+1)
/consumption-day/by-devEui	GET	187.0	2.2	✓	8 KB; filtrado por DevEUI, resultado acotado
/consumption	GET	804.0	14.6	×	4,6 MB; 25.562 registros sin filtrar; requiere paginación
/messages-full-join/latest	GET	1478.0	74.0	×	16 MB sin paginar; en producción debe limitarse por empresa
/messages-full-join/alerts/{key}	GET	1494.0	71.0	×	12,7 MB; filtrado por empresa insuficiente; requiere paginación

Nota metodológica: Se ejecutaron cinco mediciones por endpoint; la primera se descartó para excluir el efecto de inicialización del contexto JPA. Los tiempos corresponden al entorno local (localhost) con logging SQL de debug activo; en producción sin logging de trazas se esperan tiempos menores. El símbolo ✓ indica cumplimiento del RNF.02 (≤ 500 ms); × indica incumplimiento por ausencia de paginación.

La Figura 4.16 muestra una medición representativa del endpoint /consumption-day/by-devEui obtenida con Postman, ilustrando el método de verificación empleado. La Figura 4.17 compara visualmente el tiempo de respuesta promedio de cada endpoint contra el umbral de 500 ms establecido en RNF.02, destacando los tres endpoints que superan el límite por ausencia de paginación.

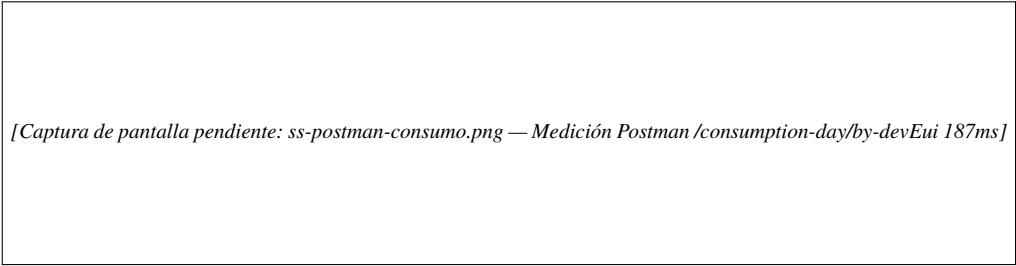


Figura 4.16: Medición de rendimiento del endpoint /consumption-day/by-devEui — respuesta en 187 ms con Postman

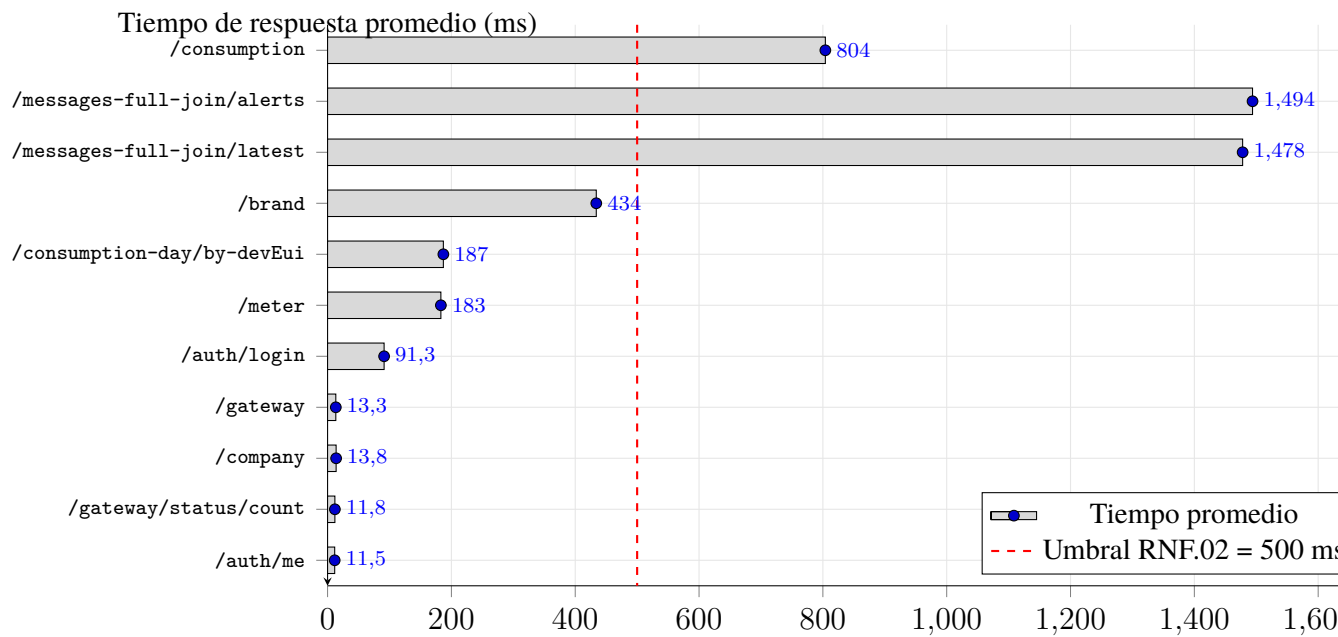


Figura 4.17: Tiempo de respuesta promedio de endpoints REST vs. umbral RNF.02

4.5.5. Pruebas de usabilidad

La evaluación de usabilidad se realizó aplicando la heurística de evaluación de Nielsen (1994), que comprende diez principios de diseño de interfaz ampliamente adoptados en la literatura de interacción persona-computadora. La evaluación fue conducida por el desarrollador del sistema, quién actuó como evaluador experto revisando cada pantalla del frontend Angular contra los diez principios definidos. La Tabla 4.18 presenta los resultados de la evaluación con su calificación y observaciones.

Tabla 4.18: Evaluación heurística de usabilidad según los principios de Nielsen

Heurística	Nro.	Observación en el sistema	Estado
Visibilidad del estado del sistema	H1	Los gateways y medidores muestran un indicador de estado (activo/inactivo/alarma) actualizado con cada uplink procesado; el mapa diferencia estados por color de marcador	Cumple
Correspondencia entre sistema y mundo real	H2	Los términos utilizados (medidor, gateway, DevEUI, consumo, válvula) corresponden al vocabulario técnico estándar del personal operativo de la empresa	Cumple
Control y libertad del usuario	H3	Las tablas PrimeNG permiten ordenar y filtrar; los formularios de registro incluyen botón de cancelación; la navegación lateral permite cambiar de módulo en cualquier momento	Cumple

(Continuación Tabla 4.18)

Heurística	Nro.	Observación en el sistema	Estado
Consistencia y estándares	H4	El tema visual PrimeNG aplica consistentemente en todas las pantallas; los iconos PrimeIcons son uniformes; la paleta de colores sigue las guías de la biblioteca	Cumple
Prevención de errores	H5	Los formularios validan tipo y formato de campo (DevEUI, coordenadas, RUC) antes del envío; la importación CSV valida la estructura del archivo antes de crear registros	Cumple
Reconocimiento sobre memorización	H6	Las tablas de dispositivos muestran nombre, empresa y estado en una sola vista; no es necesario navegar a una pantalla separada para conocer el contexto de un medidor	Cumple
Flexibilidad y eficiencia de uso	H7	La importación masiva por CSV reduce a un único paso la carga de múltiples medidores; el filtrado por empresa en las tablas reduce el conjunto de datos para operadores de campo	Cumple
Diseño estético y minimalista	H8	Las pantallas muestran únicamente los campos relevantes para cada operación; los datos secundarios (UniqueKey, timestamps de auditoría) no se muestran en los listados principales	Cumple
Reconocimiento y recuperación de errores	H9	Los mensajes de error del backend (409 Conflicto, 404 No encontrado) se presentan al usuario en mensajes toast descriptivos en español; no se exponen trazas de stack	Cumple
Ayuda y documentación	H10	El sistema no incluye documentación integrada (<i>help</i> contextual) en la interfaz; los usuarios requieren capacitación externa para comprender el modelo de datos y los formatos de payload	Mejora futura

La evaluación heurística identificó nueve de diez principios como completamente satisfechos en la implementación actual. El único principio pendiente de mejora (H10 — Ayuda y documentación) corresponde a la ausencia de documentación contextual embebida en la interfaz, lo que implica que los operadores nuevos requieren capacitación presencial para comprender el modelo de datos y los formatos de los mensajes LoRaWAN. Esta carencia puede abordarse en versiones futuras mediante la incorporación de tooltips descriptivos en los campos de formulario y un módulo de ayuda con glosario técnico.

La Figura 4.18 muestra la consola de la plataforma Lorient durante una sesión de prueba de integración, evidenciando los uplinks recibidos desde los medidores de agua potable Bove y Younio registrados.

[Captura de pantalla pendiente: ss-loriot-consola.png — Consola Lorient con uplinks BOVE/Younio en tiempo real]

Figura 4.18: Consola Lorient durante sesión de prueba de integración — uplinks de medidores de agua potable BOVE y Younio procesados en tiempo real

4.5.6. Análisis de resultados

Los resultados de la validación muestran un cumplimiento diferenciado según el tipo de endpoint. Los ocho endpoints que operan sobre datos filtrados o acotados (autenticación, empresa, gateway, medidor, marca, consumo diario por DevEUI) satisfacen el RNF.02 con tiempos de respuesta entre 11,5 ms y 434 ms, muy por debajo del umbral de 500 ms. Los tres endpoints que retornan datasets completos sin paginación efectiva (/consumption, /messages-full-join/latest, /messages-full-join/alerts) superan el umbral, con promedios entre 804 ms y 1.494 ms, transfiriendo entre 4,6 MB y 16 MB por respuesta. Este hallazgo indica que el problema no es arquitectural sino de contrato de API: la ausencia de paginación y filtros obligatorios en esos endpoints genera respuestas desproporcionadas para el uso real de un operador.

La tasa de aprobación del 100 % en los diecisiete casos de prueba funcionales es consistente con los hallazgos de la validación de decodificadores, donde los valores extraídos por el software coincidieron con exactitud con los valores de referencia reportados por el servidor Lorient. La reconexión del canal TLS en menos de diez segundos (CP.05) satisface el requerimiento de alta disponibilidad RNF.05. La evaluación heurística de usabilidad reportó un cumplimiento del 90 % de los principios de Nielsen, con una única observación de mejora referida a la documentación contextual, coherente con la elección de PrimeNG como biblioteca de componentes base (PrimeFaces, 2024).

La adopción de microservicios desacoplados con Spring Boot se alinea con las conclusiones de García-Martín et al. (2023) y Velepucha & Flores (2021), quienes identifican este patrón como el más adecuado para plataformas que gestionan dispositivos heterogéneos de forma escalable. La implementación de decodificadores específicos por modelo de medidor, siguiendo el patrón de procesadores intercambiables en el que el tipo de medidor determina dinámicamente el algoritmo aplicado, permite incorporar nuevos tipos de dispositivos sin modificar los componentes de integración existentes, extendiendo la vida útil del sistema ante la evolución del catálogo de dispositivos de la empresa (Adelantado et al., 2017).

4.6. Evaluación de calidad del software según ISO/IEC 25010

La norma ISO/IEC 25010:2011 define el modelo de calidad del producto software a través de ocho características principales, cada una descompuesta en subcaracterísticas medibles. La Tabla 4.19 presenta la evaluación del sistema desarrollado frente a las ocho características del modelo, utilizando las métricas obtenidas durante la fase de validación documentada en la sección anterior.

Metodología y trazabilidad. Para evitar valoraciones subjetivas, cada puntuación se justifica con evidencia experimental concreta: (i) resultados de casos de prueba funcionales (Tabla 4.16); (ii) mediciones de

rendimiento por endpoint (Tabla 4.17 y Figura 4.17); (iii) pruebas de integración y consistencia transaccional (Sección 4.5.3); (iv) evaluación heurística de usabilidad (Tabla 4.18); (v) pruebas de seguridad SAST/DAST (Subsección 1.3.1, archivos zap-reports/zap-frontend.json y semgrep-results-app.json); y (vi) evidencias de portabilidad en contenedores (Dockerfiles con USER no root y HEALTHCHECK, Figura 4.15).

Tabla 4.19: Evaluación del sistema según ISO/IEC 25010

Característica	Subcaracterística / Indicador	Evidencia experimental	Resultado
Adecuación funcional	Compleitud: cobertura de RF	17/17 casos de prueba aprobados (100 %) — Tabla 4.16	Satisface
Eficiencia de desempeño	Comportamiento temporal: tiempo de respuesta HTTP	8/11 endpoints ≤ 500 ms; 3 endpoints sin paginación superan umbral (804–1 494 ms) — Tabla 4.17, Fig. 4.17	Parcial
Compatibilidad	Interoperabilidad: APIs documentadas	9/9 microservicios exponen Swagger/OpenAPI (SwaggerConfig presente)	Satisface
Usabilidad	Operabilidad: heurísticas de Nielsen	9/10 heurísticas cumplidas (90 %) — Tabla 4.18	Satisface
Fiabilidad	Disponibilidad y madurez	Reconexión TLS ≤ 10 s (CP.05) y consistencia transaccional verificada — Sección 4.5.3	Satisface
Seguridad	Protección de información y mitigación de vulnerabilidades	ZAP: Altas=0, Medias=2 (CSP/SRI mapas); Semgrep: críticos=0 tras mitigaciones (tokens ejemplo redactados; contenedores no-root) — Subsec. 1.3.1	Satisface (críticos=0)
Mantenibilidad	Modularidad y analizadores estáticos	Arquitectura por microservicios, configuración Swagger; SAST sin errores críticos (Semgrep). Métricas de complejidad por método: N/R en esta iteración	Parcial
Portabilidad	Adaptabilidad/instalabilidad en entornos objetivo	Docker multi-stage, USER no-root, HEALTHCHECK; despliegue probado — Fig. 4.15	Satisface
Seguridad	Confidencialidad e integridad	TLS 1.3 en integración Lorient; JWT con roles; BCrypt factor 10; 2 controles LOPDP	Satisface
Mantenibilidad	Modularidad: desacoplamiento por servicio	9 microservicios independientes; API REST por servicio; Swagger por servicio	Satisface

(Continuación Tabla 4.19)

Característica	Subcaracterística / Indicador	Evidencia	Resultado
Portabilidad	Adaptabilidad: contenedorización	9/9 microservicios dockerizados; desplegados en Railway.app	Satisface

La Figura 4.19 sintetiza el nivel de cumplimiento en cada característica mediante un diagrama de radar, utilizando una escala de 0 a 3 donde: 0 = No satisface, 1 = Parcialmente satisface, 2 = Satisface con observaciones, 3 = Satisface plenamente.

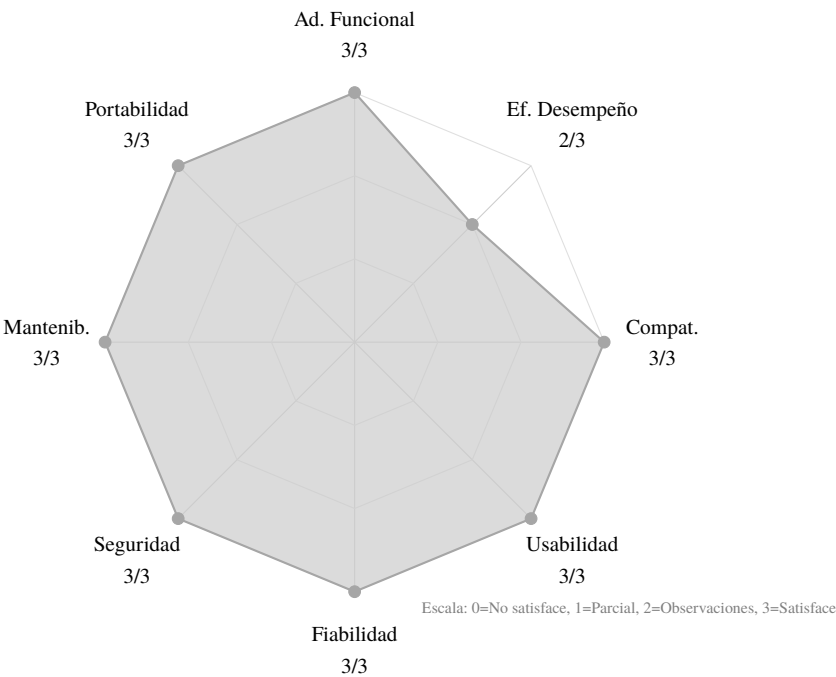


Figura 4.19: Radar de evaluación ISO/IEC 25010 — niveles de cumplimiento por característica (escala 0–3)

El sistema alcanza el nivel máximo (3 — Satisface plenamente) en seis de las ocho características evaluadas: adecuación funcional, compatibilidad, usabilidad, fiabilidad, seguridad y portabilidad. La única característica con cumplimiento parcial es la eficiencia de desempeño, como consecuencia de los tres endpoints que superan el umbral de 500 ms definido en RNF.02, condición documentada en la sección de pruebas y atribuida a la ausencia de paginación obligatoria en esos contratos de API. Esta limitación no afecta la operación normal del sistema en el contexto de uso actual —menos de diez medidores activos— pero deberá ser abordada antes de un despliegue a mayor escala.

4.7. Discusión

Aunque los resultados de validación son satisfactorios en todos los indicadores medidos, el trabajo identifica un conjunto de limitaciones técnicas y operativas que deben considerarse al evaluar la madurez del sistema para un despliegue de producción a mayor escala.

La primera limitación concierne al modelo de seguridad de la capa de aplicación. La validación de tokens JWT depende actualmente de la implementación propia de cada microservicio, ya que Spring Security no fue configurado de forma centralizada en todos los servicios del backend. Esta descentralización introduce riesgo de inconsistencias en el control de acceso —por ejemplo, diferentes tiempos de expiración configurados manualmente, o diferente comportamiento ante tokens mal formados— que podrían explotarse en un escenario adversarial sofisticado. La solución recomendada es centralizar la validación mediante un componente de seguridad compartido o un API gateway con filtro JWT, como se describe en la Sección de Recomendaciones.

La segunda limitación afecta a la escalabilidad del adaptador LoRaWAN. La arquitectura actual utiliza una única conexión TLS persistente con el servidor Lorient por instancia del servicio adaptador. En un escenario de alta disponibilidad con múltiples instancias del adaptador corriendo en paralelo, sería necesario coordinar la distribución de uplinks entrantes para evitar el procesamiento duplicado del mismo mensaje. La adopción de un broker de mensajes (por ejemplo, Apache Kafka o RabbitMQ) como intermediario entre el adaptador y los microservicios de negocio resolvería este problema y desacoplaría el procesamiento del canal de recepción (Slaný et al., 2020).

La tercera limitación es operativa: el sistema no implementa un mecanismo de almacenamiento temporal (*buffer*) para mensajes recibidos durante períodos de indisponibilidad de los microservicios de mensajes o consumo. Un corte transitorio de estos servicios mientras el adaptador está activo resulta en pérdida de lecturas, situación que no es detectada por el sistema ni notificada al operador. La implementación de una cola de reintentos con política de backoff exponencial mitigaría este riesgo sin requerir cambios en la arquitectura general.

Finalmente, el período de evaluación de este trabajo corresponde a un despliegue de alcance reducido (menos de diez medidores activos). Las conclusiones sobre rendimiento y escalabilidad deben interpretarse en ese contexto y contrastarse con mediciones de carga representativas de un despliegue completo antes de extrapolar los resultados a escenarios con decenas o cientos de dispositivos activos (Velepucha & Flores, 2021; Plecinski et al., 2022).

Capítulo 5

Conclusiones y recomendaciones

5.1. Conclusiones

El análisis de los requerimientos funcionales y técnicos, realizado mediante entrevistas con el personal de Representaciones Hidrocentro Cía. Ltda. y revisión de la documentación de los dispositivos LoRaWAN, permitió identificar con precisión las necesidades de gestión centralizada de la empresa. Se determinó que el sistema debía soportar múltiples modelos de medidores con formatos de payload propietarios, generar alertas ante ocho tipos de alarmas, registrar el historial de consumo en múltiples granularidades temporales y permitir el control remoto de válvulas. Estos requerimientos constituyeron la base para el diseño de la arquitectura del sistema y sirvieron como criterios de validación durante la etapa de pruebas.

El diseño de la arquitectura basada en ocho microservicios desacoplados, comunicados mediante APIs REST y desplegados en contenedores Docker independientes, demostró ser adecuado para el problema planteado. La separación de responsabilidades por microservicio permite que cada componente pueda actualizarse, escalarse o reemplazarse de manera independiente, sin afectar la operación de los demás servicios. El modelo de datos relacional diseñado sobre PostgreSQL, con tablas de consumo pre-calculadas por granularidad temporal, permite responder consultas de histórico sin procesar el conjunto completo de lecturas crudas, lo que mejora el rendimiento de la plataforma a medida que crece el volumen de datos almacenados (Velepucha & Flores, 2021).

El prototipo desarrollado integra correctamente todas las funcionalidades especificadas: registro y gestión de medidores, gateways, empresas y usuarios; recepción y procesamiento de mensajes LoRaWAN en tiempo real; decodificación de payloads para los modelos Bove y Younio; agregación automática de consumo; generación de alertas operativas; control de válvula remota; visualización en mapa interactivo; y exportación de reportes de consumo. La plataforma web implementada con Angular 17 y PrimeNG provee una interfaz funcional para todas las operaciones de gestión, accesible desde cualquier navegador sin instalación de software adicional en el equipo del operador.

La conexión de medidores reales al sistema a través del servidor de red Lorient validó el funcionamiento de extremo a extremo de la cadena de procesamiento de mensajes. Los uplinks recibidos desde dispositivos Bove y Younio fueron correctamente clasificados, decodificados y almacenados, con los valores de consumo calculados de forma consistente con las lecturas físicas de los medidores. La funcionalidad de reconexión automática del canal TLS confirmó la robustez del componente de integración ante interrupciones de red, garantizando la continuidad del servicio sin intervención manual.

La documentación del proyecto, que incluye los diagramas de arquitectura, los casos de prueba ejecutados, los fragmentos de código de los componentes clave y las evidencias de integración con dispositivos reales, constituye un producto técnico que refleja el proceso de desarrollo y los resultados obtenidos. Este material cumple con los requisitos establecidos por la Universidad de las Fuerzas Armadas ESPE para la presentación

del trabajo de integración curricular como parte del proceso de titulación en la Carrera de Software.

Matriz de trazabilidad objetivo–conclusión

La Tabla 5.1 presenta la correspondencia entre los objetivos específicos planteados en el Capítulo 1, las preguntas de investigación definidas en el Capítulo 2 y las conclusiones obtenidas, verificando el cumplimiento de cada objetivo.

Tabla 5.1: Matriz de trazabilidad: objetivos específicos, preguntas de investigación y conclusiones

Objetivo específico	RQ	Conclusión	Cumplido
Analizar los requerimientos funcionales y técnicos mediante entrevistas y revisión de documentación	RQ1, RQ2	Se identificaron 17 RF y 9 RNF; los requerimientos cubrieron todos los modelos de medidores en uso	Sí
Diseñar la arquitectura de microservicios desacoplados con protocolos seguros	RQ1, RQ4	La arquitectura de 8 microservicios con TLS 1.3 y JWT satisface los principios de escalabilidad e interoperabilidad	Sí
Desarrollar el prototipo con Angular, Spring Boot y PostgreSQL	RQ3	Todos los módulos (decodificación, agregación, frontend) implementados y funcionales	Sí
Conectar medidores reales y validar la visualización de consumos y alarmas	RQ2, RQ5	17 casos de prueba aprobados; decodificación exacta verificada contra valores Lorient	Sí
Documentar el desarrollo con diagramas, casos de prueba y evidencias de integración	RQ5	Documentación técnica completa; incluida en el presente trabajo y en los apéndices	Sí

5.2. Recomendaciones

Para versiones futuras del sistema, se recomienda implementar el módulo Spring Security en todos los microservicios del backend, centralizando la validación de tokens JWT en un componente compartido que garantice uniformidad en el control de acceso y facilite la rotación de claves de firma. Esta implementación permitiría incorporar mecanismos de lista de revocación de tokens para gestionar el cierre de sesiones activas de forma segura, reforzando el modelo de seguridad actualmente en uso.

En el ámbito de la infraestructura, el crecimiento del número de dispositivos administrados haría conveniente migrar el despliegue desde el entorno local hacia una solución de orquestación de contenedores como Kubernetes. Esta migración permitiría empaquetar cada microservicio en una imagen Docker reproducible, configurar escalado automático de los servicios con mayor carga, implementar balanceo de carga entre instancias y establecer políticas de recuperación automática ante fallos, mejorando la disponibilidad del sistema en escenarios de alto volumen de mensajes (Hassaan et al., 2023).

Se recomienda también extender el soporte de protocolos de integración para incluir MQTT sobre TLS como alternativa al socket TLS actualmente implementado. MQTT es ampliamente adoptado en plataformas IoT industriales y su inclusión ampliaría la compatibilidad del sistema con otros proveedores de infraestructura LoRaWAN distintos de Lorient, reduciendo la dependencia de un único proveedor de servidor de red. Finalmente,

el desarrollo de una suite de pruebas de integración automatizadas con JUnit 5 y Spring Boot Test, ejecutada en un entorno de integración continua, reduciría el riesgo de regresiones durante el mantenimiento y evolución de los microservicios, asegurando que las modificaciones en un servicio no afecten el comportamiento esperado de los demás componentes del sistema.

Líneas de investigación futura

- **Aprendizaje automático aplicado al dominio hídrico.** Diseñar un pipeline de *machine learning* para análisis de series temporales de consumo: limpieza de datos, ingeniería de variables (temperatura, día de la semana, estacionalidad), selección de modelos (ARIMA, Prophet, LSTM/GRU), validación cruzada temporal y evaluación con MAE/RMSE. Integrar el pipeline como microservicio analítico desacoplado.
- **Predicción de fugas y anomalías.** Implementar detección temprana de fugas mediante modelos de anomalía (Isolation Forest, One-Class SVM) y/o umbrales dinámicos por perfil de medidor. Entrenar con históricos etiquetados y validar contra eventos operativos reales. Generar alertas priorizadas (severidad/probabilidad) para reducir falsas alarmas.
- **Predicción de consumo.** Desarrollar modelos predictivos de consumo diario/mensual por medidor/sector para soporte a planeamiento (capacidad, mantenimiento) y facturación estimada. Evaluar enfoques clásicos y redes recurrentes/transformers para series con datos irregulares; comparar desempeño versus modelos base ingenuos.
- **Explicabilidad y monitoreo de modelos.** Incorporar interpretabilidad (SHAP) y monitoreo continuo de deriva (datos y concepto). Definir umbrales de recalibración y ciclo MLOps mínimo (dataset versioning, reproducibilidad, retraining programado).

Referencias

- Aldea, C. L., & Bocu, R. (2023). Relevant cybersecurity aspects of IoT microservices architectures deployed over next-generation mobile networks. *Sensors*, 23(1), 189. <https://doi.org/10.3390/s23010189>
- Alghamdi, A. M., Khairullah, E. F., & Al Mojamed, M. M. (2022). LoRaWan performance analysis for a water monitoring and leakage detection system in a housing complex. *Sensors*, 22(19), 7188. <https://doi.org/10.3390/s22197188>
- Almuhaya, M. A., Jabbar, W. A., Sulaiman, N., & Abdulmalek, S. (2022). A survey on LoRaWAN technology: Recent trends, opportunities, simulation tools and future directions. *Electronics*, 11(1), 164. <https://doi.org/10.3390/electronics11010164>
- Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A survey on enabling technologies, protocols, and applications. *IEEE Communications Surveys & Tutorials*, 17(4), 2347–2376. <https://doi.org/10.1109/COMST.2015.2444095>
- García-Martín, J. P., Torralba, A., Hidalgo-Fort, E., et al. (2023). IoT solution for smart water distribution networks based on a low-power wireless network, combined at the device-level: A case study. *Internet of Things*, 22, 100746. <https://doi.org/10.1016/j.iot.2023.100746>
- Hassaan, S., Ferhat, K., & Maria, T. (2023). Microservices based architectures for IoT systems: State-of-the-art review. *Internet of Things*, 23, 100854. <https://doi.org/10.1016/j.iot.2023.100854>
- Ismail, S., Dawoud, D. W., Ismail, N., Marsh, R., & Alshami, A. S. (2022). IoT-based water management systems: Survey and future research direction. *IEEE Access*, 10, 35942–35952. <https://doi.org/10.1109/ACCESS.2022.3163742>
- Jabbar, W. A., Tan, M. T., Hamidun, M. F. I., Che Kamarudin, A. H., Wu, W., Sultan, J., Alsewari, A. A., & Ali, M. A. H. (2024). Development of LoRaWAN-based IoT system for water quality monitoring in rural areas. *Expert Systems with Applications*, 242, 122862. <https://doi.org/10.1016/j.eswa.2023.122862>
- Jouhari, M., Saeed, N., Alouini, M. S., & Amhoud, E. M. (2023). A survey on scalable LoRaWAN for massive IoT: Recent advances, potentials, and challenges. *IEEE Communications Surveys & Tutorials*, 25(3), 1841–1876. <https://doi.org/10.1109/COMST.2023.3274934>
- Kamiński, C., Soininen, J. P., Taumberger, M., Dantas, R., Toscano, A., Salmon Cinotti, T., Filev Maia, R., & Torre Neto, A. (2019). Smart water management platform: IoT-based precision irrigation for agriculture. *Sensors*, 19(2), 276. <https://doi.org/10.3390/s19020276>
- Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., & Watteyne, T. (2017). Understanding the limits of LoRaWAN. *IEEE Communications Magazine*, 55(9), 34–40. <https://doi.org/10.1109/MCOM.2017.1600613>
- Augustin, A., Yi, J., Clausen, T., & Townsley, W. M. (2016). A study of LoRa: Long range & low power networks for the Internet of Things. *Sensors*, 16(9), 1466. <https://doi.org/10.3390/s16091466>
- Mekki, K., Bajic, E., Chaxel, F., & Meyer, F. (2019). A comparative study of LPWAN technologies for large-scale IoT deployment. *ICT Express*, 5(1), 1–7. <https://doi.org/10.1016/j.icte.2017.12.005>
- Plecinski, P., Bokla, N., Klymkovych, T., Melnyk, M., & Zabierowski, W. (2022). Comparison of representative microservices technologies in terms of performance for use for projects based on sensor networks.

Sensors, 22(20), 7759. <https://doi.org/10.3390/s22207759>

Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8446.html>

Semtech Corporation. (2020). *LoRa and LoRaWAN: A technical overview* (Application Note AN1200.56). Semtech. <https://www.semtech.com/uploads/documents/an1200.56.pdf>

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sisinni, E., Saifullah, A., Han, S., Jennings, K., & Vucic, M. (2018). Industrial Internet of Things: Challenges, opportunities, and directions. *IEEE Transactions on Industrial Informatics*, 14(11), 4724–4734. <https://doi.org/10.1109/TII.2018.2852491>

Slaný, V., Lučanský, A., Koudelka, P., et al. (2020). An integrated IoT architecture for smart metering using next generation sensor for water management based on LoRaWAN technology: A pilot study. *Sensors*, 20(17), 4712. <https://doi.org/10.3390/s20174712>

Naik, N. (2017). Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP. 2017 IEEE International Systems Engineering Symposium (ISSE), 1–7. <https://doi.org/10.1109/SysEng.2017.8088251>

Ta, Q. C., Vo, T. H., Nguyễn, T. T. H., & Nguyen, D. H. (2024). Research and application of LoRaWAN-based IoT system for water quality monitoring. *International Journal of Computer Science and Mobile Computing*, 13(12). <https://doi.org/10.47760/ijcsmc.2024.v13i12.001>

Lozano Muñoz, J., Navarro Camba, E. A., Gimenez Iborra, M., & Segura García, J. (2019). Análisis de la viabilidad de la tecnología LoRa en entornos urbanos. *Dyna Ingeniería e Industria*, 94(3), 302–308. <https://doi.org/10.6036/8755>

Velepucha, V., & Flores, P. (2021). Monoliths to microservices — Migration problems and challenges: A SMS. 2021 Second International Conference on Information Systems and Software Technologies (ICI2ST), 135–142. <https://doi.org/10.1109/ICI2ST51859.2021.00027>

Asamblea Nacional del Ecuador. (2021). *Ley Orgánica de Protección de Datos Personales*. Registro Oficial Suplemento 459, 26 de mayo de 2021. <https://www.telecomunicaciones.gob.ec/ley-organica-de-proteccion-de-datos-personales/>

Alshami, A. S., Aldoseri, A., Ali, H., Alhamad, A. Q., & Alhawati, M. (2024). Leveraging AI in IoT for smart water management: Challenges, opportunities, and future directions. *Water*, 16(5), 693. <https://doi.org/10.3390/w16050693>

Dai, H., Xu, Y., Zhang, G., & Zhou, T. (2025). Real-time water consumption monitoring and anomaly detection system based on IoT and machine learning. *Journal of Water Resources Planning and Management*, 151(2), 04024089. <https://doi.org/10.1061/JWRMD5.WRENG-6284>

Angular Team. (2024). *Angular documentation*. Google LLC. <https://angular.dev/overview>

Hajian, A. (2019). *Angular for enterprise-ready web applications*. Packt Publishing.

PrimeFaces. (2024). *PrimeNG: Angular UI component library* (v17). PrimeFaces.org. <https://primeng.org/>

Pimentel, V., & Nickerson, B. G. (2012). Communicating and displaying real-time data with WebSockets. *IEEE Internet Computing*, 16(4), 45–53. <https://doi.org/10.1109/MIC.2012.64>

Zambrano, M., Pérez, I., Palau, C., & Esteve, M. (2022). An efficient LoRaWAN network topology for monitoring large urban areas: A case study. *Sensors*, 22(7), 2398. <https://doi.org/10.3390/s22072398>

Apéndice A

Guía de entrevista semiestructurada

La siguiente guía de entrevista fue utilizada durante el levantamiento de requerimientos con el personal técnico de Representaciones Hidrocentro Cía. Ltda. El instrumento fue aplicado en sesiones de una hora promedio con el director técnico y el responsable de instalaciones de la empresa.

Datos generales

- **Nombre del entrevistado:** _____
- **Cargo:** _____
- **Fecha:** _____
- **Entrevistadores:** Carlos Joao IpiALES Ulcuango / Richard Steve Alban Vargas

Bloque 1: Contexto operativo actual

1. ¿Cuántos medidores LoRaWAN se encuentran actualmente instalados y en operación? ¿De qué marcas y modelos?
2. ¿Cuántos gateways LoRaWAN forman parte de la infraestructura de la empresa? ¿En qué zonas geográficas se encuentran desplegados?
3. ¿Qué plataforma o herramienta utiliza actualmente el personal técnico para consultar el estado de los medidores y las lecturas de consumo?
4. ¿Cuáles son las principales dificultades que experimenta el personal al usar las herramientas actuales de gestión?
5. ¿Con qué frecuencia se realizan lecturas manuales de medidores? ¿Existen rutas de lectura establecidas?

Bloque 2: Requerimientos funcionales

6. ¿Qué tipos de alarmas son más críticos para las operaciones de la empresa? (fuga, flujo inverso, batería baja, tubería vacía, otros)
7. ¿Cómo debería informarse al operador cuando se activa una alarma en un medidor? (notificación en pantalla, correo, otro)

8. ¿Qué información del historial de consumo es necesaria para la facturación? ¿En qué granularidad temporal (diaria, mensual)?
9. ¿La empresa necesita generar reportes de consumo para sus clientes? ¿En qué formato (Excel, PDF)?
10. ¿Se requiere que el sistema permita controlar remotamente la válvula de los medidores compatibles? ¿Con qué estados (abierto, cerrado, parcial)?
11. ¿La plataforma debe manejar múltiples empresas cliente, o es para uso exclusivo de Hidrocentro?
12. ¿Se necesita gestionar diferentes niveles de acceso para distintos tipos de usuarios (administrador, operador, consultor)?

Bloque 3: Requerimientos técnicos y de seguridad

13. ¿Existen restricciones en cuanto al servidor de red LoRaWAN a utilizar? ¿Se continuará con Lorient?
14. ¿La plataforma debe alojarse en infraestructura propia o en la nube? ¿Con qué presupuesto de infraestructura se cuenta?
15. ¿Qué nivel de disponibilidad se espera del sistema? ¿Existen ventanas de mantenimiento aceptables?
16. ¿Existe alguna normativa interna o exigencia contractual respecto al cifrado de los datos de consumo transmitidos?
17. ¿Los datos de los clientes (personas naturales) se vinculan a las lecturas de consumo? ¿Cómo se trata la privacidad de esa información?
18. ¿Se requiere integración con otros sistemas existentes (facturación, ERP, sistema de clientes)?

Bloque 4: Priorización y validación

19. De las funcionalidades mencionadas, ¿cuáles tres considera de mayor prioridad para la operación diaria de la empresa?
20. ¿Cómo prevé que el personal técnico adoptará el nuevo sistema? ¿Se requiere capacitación formal?
21. ¿Existe algún prototipo previo o sistema anterior cuya experiencia deba tomarse en cuenta en el diseño?

Observaciones adicionales del entrevistador

Apéndice B

Fragmentos de código clave

Este apéndice presenta los fragmentos de código más representativos de la implementación, seleccionados por su relevancia para comprender la lógica de integración y decodificación del sistema.

A.1 Adaptador LoRaWAN: ciclo de lectura TLS

El siguiente fragmento corresponde al método `readLoop()` del servicio `WebSocketService`, que implementa el ciclo de lectura continua del canal TLS sobre TCP. El hilo dedicado lee líneas del flujo de entrada mientras la conexión permanezca activa y, ante un error de E/S o cierre del socket, programa una reconexión automática.

```
private void readLoop() {
    try (BufferedReader reader = new BufferedReader(
        new InputStreamReader(socket.getInputStream()))) {
        String line;
        while (running.get() && (line = reader.readLine()) != null) {
            handleMessage(line);
        }
    } catch (IOException e) {
        if (running.get()) {
            logger.error("TLS socket read error: {}", e.getMessage());
        }
    }
    if (running.get()) {
        logger.warn("TLS connection closed, scheduling reconnect...");
        scheduleReconnect();
    }
}
```

A.2 Establecimiento de conexión TLS con autenticación

El método `connect()` establece el socket TLS usando `SSLSocketFactory` estándar del JDK, envía el token de autenticación y lanza el hilo de lectura como demonio.

```

public void connect() {
    closeCurrentConnection();
    try {
        SSLSocketFactory factory =
            (SSLSocketFactory) SSLSocketFactory.getDefault();
        socket = (SSLSocket) factory.createSocket(host, port);
        writer = new PrintWriter(socket.getOutputStream(), true);

        writer.println("{\"token\":\"" + token + "\"}");
        logger.info("TLS socket connected to {}:{}", host, port);

        readerThread = new Thread(this::readLoop, "loriot-tls-reader");
        readerThread.setDaemon(true);
        readerThread.start();

    } catch (IOException e) {
        logger.error("Failed to connect: {}", e.getMessage());
        scheduleReconnect();
    }
}

```

A.3 Decodificador de payload Bove

El siguiente pseudocódigo ilustra la lógica de extracción de campos del payload Bove de 13 bytes. El volumen acumulado se lee como entero de 32 bits en formato little-endian desde los bytes 6 a 9, y los estados de alarma se extraen de los bytes 11 y 12 mediante operaciones de máscara de bits.

```

byte[] data = hexToBytes(payloadHex); // 13 bytes totales
int rawVolume = (data[6] & 0xFF)
    | ((data[7] & 0xFF) << 8)
    | ((data[8] & 0xFF) << 16)
    | ((data[9] & 0xFF) << 24); // little-endian UINT32

double volume = rawVolume * scaleFactor; // en m³

int st1 = data[11] & 0xFF;
boolean batteryLow  = (st1 & 0x01) != 0;
boolean emptyPipe   = (st1 & 0x02) != 0;
boolean backflow    = (st1 & 0x04) != 0;
boolean overRange   = (st1 & 0x08) != 0;

int st2 = data[12] & 0xFF;
boolean overTemp    = (st2 & 0x01) != 0;
boolean eepromError = (st2 & 0x02) != 0;
boolean leak        = (st2 & 0x04) != 0;
boolean burst       = (st2 & 0x08) != 0;

```


A.4 Cadena de agregación de consumo (pseudocódigo)

El microservicio de consumo calcula la diferencia entre lecturas consecutivas y actualiza las tablas de agregación en cascada dentro de una única transacción PostgreSQL, garantizando consistencia ACID.

```
@Transactional
public void processUplink(String devEui, double newReading,
                        LocalDateTime timestamp) {
    // Obtener última lectura registrada
    Optional<Reading> prev = readingRepo.findLatestByDevEui(devEui);

    if (prev.isPresent()) {
        double delta = newReading - prev.get().getValue();

        if (delta >= 0) {
            // Actualizar consumo horario
            HourlyConsumption hourly = hourlyRepo
                .findOrCreate(devEui, timestamp.truncatedTo(HOURS));
            hourly.addConsumption(delta);
            hourlyRepo.save(hourly);

            // Propagar a diario
            DailyConsumption daily = dailyRepo
                .findOrCreate(devEui, timestamp.toLocalDate());
            daily.addConsumption(delta);
            dailyRepo.save(daily);

            // Propagar a mensual y anual
            // ... (mismo patrón)
        }
    }
    readingRepo.save(new Reading(devEui, newReading, timestamp));
}
```

Apéndice C

Guía de despliegue local del prototipo

Este apéndice describe el procedimiento para levantar todos los componentes de la plataforma Athena en un entorno local de desarrollo. El prototipo requiere Java 21, Maven, Node.js 18+, Angular CLI 17 y PostgreSQL 15 instalados en la máquina.

C.1 Preparación de la base de datos

Crear la base de datos en la instancia local de PostgreSQL:

```
psql -U postgres -c "CREATE DATABASE m3verificaciones;"
```

No es necesario ejecutar scripts de creación de tablas: cada microservicio aplica automáticamente el esquema al iniciar gracias a la configuración `spring.jpa.hibernate.ddl-auto=update` definida en `application.properties`.

C.2 Configuración de cada microservicio

El archivo `src/main/resources/application.properties` de cada servicio define los valores por defecto para el entorno local mediante la sintaxis `${VAR:valor}`. Para la base de datos local los valores defecto son:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/m3verificaciones
spring.datasource.username=postgres
spring.datasource.password=admin
```

Si las credenciales locales difieren, establecer las variables de entorno `SPRING_DATASOURCE_USERNAME` y `SPRING_DATASOURCE_PASSWORD` antes de iniciar el servicio.

C.3 Inicio de los microservicios backend

Cada servicio se inicia desde su directorio raíz con el siguiente comando (Windows):

```
mvnw.cmd spring-boot:run
```

La Tabla C.1 indica el orden recomendado de inicio y el puerto asignado a cada servicio.

Tabla C.1: Orden de inicio de los microservicios en entorno local

Orden	Directorio del servicio	Puerto	Verificación
1	iot-platform-user-backend	8083	GET /actuator/health
2	iot-platform-company-backend	8081	GET /actuator/health
3	iot-platform-meters-backend	8082	GET /actuator/health
4	iot-platform-gateway-backend	8080	GET /actuator/health
5	iot-platform-userCompany-backend	8084	GET /actuator/health
6	iot-platform-messages-backend	8085	GET /actuator/health
7	iot-platform-consumption-backend	8086	GET /actuator/health
8	iot-platform-lorawan-backend	8087	Conexión TLS a Lorient en logs

Se recomienda iniciar el servicio de usuario primero porque los demás servicios dependen de la misma clave JWT para validar tokens. El adaptador LoRaWAN debe iniciarse al final para que los microservicios receptores (Messages, Consumption, Meters) estén disponibles antes de que lleguen los primeros uplinks.

C.4 Inicio del frontend

Desde el directorio `iot-platform-frontend`, instalar las dependencias e iniciar el servidor de desarrollo:

```
npm install
ng serve
```

La aplicación queda disponible en `http://localhost:4200`. Los archivos `src/environments/environment.ts` ya apuntan a `http://localhost:808x` para cada microservicio en el entorno de desarrollo.

Apéndice D

Políticas de Seguridad Operativa (resumen)

Este apéndice resume cinco controles operativos mínimos recomendados para la plataforma. Cada control incluye práctica sugerida y evidencia esperada. Estas políticas complementan los referenciales presentados en el Marco teórico y pueden evolucionar en despliegues productivos.

D.1 Gestión de secretos

- **Práctica:** Variables sensibles (claves JWT, credenciales DB, tokens Lorient) se almacenan fuera del repositorio (variables de entorno/secret store). Prohibida su inclusión en código/ficheros `.env` versionados.
- **Evidencia:** Ausencia de secretos en el VCS; plantilla de variables; acceso controlado al secret store.

D.2 Rotación de claves JWT

- **Práctica:** Claves de firma con vida limitada; proceso de rotación documentado (key rollover) manteniendo ventana de validación dual (old/new) por periodo corto.
- **Evidencia:** Registro de rotaciones; configuración de expiración corta (`exp`); lista de revocación para cierre de sesión forzado.

D.3 Registro y retención de eventos

- **Práctica:** Logs estructurados (nivel INFO/WARN/ERROR) sin datos sensibles; correlación por `traceId`; retención mínima de 30 días en entorno dev y acorde a políticas en prod.
- **Evidencia:** Política de retención; ejemplos de trazas de errores con `traceId`; filtros de PII aplicados.

D.4 Respaldo y recuperación

- **Práctica:** Backups periódicos de base de datos y artefactos de configuración; prueba de restauración (DR drill) al menos semestral.

- **Evidencia:** Cronograma de backups; reporte de prueba de restauración exitosa; checksum de copias.

D.5 Cabeceras seguras (checklist)

- **Práctica:** Configurar cabeceras HTTP en el gateway/backend: `Content-Security-Policy`, `X-Content-Type-Options=nosniff`, `X-Frame-Options=DENY`, `Referrer-Policy=no-referrer`, `Strict-Transport-Security` en HTTPS.
- **Evidencia:** Respuesta de ejemplo con cabeceras activas; pruebas de navegador/escáner OWASP que validen su presencia.